THE STUDY CAFÉ · OPEN EVERY DAY

DSA Placement Playbook

144 days · 443 problems · one cup at a time



Day's cup · 0 / 4 blocks

OPENING HOURS · 5 NOV 2026 → 28 MAR 2027

Today

Friday, 13 November 2026

◀ 11/13/2026 ▶ Today

9 / 144	0 / 4	0	0
café day	blocks done today	day streak	full days logged

Dish Washer
RANK 1 OF 9

0 XP

438 XP to Bean Sorter · Easy 10 · Medium 20 · Hard 40 · AWS item 20 · block 5

DSA — Mark done

Step 4: Binary search

- [ ] Binary search to find X E +10
- [ ] Implement lower bound E +10
- [ ] Implement upper bound E +10
- [ ] Search insert position E +10
- [ ] Floor and ceil in a sorted array E +10
- [ ] First and last occurrence E +10
- [ ] Count occurrences in a sorted array E +10
- [ ] Search in a rotated sorted array I M +20

0 / 8 ticked today · 0 of 443 overall

OVERDUE · OLDEST FIRST — 61 LEFT

Problems from earlier days that are not ticked yet. Clear these first, about 4 extra a day, until this card disappears.

- [ ] Step 1: Things to know in C++: input/output, data types, if/else, switch, loops E +10
- [ ] Step 1: Functions: pass by value vs reference, pointers basics E +10
- [ ] Step 1: Arrays and strings in C++ E +10
- [ ] Step 1: Patterns 1 to 11 (squares, triangles, number triangles) E +10
- [ ] Step 1: Patterns 12 to 17 (letter and mirrored patterns) E +10

- [ ] Step 1: Patterns 18 to 22 (hollow, diamond, butterfly) E +10

LEETCODE

Mark done

- T **#704** Binary Search (Easy)
- T **#35** Search Insert Position (Easy)
- R **#128** Longest Consecutive Sequence (Medium)

Revise first: #217, #283, #912, #11, #560, #147

CORE CS · 45 MIN

Mark done

Object-oriented programming (C++)

Classes and encapsulation

- Classes and objects, members, this
- Access specifiers: public, private, protected
- Constructors: default, parameterised, copy; initialiser lists
- Destructors and object lifetime; shallow vs deep copy
- Static members and functions; const member functions

CYBER · 1 H

Mark done

Week 1: Linux command line

TryHackMe Linux Fundamentals parts 1 to 3; OverTheWire Bandit levels 0 to 5

Proof of work: A notes file of 30 commands, each with one example

Full week plan

AWS · CLOUD

DAILY FROM 16 NOV

Getting started

Tick each item when it is done:

- [ ] On any personal AWS account: billing alarm and budget before the first lab +20
- [ ] Module 1: Welcome to the course +20

0 / 35 AWS items

▼ All 35 items

AWS Academy Cloud Architecting

- [ ] Module 1: Welcome to the course +20
- [ ] Module 2: Introducing cloud architecting +20
- [ ] Module 3: Adding a storage layer (S3) +20
- [ ] Module 4: Adding a compute layer (EC2) +20
- [ ] Module 5: Adding a database layer (RDS, DynamoDB) +20
- [ ] Module 6: Creating a networking environment (VPC) +20

- [ ] Module 7: Connecting networks +20
- [ ] Module 8: Securing user and application access (IAM) +20
- [ ] Module 9: Elasticity, high availability and monitoring +20
- [ ] Module 10: Automating your architecture (CloudFormation) +20
- [ ] Module 11: Caching content (CloudFront, ElastiCache) +20
- [ ] Module 12: Building decoupled architectures (SQS, SNS) +20
- [ ] Module 13: Microservices and serverless (Lambda, API Gateway) +20
- [ ] Module 14: Planning for disaster +20
- [ ] Module 15: Bridging to certification, plus its practice exam +20

Labs worth redoing from memory

- [ ] VPC with public and private subnets, route tables and a NAT gateway +20
- [ ] EC2 behind an Application Load Balancer with an Auto Scaling group +20
- [ ] RDS in a private subnet, reachable only from the app tier +20
- [ ] S3 bucket policy, versioning and lifecycle rules, served through CloudFront +20
- [ ] IAM role for EC2 with a least-privilege policy +20
- [ ] A CloudFormation template that rebuilds one lab from scratch +20
- [ ] SQS plus Lambda decoupled pipeline +20
- [ ] On any personal AWS account: billing alarm and budget before the first lab +20

Solutions Architect Associate (SAA-C03)

- [ ] Read the official exam guide and its four domains +20
- [ ] Design secure architectures (30%): IAM, KMS, VPC security, S3 security +20
- [ ] Design resilient architectures (26%): multi-AZ, Route 53, SQS, backups, DR +20
- [ ] Design high-performing architectures (24%): caching, EBS types, scaling, databases +20
- [ ] Design cost-optimised architectures (20%): pricing models, storage classes, right-sizing +20
- [ ] Two full timed practice exams scored above 80% +20
- [ ] Ask your faculty about an AWS Academy exam discount before booking +20
- [ ] Book and sit the exam +20

Tie it to your profile

- [ ] Write TenantWall up with an architecture diagram and the IAM and ABAC choices explained +20
- [ ] Publish one AWS project on GitHub with a diagram and its CloudFormation template +20
- [ ] Resume line: what you built on AWS, which services, one real number +20
- [ ] Map AWS security services to your cyber skills: CloudTrail, GuardDuty, Security Hub, Config, WAF +20

▼ BADGES 0 / 16



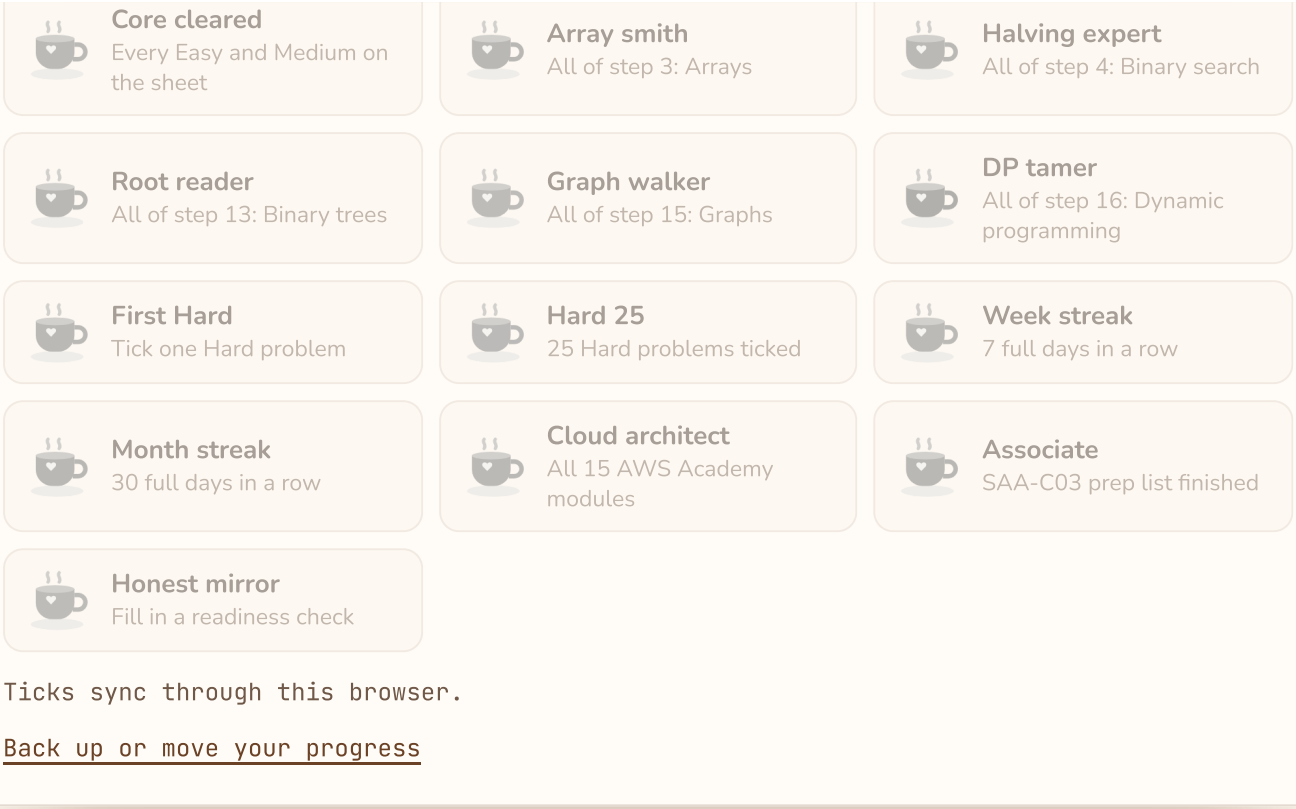
First solve
Tick your first problem



Warmed up
25 problems ticked



Century
100 problems ticked



C++ in two days, Striver's A2Z sheet built practically, a LeetCode month, five core CS subjects, a one-hour-a-day cyber track, 30 minutes of AWS every day, CP31 from mid-December, a two-hour AI track from January, a later maths and probability track for HFT roles, skill maps for SDE and security roles, and an off-campus internship guide.

Core problems	Hard pass	LeetCode	Builds	CS subjects	Cyber weeks
359	84	75	18	5	20

Index

13 parts

00

The plan

Today
Five tracks, one route
The daily shape
AWS every day
The big picture
Mac setup

01

C++ in two days

Day 1: Setup and your first program
Day 1: Types, operators and overflow
Day 1: Control flow, loops and patterns
Day 1: Functions and references
Day 1: Arrays, vectors and strings
Day 2: STL I: pair, sort, lambdas and algorithms
Day 2: STL II: map, set and their hash versions
Day 2: STL III: stack, queue, deque, priority_queue
+ 6 more

02

Striver A2Z, built

The 6-step loop

1. Learn the basics
2. Sorting techniques
3. Arrays
4. Binary search
5. Strings (basic and medium)
6. Linked list
7. Recursion and backtracking

+ 12 more

03

LeetCode month

Rules and contests

The 30-day plan

CP31 sheet

04

Core CS

How the 45 minutes work

Object-oriented programming (C++)

Operating systems

DBMS and SQL

Computer networks

System design basics

05

Cyber track

The hour

20-week lab plan

Setup, commands, OWASP

06

Career skill maps

Which role, which skills

Software Development Engineer (SDE)

Network Security Engineer

Software Security Engineer (Application Security)

Secure code review drill

07

Off-campus internships

Programs open to you

Real selection stories

What every selected intern had

Where and how to apply

Collect RAIT alumni stories

08

Interview kit

Constraints and signals

How to talk and debug

09

Logs and reviews

Daily checklist

Back up progress

10

AI track

The AI hour

The 12-week plan

What you will have by March

11

HFT track (later)

What HFT firms test

The 16-week plan

Topics to cover

12

TenantWall research

The four Saturdays

The requests, explained

After December

▼ 00 The plan

1. Today
2. Five tracks, one route
3. The daily shape
4. AWS every day
5. The big picture
6. Mac setup

▼ 01 C++ in two days

1. Day 1: Setup and your first program
2. Day 1: Types, operators and overflow
3. Day 1: Control flow, loops and patterns
4. Day 1: Functions and references
5. Day 1: Arrays, vectors and strings
6. Day 2: STL I: pair, sort, lambdas and algorithms
7. Day 2: STL II: map, set and their hash versions
8. Day 2: STL III: stack, queue, deque, priority_queue
9. Day 2: Recursion, pointers and structs
10. Day 2: Complexity and hashing
11. Complexity quiz
12. STL cheat sheet
13. Practice set
14. Gate test

▼ 02 Striver A2Z, built

1. The 6-step loop
2. 1. Learn the basics
3. 2. Sorting techniques
4. 3. Arrays
5. 4. Binary search
6. 5. Strings (basic and medium)
7. 6. Linked list
8. 7. Recursion and backtracking
9. 8. Bit manipulation
10. 9. Stacks and queues
11. 10. Sliding window and two pointers
12. 11. Heaps
13. 12. Greedy algorithms
14. 13. Binary trees
15. 14. Binary search trees
16. 15. Graphs

17. 16. Dynamic programming
18. 17. Tries
19. 18. Strings (advanced)
20. Hard pass

▼ 03 LeetCode month

1. Rules and contests
2. The 30-day plan
3. CP31 sheet

▼ 04 Core CS

1. How the 45 minutes work
2. Object-oriented programming (C++)
3. Operating systems
4. DBMS and SQL
5. Computer networks
6. System design basics

▼ 05 Cyber track

1. The hour
2. 20-week lab plan
3. Setup, commands, OWASP

▼ 06 Career skill maps

1. Which role, which skills
2. Software Development Engineer (SDE)
3. Network Security Engineer
4. Software Security Engineer (Application Security)
5. Secure code review drill

▼ 07 Off-campus internships

1. Programs open to you
2. Real selection stories
3. What every selected intern had
4. Where and how to apply
5. Collect RAIT alumni stories

▼ 08 Interview kit

1. Constraints and signals
2. How to talk and debug

▼ 09 Logs and reviews

1. Daily checklist
2. Back up progress

▼ 10 AI track

1. The AI hour
2. The 12-week plan
3. What you will have by March

▼ 11 HFT track (later)

1. What HFT firms test
2. The 16-week plan
3. Topics to cover

▼ 12 TenantWall research

1. The four Saturdays
2. The requests, explained
3. After December

PART 00

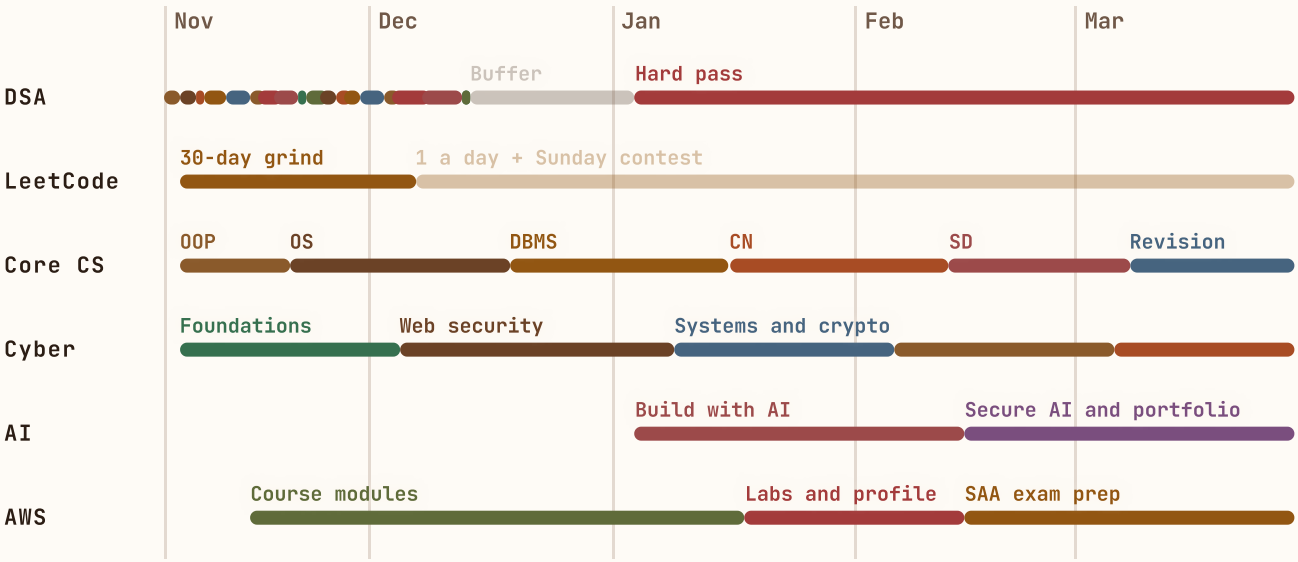
00 The plan

The route, how a day runs, and your Mac set up.

PART 00 | THE ROUTE

Five tracks, one route

Each line is a track from 5 Nov 2026 to 28 Mar 2027. Hover a segment to see its name.



TRACK	DATES	DAILY TIME	DONE WHEN
C++ sprint	5 to 6 Nov	9 h (both days)	Gate test passed: 4 of 5
Striver A2Z core	7 Nov to 13 Dec	2.5 h + 30 min build	359 problems and 18 builds

TRACK	DATES	DAILY TIME	DONE WHEN
LeetCode month	7 Nov to 6 Dec	2 h	75 problems, 5 contests
Core CS	7 Nov to 28 Mar	45 min	5 subjects, 60 answers practised
Cyber	7 Nov to 28 Mar	1 h (2 h Sundays)	20 weekly proofs of work
AI	4 Jan to 28 Mar	2 h	12 weekly proofs of work
AWS	16 Nov to 28 Mar	30 min	Course, labs and exam prep done; SAA-C03 booked for April
CP31	14 Dec to 28 Mar	1 h	Ratings 800 to 1400 done
TenantWall	Saturdays, 21 Nov to 12 Dec	3 h	Professor's changes done; paper submitted by March
Hard pass	4 Jan to 28 Mar	1 to 1.5 h	84 Hard problems attempted

PART 00 | A DAY

The daily shape from 7 Nov

BLOCK	TIME	WHAT
Revision	30 min	Re-solve problems due today (day +1, +3, +7). Always first.
Striver	2.5 h	Today's problems from the step, by due date.
Build	30 min	One feature of this step's tool, then commit.
LeetCode	2 h	2 topic + 1 recall problem from the day table.
Core CS	45 min	This week's topic, 3 flashcards, one answer out loud.
Cyber	1 h (2 h Sun)	One lab with a timer. Stop at 60 minutes, notes written.
TenantWall	3 h, Saturdays	21 Nov to 12 Dec, in LeetCode's slot. The four Saturdays.
AWS	30 min	This week's module, lab or exam topic, from 16 Nov. The 19-week plan.
Log	15 min	One line per problem; tick the tracker.

TOTAL: ABOUT 8 HOURS

On college days use the split in Part 09. The minimum day: revision, 2 Striver problems, 1 LeetCode problem, 20 minutes of Core CS flashcards. Cyber is the first thing to drop that day, never revision.

Three rules

1. **Think before you look.** 30 minutes alone before a hint, 60 before a solution. Rewrite it from memory the next day.
2. **Run everything.** Every problem compiled, run, and tested on two cases you invent.
3. **Log everything.** The log decides what you revise.

PART 00 | AWS, 30 MINUTES A DAY FROM 16 NOV

AWS every day

Thirty minutes a day takes you through the AWS Academy Cloud Architecting course, the labs worth redoing, and Solutions Architect Associate (SAA-C03) prep, with the exam booked for April. IAM and monitoring come first because TenantWall needs them now. Tick each course item in the AWS card on Today (+20 XP), and mark the block done (+5 XP).

WEEK	INSIDE THE 30 MINUTES
W01 16 Nov to 22 Nov	Modules 1, 2 and 8 (IAM): users, groups, roles, policies, explicit vs implicit deny. Set a billing alarm and budget first. Feeds TenantWall's Policy Generator work.
W02 23 Nov to 29 Nov	Module 9: elasticity, high availability and monitoring. CloudWatch logs, metrics and alarms; CloudTrail. Feeds TenantWall's trail and alarm.
W03 30 Nov to 06 Dec	Module 3 (S3): bucket policies, encryption, versioning, lifecycle, Block Public Access.
W04 07 Dec to 13 Dec	Module 4 (EC2 and EBS): instance types, AMIs, security groups. Start module 5 (RDS, DynamoDB).
W05 14 Dec to 20 Dec	Finish module 5, then module 6 (VPC): subnets, route tables, internet and NAT gateways.

WEEK	INSIDE THE 30 MINUTES
W06 21 Dec to 27 Dec	Module 7 (connecting networks) and module 10 (CloudFormation).
W07 28 Dec to 03 Jan	Modules 11 and 12: caching (CloudFront, ElastiCache) and decoupling (SQS, SNS).
W08 04 Jan to 10 Jan	Modules 13 and 14: serverless (Lambda, API Gateway) and planning for disaster.
W09 11 Jan to 17 Jan	Module 15 and its practice exam.
W10 18 Jan to 24 Jan	Labs from memory: a VPC with public and private subnets and a NAT gateway; an IAM role for EC2.
W11 25 Jan to 31 Jan	Labs from memory: EC2 behind a load balancer with Auto Scaling; RDS in a private subnet.
W12 01 Feb to 07 Feb	Labs from memory: S3 with CloudFront; SQS plus Lambda; a CloudFormation template that rebuilds one lab.
W13 08 Feb to 14 Feb	Tie it to your profile: TenantWall write-up with an AWS-style diagram, one AWS project on GitHub, a resume line.
W14 15 Feb to 21 Feb	Exam prep: read the SAA-C03 exam guide; start the secure-architectures domain (30%). 10 practice questions a day from now on.
W15 22 Feb to 28 Feb	Secure architectures, continued: KMS, VPC security, S3 security. Map AWS security services to your cyber skills.
W16 01 Mar to 07 Mar	Resilient architectures (26%): multi-AZ, Route 53, SQS, backups, disaster recovery.
W17 08 Mar to 14 Mar	High-performing architectures (24%): caching, EBS types, scaling, databases.

WEEK	INSIDE THE 30 MINUTES
W18 15 Mar to 21 Mar	Cost-optimised architectures (20%): pricing models, storage classes, right-sizing. First full timed practice exam.
W19 22 Mar to 28 Mar	Second timed practice exam (aim for 80%+). Ask your faculty about an AWS Academy exam discount and book the exam for April.

> KEEP IT CHEAP
>
> Set the billing alarm in week 1, use free-tier sizes, and delete what you build at the end of each lab.

PART 00 | THE BIG PICTURE

Where this plan takes you

Positioning: an SDE with cloud-security depth. Main target: SDE at product companies in final placements (Aug to Dec 2027). Strong second: cloud, DevSecOps or AppSec roles, using TenantWall, AWS and the cyber track. Stretch: SDE at an HFT firm.

WHEN	MAIN FOCUS	DAILY
16 to 17 Nov	C++ and STL, gate test	Full days
18 Nov to 13 Dec	**DSA heavy**: Striver with the Overdue card, LeetCode month · OOP then OS · cyber · AWS · TenantWall on Saturdays	About 8.5 h
14 Dec to 3 Jan	Buffer: leftovers, builds, missed LeetCode · **CP31 starts** · AWS · cyber · resume v1	About 7 h
4 Jan to 28 Mar	**AI 2 h** · Hard pass · LeetCode 1 a day · CP31 · cyber · Core CS · AWS exam prep · paper submitted	About 8 h
Apr to May 2027	AWS Solutions Architect exam · one capstone project · HFT probability starts	—
Jun to Jul 2027	Placement sprint: revise Striver, timed tests, weekly mock interviews	—
Aug to Dec 2027	**Final placements**, with off-campus applications alongside	—

WHEN	MAIN FOCUS	DAILY
Jan to Jun 2028	Final-year project; keep applying if there is no offer yet	—

Checkpoints

- ☐ 17 Nov: gate test 4 of 5
- ☐ 13 Dec: Striver caught up or close · AWS weeks 1 to 4 · TenantWall changes done
- ☐ 20 Dec: readiness check 1
- ☐ 17 Jan: AWS course and its practice exam done
- ☐ 15 Mar: readiness check 2 · Hard pass about 70%
- ☐ 28 Mar: two AWS practice exams at 80%+ · AI capstone · paper submitted
- ☐ May 2027: AWS certification · Jul 2027: Codeforces 1400 to 1600

When time is short, cut in this order

1. HFT (later anyway)
2. CP31
3. AI down to 1 hour
4. Cyber down to Sundays only

Never cut DSA, revision or Core CS. On a bad day do the minimum day, never zero.

> THE BIGGEST RISK
>
> Not talent but restarts. The plan only works if you show up most days. Check your streak every night.

PART 00 | SETUP

Setting up your MacBook Air

Apple's built-in `g++` is really clang, which has no `bits/stdc++.h`. Install real GCC and a one-word runner (about 20 minutes).

● ● ● TERMINAL Copy

```
xcode-select --install
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
```

```
eval "$(/opt/homebrew/bin/brew shellenv)"
brew install gcc
ls /opt/homebrew/bin/g++-*        # note the version

xcode-select --
install
```

Paste into `~/.zshrc`, then run `source ~/.zshrc`:

~/.ZSHRC Copy

```
run() {
  local GXX=$(ls /opt/homebrew/bin/g++-* | head -n 1)
  "$GXX" -std=c++17 -O2 -Wall -Wextra "$1" -o /tmp/a.out && /tmp/a.out
}
rund() {   # debug build: stops on out-of-range vector access
  local GXX=$(ls /opt/homebrew/bin/g++-* | head -n 1)
  "$GXX" -std=c++17 -g -D_GLIBCXX_DEBUG -Wall -Wextra "$1" -o /tmp/a.out && /tmp/a.out
}

run() {
  local GXX=$(ls
```

YOU SEE	CAUSE AND FIX
`'bits/stdc++.h' file not found`	You used Apple's clang. Use `run`, or `g++-15` directly.
`zsh: command not found: brew`	Run the `echo` and `eval` lines again, open a new Terminal.
`zsh: command not found: run`	Run `source ~/.zshrc`.
The program seems frozen	It is waiting for input. Ctrl+D ends input.
`Segmentation fault`	Bad index, null pointer or infinite recursion. Rerun with `rund`.

PART 01

01

C++ in two days

Moved to Mon 16 and Tue 17 Nov. Ten lessons with code and the output you should see, 20 practice problems with solutions, and a gate test.

DAY 1 | BLOCK 1 | 9:00 to 10:00

Setup and your first program

> **Goal.** Compile and run a C++ file from Terminal in under 10 seconds, and read input in every shape a problem gives it.

Concepts

- Every program has the same skeleton: `#include` lines pull in libraries, `using namespace std;` lets you write `vector` instead of `std::vector`, and execution starts in `int main()`.
- `#include <bits/stdc++.h>` pulls in the whole standard library at once. It only exists in GCC, which is why you installed GCC. It is normal in DSA and contests, not in production code.
- `cin >> x` skips spaces and newlines, then reads one value. `cout << x` prints. Chain them: `cin >> a >> b;`.
- Use `'\n'` for a newline, not `endl`. `endl` also flushes the output buffer every time, which is slow when you print a million lines.
- The two fast-I/O lines (`ios::sync_with_stdio(false); cin.tie(nullptr);`) unhook C++ streams from C's `printf`/`scanf` and stop `cin` from flushing `cout` before each read. After them, do not mix `printf` and `cout`.
- Compile flags: `-std=c++17` picks the language version, `-O2` optimises like judges do, `-Wall -Wextra` warn you about likely bugs. Read every warning.

Type these out

HELLO.CPP

Copy

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int a, b;
    cin >> a >> b;
    cout << a + b << '\n';
    return 0;
}
```

```
#include
<bits/stdc++.h>
```

OUTPUT

```
input:  3 4
output: 7
```

READING EVERY INPUT SHAPE

Copy

```cpp
int n;  cin >> n;                       // one number
vector<int> a(n);  for (int &x : a) cin >> x;   // n numbers

string word;  cin >> word;              // one word (stops at a space)
string line;  getline(cin, line);       // a whole line with spaces

cin >> n;  cin.ignore();  getline(cin, line);   // number, THEN a line: ignore the leftover '\n'

int x, cnt = 0;  long long sum = 0;
while (cin >> x) { cnt++; sum += x; }   // read until end of input (Ctrl+D in Terminal)

cout << fixed << setprecision(2) << 1.0 * sum / cnt << '\n';   // 2 decimal places
```

```
int n;  cin >> n;
// one number
```

TERMINAL

Copy

```
run a.cpp                   # compile and run, type input, Ctrl+D to end
run a.cpp < in.txt          # read input from a file (save samples in in.txt)
run a.cpp < in.txt > out.txt # and write output to a file
```

```
run a.cpp
# compile and run,
```

Traps

- ⚠ Forgetting `cin.ignore()` between `cin >> n` and `getline`: the line comes back empty.
- ⚠ Printing extra spaces or missing the final newline usually passes on LeetCode but can fail strict online-test judges. Match the sample exactly.
- ⚠ Mixing `printf` with `cout` after the fast-I/O lines gives output in the wrong order.

Drill

1. **Greeting.** Read a name (one word) and an age. Print `Hello <name>, next year you will be <age+1>`.
2. **Sum of n.** Read n, then n integers. Print their sum. Use `long long`.
3. **Average to EOF.** Read integers until input ends. Print the count and the average with exactly 2 decimals.

Check yourself

- [ ] Why is `'\n'` faster than `endl`?
- [ ] What goes wrong if you read a number and then call `getline` straight away?
- [ ] What does `-Wall` do and why keep it on?

DAY 1 | BLOCK 2 | 10:15 to 11:45

Types, operators and overflow

> **Goal.** Pick the right type every time and never lose marks to overflow or integer division.

Concepts

- Sizes you must know: `int` is 4 bytes, about plus or minus 2.1e9. `long long` is 8 bytes, about plus or minus 9.2e18. `double` keeps about 15 significant digits. `char` is 1 byte. `bool` is true or false.
- Rule of thumb: if a value can pass 2e9 (sums of big arrays, products, counts of pairs), use `long long`.
- Overflow happens in the expression, before the assignment. `long long x = a * b;` with `int a, b` still overflows. Write `1LL * a * b`.
- Integer division truncates toward zero: `7/2` is 3 and `-7/2` is -3. The `%` result takes the sign of the left side: `-7 % 3` is -1.
- To get a real answer, make one side a double: `1.0 * a / b`.
- Modular arithmetic: `const int MOD = 1e9 + 7;` then take `% MOD` after every add and multiply, with the product in `long long`.
- Characters are small integers. `c - '0'` turns `'7'` into 7. `c - 'a'` gives 0 to 25, the index you use for letter counts.
- Limits live in `INT_MAX`, `INT_MIN`, `LLONG_MAX`, `LLONG_MIN`.

Type these out

OVERFLOW AND DIVISION

Copy

```
int a = 1e9, b = 3;
long long bad  = a * b;          // overflow: computed in int first
long long good = 1LL * a * b;    // 3000000000
cout << good << '\n';

cout << 7 / 2 << ' ' << -7 / 2 << ' ' << -7 % 3 << '\n';
cout << fixed << setprecision(3) << 7.0 / 2 << '\n';

char c = '7';
```

```cpp
int d = c - '0';
cout << d + 1 << ' ' << char('a' + 2) << '\n';

const int MOD = 1e9 + 7;
long long p = 1;
for (int i = 0; i < 100; i++) p = p * 2 % MOD;   // 2^100 mod 1e9+7
cout << p << '\n';

int a = 1e9, b = 3;
long long bad  = a *
```

▶ OUTPUT

```
3000000000
3 -3 -1
3.500
8 c
976371285
```

Traps

- ⚠ Comparing doubles with `==`. Use `fabs(a - b) < 1e-9`.
- ⚠ `sqrt(n)` returns a double. For a perfect-square check use `long long r = sqrtl(n); while (r*r > n) r--; while ((r+1)*(r+1) <= n) r++;`.
- ⚠ `1 << 40` overflows an int. Write `1LL << 40`.
- ⚠ `a % MOD` can be negative if `a` is negative. Use `((a % MOD) + MOD) % MOD`.

Drill

1. **Where int breaks.** Print n! for n = 1 to 20 twice, once with `int` and once with `long long`. Find the first n where the int version goes wrong.
2. **Digit sum.** Read n (up to 1e18) and print the sum of its digits.
3. **Fast enough power.** Read a and b (b up to 1e6). Print a^b mod 1e9+7 with a plain loop.
4. **Temperature.** Read Celsius as a double, print Fahrenheit with 2 decimals.

Check yourself

- ☐ Why does `long long x = a * b;` still overflow when a and b are ints?
- ☐ What is `-7 % 3` in C++?
- ☐ How do you turn the character `'5'` into the number 5?

DAY 1 | BLOCK 3 | 12:00 to 1:30

Control flow, loops and patterns

> **Goal.** Turn any shape or repetition into nested loops, and know exactly how many times each loop runs.

Concepts

- `if / else if / else` checks in order and runs the first true branch. `switch` jumps on an int or char value; every `case` needs a `break` or it falls into the next one.
- The ternary `cond ? a : b` is an if/else that returns a value: `int mx = (a > b) ? a : b;`.
- `for (init; condition; step)` when you know the count, `while (condition)` when you do not, `do { } while (condition);` when the body must run at least once.
- `break` leaves the innermost loop. `continue` skips to its next iteration.
- Patterns method: the outer loop is the row, the inner loops print what each row contains. Write a table of row number vs spaces vs stars before you code. For a pyramid of n rows, row i has `n - i` spaces and `2i - 1` stars.
- Count the work: a loop from 0 to n runs n times. Two nested loops to n run n*n times.

Type these out

PYRAMID AND NUMBER TRIANGLE

Copy

```cpp
int n = 4;
for (int i = 1; i ⩽ n; i++) {                    // pyramid
    for (int s = 0; s < n - i; s++) cout << ' ';
    for (int k = 0; k < 2 * i - 1; k++) cout << '*';
    cout << '\n';
}
for (int i = 1; i ⩽ n; i++) {                    // number triangle
    for (int j = 1; j ⩽ i; j++) cout << j;
    cout << '\n';
}
```

```
int n = 4;
for (int i = 1; i <=
```

▶ OUTPUT

```
   *
  ***
 *****
*******
1
12
123
1234
```

SWITCH WITH FALLTHROUGH ON PURPOSE

Copy

```cpp
char g; cin >> g;
switch (g) {
    case 'A': case 'B': cout << "pass with credit\n"; break;
    case 'C':           cout << "pass\n"; break;
    default:            cout << "retake\n";
}
```

```cpp
char g; cin >> g;
switch (g) {
```

Traps

- ⚠ `if (x = 5)` assigns 5 and is always true. With `-Wall` you get a warning; read it.
- ⚠ `for (int i = 0; i < v.size() - 1; i++)` on an empty vector: `v.size() - 1` wraps around to a huge unsigned number. Cast first: `(int)v.size() - 1`.
- ⚠ Off-by-one: decide whether your loop is `< n` or `<= n` from a tiny example, not by feel.

Drill

1. **Six patterns.** For n = 5 print: a solid square, a right triangle, an inverted right triangle, the number triangle, a pyramid, and a diamond (pyramid plus its upside-down copy).
2. **FizzBuzz.** Print 1 to n, but Fizz for multiples of 3, Buzz for 5, FizzBuzz for 15.
3. **Primes up to n.** Print all primes up to n (n up to 1000) with a nested loop. You will meet the faster sieve in Step 8.

Check yourself

- ☐ How many times does `for (i = 0; i < n; i++) for (j = i; j < n; j++)` run its body?
- ☐ What happens in a switch when you forget `break`?

DAY 1 | BLOCK 4 | 2:30 to 4:00

Functions and references

> **Goal.** Split a program into small functions, and know exactly when a function can change the caller's data.

Concepts

- A function has a return type, a name, parameters and a body: `long long square(int x) { return 1LL * x * x; }`. `void` returns nothing.
- **Pass by value** (`int x`) copies the argument. Changes stay inside the function.

- **Pass by reference** (`int &x`) gives the function the caller's variable itself. Changes are visible outside.
- **Pass by const reference** (`const vector<int> &v`) avoids copying a large vector and promises not to change it. Use it for every big input you only read.
- A function can hand back two answers through reference parameters or by returning a `pair` .
- Euclid's algorithm: `gcd(a, b) = gcd(b, a % b)` until b is 0. LCM is `a / gcd(a, b) * b` (divide first, to avoid overflow).
- A function may call itself (recursion). You will go deeper in Day 2, Block 4.

Type these out

VALUE VS REFERENCE

Copy

```
void byValue(int x)     { x = 100; }
void byRef(int &x)      { x = 100; }
long long sumOf(const vector<int> &v) {          // no copy, read-only
    long long s = 0;
    for (int x : v) s += x;
    return s;
}
long long gcdll(long long a, long long b) { return b == 0 ? a : gcdll(b, a % b); }

int main() {
    int a = 1, b = 1;
    byValue(a); byRef(b);
    cout << a << ' ' << b << '\n';
    cout << sumOf({1, 2, 3}) << ' ' << gcdll(12, 18) << ' ' << 12 / gcdll(12, 18) * 18 <<
'\n';
}
```

```
void byValue(int x)
{ x = 100; }
```

▶ OUTPUT

```
1 100
6 6 36
```

Traps

⚠ Passing a big vector by value into a recursive function copies it on every call: an O(n) cost hidden in each call.

⚠ A non-void function that reaches the end without `return` is undefined behaviour. `-Wall` warns you.

⚠ Never return a reference or pointer to a local variable. It dies when the function ends.

Drill

1. **Number library.** Write `isPrime(n)` (O(sqrt n)), `countDigits(n)` (return 1 for 0), `reverseNum(n)`, `isArmstrong(n)` and `power(a, b)`. Call each from `main` on 3 test values.
2. **Swap.** Write `swap2(int &a, int &b)` and show that a version without `&` does nothing.
3. **Min and max together.** Write `void minMax(const vector<int> &v, int &mn, int &mx)` that fills both answers.

Check yourself

- ☐ When should a parameter be `const vector<int> &` instead of `vector<int>`?
- ☐ Why compute LCM as `a / g * b` and not `a * b / g`?

DAY 1 | BLOCK 5 | 4:15 to 5:45

Arrays, vectors and strings

> **Goal.** Store, walk and change sequences fluently. Most of Striver's first 100 problems live here.

Concepts

- C arrays (`int a[100];`) have a fixed size. `vector` grows. Use vector unless a problem needs a big global array.
- Vector essentials: `vector<int> v(n)` (n zeros), `v(n, -1)`, `push_back`, `pop_back`, `size`, `empty`, `front`, `back`, `clear`, `resize`. `insert` and `erase` in the middle cost O(n).
- 2D grid: `vector<vector<int>> g(r, vector<int>(c, 0));` then `g[i][j]`.
- Walk by index when you need positions, with range-for (`for (int x : v)`) when you only need values. Use `for (int &x : v)` to change them.
- `string` is a vector of chars with extras: `s.size()`, `s[i]`, `s += 'x'`, `s.substr(pos, len)` (length, not end index), `s.find("ab")` (returns `string::npos` if absent), `reverse(s.begin(), s.end())`, `to_string(42)`, `stoi("42")`, `stoll`.
- Character tests from `<cctype>`: `isdigit`, `isalpha`, `isalnum`, `tolower`, `toupper`.
- Counting letters: `int cnt[26] = {0}; for (char c : s) cnt[c - 'a']++;` This one line is the start of hashing.

Type these out

VECTORS AND STRINGS IN ACTION Copy

```cpp
vector<int> v = {5, 1, 4};
v.push_back(2);
for (int &x : v) x *= 10;                  // change in place
cout << v.size() << ' ' << v.back() << '\n';

for (int i = 0, j = (int)v.size() - 1; i < j; i++, j--) swap(v[i], v[j]);   // reverse
for (int x : v) cout << x << ' ';
cout << '\n';

string s = "Racecar";
string t = s;
for (char &c : t) c = tolower(c);
string r(t.rbegin(), t.rend());
cout << (t == r ? "palindrome" : "no") << ' ' << s.substr(1, 3) << ' ' << s.find("car") <<
'\n';

int cnt[26] = {0};
for (char c : t) cnt[c - 'a']++;
cout << cnt['r' - 'a'] << '\n';
```

```
vector<int> v = {5,
1, 4};
```

▶ OUTPUT

```
4 20
20 40 10 50
palindrome ace 4
2
```

Traps

- ⚠ `v[i]` past the end does not raise an error, it reads garbage. While debugging compile with `-D_GLIBCXX_DEBUG` and GCC will stop at the bad index.
- ⚠ `substr(2, 5)` means 5 characters from index 2, not index 2 to 5.
- ⚠ `s = s + c` inside a loop copies the whole string each time (O(n^2) total). Use `s += c` or `push_back`.

Drill

1. **Array stats.** Read n numbers. Print max, min, sum and the index of the max.
2. **Second largest.** Find the second largest distinct value in one pass, without sorting. Print -1 if none.
3. **Rotate by one.** Left-rotate an array by one place in place.
4. **Vowel count and palindrome.** Read a line with spaces. Count vowels, then check if it is a palindrome ignoring case and non-letters.

Check yourself

- [ ] What does `s.find("zz")` return when "zz" is not in s, and how do you test for it?
- [ ] What is the cost of `v.erase(v.begin())` on a vector of n items?

DAY 2 | BLOCK 1 | 9:00 to 10:30

STL I: pair, sort, lambdas and algorithms

> **Goal.** Sort anything by any rule and use the algorithm library instead of hand-written loops.

Concepts

- `pair<int,int> p = {3, 7};` then `p.first`, `p.second`. Pairs compare by first, then by second, so sorting a vector of pairs just works.
- Structured bindings unpack a pair: `auto [x, y] = p;` and `for (auto &[k, v] : m)`.
- `sort(v.begin(), v.end())` is O(n log n). Descending: `sort(v.rbegin(), v.rend())` or pass `greater<int>()`.
- Custom order with a lambda: `sort(v.begin(), v.end(), [](const T &a, const T &b) { return ...; });` The lambda answers "should a come before b?". It must use strict `<` or `>`, never `≤`.
- Algorithms you will use weekly: `reverse`, `max_element`, `min_element`, `accumulate`, `count`, `find`, `binary_search`, `lower_bound` (first >= x), `upper_bound` (first > x), `unique`, `next_permutation`, `fill`, `iota`, `__builtin_popcount`.
- `max_element` returns an iterator; `*it` is the value, `it - v.begin()` is the index.

Type these out

SORTING AND ALGORITHMS

Copy

```
vector<pair<string,int>> people = {{"ravi", 21}, {"anu", 19}, {"kiran", 21}};
sort(people.begin(), people.end(), [](const auto &a, const auto &b) {
    if (a.second ≠ b.second) return a.second > b.second;   // older first
    return a.first < b.first;                               // then by name
});
for (auto &[name, age] : people) cout << name << ' ' << age << '\n';

vector<int> v = {4, 1, 4, 2, 9, 2};
sort(v.begin(), v.end());                                   // 1 2 2 4 4 9
v.erase(unique(v.begin(), v.end()), v.end());               // 1 2 4 9
cout << *max_element(v.begin(), v.end()) << ' '
     << accumulate(v.begin(), v.end(), 0LL) << ' '
     << lower_bound(v.begin(), v.end(), 3) - v.begin() << '\n';   // index of first ≥ 3

string s = "abc";
```

```
do cout << s << ' '; while (next_permutation(s.begin(), s.end()));
cout << '\n' << __builtin_popcount(13) << '\n';               // 1101 has 3 set bits

vector<pair<string,in
t>> people =
```

▶ OUTPUT

```
kiran 21
ravi 21
anu 19
9 16 2
abc acb bac bca cab cba
3
```

Traps

- ⚠ `accumulate(v.begin(), v.end(), 0)` sums in int and overflows. Use `0LL`.
- ⚠ A comparator with `⩽` breaks sort's rules and can crash.
- ⚠ `unique` only removes adjacent duplicates. Sort first, then erase what it returns.
- ⚠ `lower_bound` on an unsorted vector gives nonsense without any error.

Drill

1. **Sort pairs.** Sort pairs by second value descending, ties by first ascending.
2. **Distinct count.** Count distinct values with sort plus unique.
3. **All permutations.** Print every permutation of a string in sorted order using `next_permutation`.
4. **Count below x.** For q queries, print how many array values are less than x using `lower_bound` (O(log n) per query).

Check yourself

- ☐ What must a sort comparator return for equal elements?
- ☐ What is the difference between `lower_bound` and `upper_bound`?

DAY 2 | BLOCK 2 | 10:45 to 12:15

STL II: map, set and their hash versions

> **Goal.** Choose between ordered and hashed containers on purpose and use them without accidental inserts.

Concepts

- `set<int>` : sorted, unique values, O(log n) insert, find and erase. `multiset` allows duplicates.
- `map<K,V>` : sorted by key, O(log n). `unordered_map` and `unordered_set` : hash tables, O(1) on average, O(n) in the rare worst case, no order.
- Use the ordered version when you need sorted iteration, the smallest or largest key, or `lower_bound` . Otherwise prefer unordered for speed.
- `m[key]` creates the key with value 0 if missing. That is perfect for counting (`m[x]++`) and a bug when you only meant to check. To check, use `m.count(key)` or `m.find(key) != m.end()` .
- Iterating a map gives pairs in key order: `for (auto &[k, v] : m)` .
- `multiset::erase(x)` removes every copy of x. To remove one, `ms.erase(ms.find(x))` .
- Erasing while iterating: `it = m.erase(it);` and do not increment in that case.

Type these out

COUNTING AND LOOKUP

Copy

```
vector<int> a = {3, 1, 3, 2, 1, 3};
map<int,int> freq;
for (int x : a) freq[x]++;
for (auto &[val, c] : freq) cout << val << ':' << c << ' ';
cout << '\n';

unordered_set<int> seen;
int target = 5;
for (int x : a) {
    if (seen.count(target - x)) { cout << "pair " << target - x << '+' << x << '\n'; break;
}
    seen.insert(x);
}

set<int> s = {10, 20, 30};
auto it = s.lower_bound(15);
cout << *it << ' ' << *s.begin() << ' ' << *s.rbegin() << '\n';

multiset<int> ms = {5, 5, 5};
ms.erase(ms.find(5));
cout << ms.size() << '\n';
```

```
vector<int> a = {3,
1, 3, 2, 1, 3};
```

OUTPUT

```
1:2 2:1 3:3
pair 3+2
```

```
20 10 30
2
```

Traps

- ⚠ `if (m[x] > 0)` inserts x into the map. Over many queries the map grows and slows down.
- ⚠ `unordered_map<pair<int,int>,int>` does not compile without a custom hash. Use `map` with pair keys.
- ⚠ Never change a set element in place; erase and re-insert.

Drill

1. **Most frequent.** Print the most frequent number; on a tie, the smallest.
2. **First unique character.** Print the first character that appears exactly once in a string, or -1.
3. **Two-sum indices.** Print indices of two numbers adding to target in O(n) with `unordered_map`.
4. **Anagram groups.** Group words that are anagrams by using the sorted word as a map key.

Check yourself

- ☐ When would you pick `map` over `unordered_map`?
- ☐ Why is `if (m[x])` a hidden bug?

DAY 2 | BLOCK 3 | 1:15 to 2:45

STL III: stack, queue, deque, priority_queue

> **Goal.** Know which container models which real behaviour: undo, waiting line, both ends, best-first.

Concepts

- `stack`: last in, first out. `push`, `pop`, `top`, `empty`. Models undo, matching brackets, the call stack.
- `queue`: first in, first out. `push`, `pop`, `front`, `back`. Models waiting lines and BFS.
- `deque`: push and pop at both ends in O(1), plus indexing. Models sliding windows.
- `priority_queue<int>`: a max-heap. `top` is the largest; `push` and `pop` are O(log n).
- Min-heap: `priority_queue<int, vector<int>, greater<int>>`. With pairs, `greater<pair<int,int>>` orders by first then second, which is how Dijkstra stores (distance, node).

- `pop()` returns nothing. Read `top()` or `front()` first, then pop.

Type these out

FOUR CONTAINERS

Copy

```
stack<int> st;  st.push(1); st.push(2); st.push(3);
cout << st.top() << ' ';  st.pop();  cout << st.top() << '\n';

queue<string> q;  q.push("a"); q.push("b");
cout << q.front() << ' ' << q.back() << '\n';

deque<int> d = {2, 3};  d.push_front(1);  d.push_back(4);
cout << d.front() << ' ' << d.back() << ' ' << d[1] << '\n';

priority_queue<int> mx;                                  // max-heap
priority_queue<int, vector<int>, greater<int>> mn;       // min-heap
for (int x : {5, 1, 8, 3}) { mx.push(x); mn.push(x); }
cout << mx.top() << ' ' << mn.top() << '\n';

priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
pq.push({7, 0}); pq.push({2, 5}); pq.push({2, 1});
cout << pq.top().first << ',' << pq.top().second << '\n';
```

```
stack<int> st;
st.push(1);
```

▶ OUTPUT

```
3 2
a b
1 4 2
8 1
2,1
```

Traps

- ⚠ `top()` or `pop()` on an empty stack, queue or heap is undefined behaviour. Check `empty()` first.
- ⚠ The default priority_queue is a max-heap. Most bugs are a forgotten `greater`.
- ⚠ You cannot iterate a stack, queue or priority_queue. If you need to, you wanted a vector or deque.

Drill

1. **Brackets.** Check whether a string of `()[]{}` is balanced.
2. **Reverse a queue.** Reverse a queue using a stack.
3. **Kth largest.** Find the kth largest with a min-heap that never holds more than k items.

4. **Printer simulation.** Jobs arrive with a priority; print the order they are printed in. Use a max-heap of (priority, -arrival).

Check yourself

- [ ] Why keep a heap of size k for the kth largest instead of sorting?
- [ ] Which container would you use for BFS, and why?

DAY 2 | BLOCK 4 | 3:00 to 4:30

Recursion, pointers and structs

> **Goal.** Trust recursion, trace it on paper, and build the node types that linked lists and trees use.

Concepts

- Every recursive function needs a **base case** (stop) and a **smaller call** (progress). Then trust that the smaller call works.
- Each call gets its own copy of local variables on the call stack. Too deep (about 1e5 to 1e6 calls) and the program crashes with a stack overflow.
- Two styles: **parameterised** (carry the answer down: `f(i, sum)`) and **functional** (return the answer up: `return n + f(n-1)`).
- Draw the recursion tree for `fib(4)`: it calls `fib(2)` twice. That repeated work is what memoisation (Step 16) removes.
- A pointer stores an address: `int *p = &x;` then `*p` is x. `nullptr` means "points at nothing".
- A `struct` groups fields. With a pointer to a struct, use `p→field`. Linked lists and trees are structs that point at more structs.

Type these out

RECURSION

Copy

```
void printUp(int i, int n) { if (i > n) return; cout << i << ' '; printUp(i + 1, n); }
long long sumTo(int n) { return n == 0 ? 0 : n + sumTo(n - 1); }
bool isPal(const string &s, int i) {
    if (i ≥ (int)s.size() / 2) return true;
    return s[i] == s[s.size() - 1 - i] && isPal(s, i + 1);
}
void revArr(vector<int> &a, int l, int r) { if (l ≥ r) return; swap(a[l], a[r]); revArr(a,
l + 1, r - 1); }
```

```
printUp(1, 5); cout << '\n' << sumTo(100) << ' ' << isPal("madam", 0) << '\n';
```

```
void printUp(int i,
int n) { if (i > n)
```

▶ OUTPUT

```
1 2 3 4 5
5050 1
```

● ● ● POINTERS AND THE NODE TYPES YOU WILL BUILD Copy

```
struct ListNode {
    int val; ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};
struct TreeNode {
    int val; TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

ListNode *head = nullptr, *tail = nullptr;
for (int x : {1, 2, 3}) {
    ListNode *node = new ListNode(x);
    if (!head) head = tail = node; else { tail->next = node; tail = node; }
}
for (ListNode *p = head; p; p = p->next) cout << p->val << (p->next ? " -> " : "\n");
```

```
struct ListNode {
    int val; ListNode
```

▶ OUTPUT

```
1 -> 2 -> 3
```

Traps

- ⚠ No base case, or a base case that is never reached, gives a segmentation fault (stack overflow).
- ⚠ Plain recursive Fibonacci is O(2^n). fib(45) already takes seconds.
- ⚠ Dereferencing `nullptr` (`p->val` when p is null) crashes. In list loops, test `p` before `p->next` .

Drill

1. **Five recursions.** Print 1 to N, print N to 1, sum of first N, factorial of N, check palindrome. Each without any loop.
2. **Trace by hand.** Draw the call tree of `fib(5)` . Count the calls (answer: 15).

3. **Linked list from a vector.** Build a list from a vector, print it, then count its nodes recursively.

Check yourself

- ☐ What two things does every recursive function need?
- ☐ What is the difference between `p.val` and `p→val`?

DAY 2 | BLOCK 5 | 4:45 to 5:45

Complexity and hashing

> **Goal.** Estimate in seconds whether your idea will pass, before you write it.

Concepts

- Big-O describes how work grows with input size, ignoring constants. Loops in sequence add; nested loops multiply.
- A loop that halves or doubles its variable runs about log2(n) times (20 for a million).
- Sorting is O(n log n). Hash map operations are O(1) on average. Ordered map and set operations are O(log n).
- Space counts too, including the recursion stack: recursion of depth n uses O(n) space.
- Judges run roughly 1e8 simple operations per second. Read the constraints first, then pick an algorithm that fits (table below and in the Interview Kit).
- **Hashing** means precomputing counts so each question is O(1): an array `cnt[x]` when values are small (up to about 1e6), a map otherwise.

Type these out

PRECOMPUTE, THEN ANSWER FAST — Copy

```
// n numbers, then q queries "how many times does x appear?"
int n; cin >> n;
vector<int> a(n); for (int &x : a) cin >> x;
unordered_map<int,int> cnt;
for (int x : a) cnt[x]++;                // O(n) once
int q; cin >> q;
while (q--) {
    int x; cin >> x;
    auto it = cnt.find(x);               // O(1) per query, no accidental insert
    cout << (it == cnt.end() ? 0 : it->second) << '\n';
}
```

```
// n numbers, then q queries "how many
```

Traps

- ⚠ Calling `s.size()` or `v.size()` is O(1); calling `count` on a vector is O(n). Do not put an O(n) call inside an O(n) loop by accident.
- ⚠ `string` concatenation and `substr` inside loops are hidden O(n) costs.

Drill

1. **Complexity quiz.** Give the Big-O of the eight snippets in the quiz below. Answers are in the Appendix.
2. **Frequency queries.** Type out the example above and test it with 5 queries, including a value that never appears.

Check yourself

- ☐ How many times does a loop that doubles i from 1 to 1e9 run?
- ☐ If n is 1e5, will an O(n^2) solution pass in 1 second?

DAY 2 | BLOCK 5 EXTRA

Complexity quiz

Q1 Copy

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) work();
```

```
for (int i = 0; i <
n; i++)
```

▼ Answer

O(n^2)

Q2 Copy

```
for (int i = 1; i < n; i *= 2) work();
```

```
for (int i = 1; i <
n; i *= 2) work();
```

▼ Answer

O(log n)

Q3 Copy

```
for (int i = 0; i < n; i++)
    for (int j = i; j < n; j++) work();
```

```
for (int i = 0; i <
n; i++)
```

▼ Answer

O(n^2): n(n+1)/2 runs

Q4 Copy

```
for (int i = 0; i < n; i++)
    for (int j = 1; j < n; j *= 2) work();
```

```
for (int i = 0; i <
n; i++)
```

▼ Answer

O(n log n)

Q5 Copy

```
sort(a.begin(), a.end());
for (int x : a) work();
```

```
sort(a.begin(),
a.end());
```

▼ Answer

O(n log n): the sort dominates

Q6 Copy

```
while (n > 0) n /= 10;
```

```
while (n > 0) n /=
10;
```

▼ Answer

O(log n): one step per digit

Q7 Copy

```
for (long long i = 1; i * i <= n; i++) work();
```

```
for (long long i = 1;
i * i <= n; i++)
```

▼ Answer

O(sqrt n)

Q8 Copy

```
long long fib(int n) { return n < 2 ? n : fib(n-1) + fib(n-2); }
```

```
long long fib(int n)
{ return n < 2 ? n :
```

▼ Answer

O(2^n), more exactly about 1.6^n

REFERENCE

STL and algorithm cheat sheet

CONTAINER	OPERATIONS	COST	USE IT FOR
vector	`push_back` `pop_back` `back` `size` `[i]`	O(1) amortised end, O(n) middle	Default sequence
string	`+=` `substr` `find` `size` `[i]`	`substr` / `find` O(n)	Text
pair	`.first` `.second`, compares first then second	O(1)	Two values together
set / multiset	`insert` `erase` `find` `count` `lower_bound` `begin` `rbegin`	O(log n)	Sorted unique values, min/max
map	`m[k]` `find` `count` `erase` `lower_bound`	O(log n)	Sorted key to value
unordered_map / set	same as map / set, no order	O(1) average	Fast counting and lookup
stack	`push` `pop` `top` `empty`	O(1)	Undo, brackets, monotonic stack
queue	`push` `pop` `front` `back`	O(1)	BFS, waiting lines

CONTAINER	OPERATIONS	COST	USE IT FOR
deque	`push_front` `push_back` `pop_front` `pop_back` `[i]`	O(1)	Sliding window
priority_queue	`push` `pop` `top` (max first; `greater` for min)	O(log n) push/pop	Top K, Dijkstra, scheduling

ALGORITHM	COST	DOES
`sort(b, e)` / `sort(b, e, cmp)`	O(n log n)	Ascending, or by your rule
`reverse(b, e)`	O(n)	Reverse in place
`*max_element(b, e)`	O(n)	Largest value (`- v.begin()` for index)
`accumulate(b, e, 0LL)`	O(n)	Sum (start with 0LL)
`count(b, e, x)`	O(n)	How many equal x
`lower_bound(b, e, x)`	O(log n)	First >= x in sorted range
`upper_bound(b, e, x)`	O(log n)	First > x in sorted range
`binary_search(b, e, x)`	O(log n)	true if x exists
`unique(b, e)` then `erase`	O(n)	Remove adjacent duplicates
`next_permutation(b, e)`	O(n)	Next ordering; false after the last
`__builtin_popcount(x)` / `popcountll`	O(1)	Number of set bits
`swap(a, b)` , `min` , `max` , `abs`	O(1)	Everyday helpers

PRACTICE

The 20-problem practice set

D1-1 Sum of N numbers

Read n (up to 1e5), then n integers (each up to 1e9 in size). Print their sum.

◀ INPUT

```
5
```

▶ OUTPUT

```
15
```

```
1 2 3 4 5
```

▼ Hint

The sum can reach 1e14. Use long long.

▼ Solution

SOLUTION Copy

```
int n; cin >> n;
long long s = 0;
for (int i = 0; i < n; i++) { long long x; cin >> x; s += x; }
cout << s << '\n';
```

```
int n; cin >> n;
long long s = 0;
```

D1-2 Largest of three

Read three integers and print the largest.

◀ INPUT

```
3 9 4
```

▶ OUTPUT

```
9
```

▼ Hint

Chain `max(a, max(b, c))`.

▼ Solution

SOLUTION Copy

```
long long a, b, c; cin >> a >> b >> c;
cout << max(a, max(b, c)) << '\n';
```

```
long long a, b, c;
cin >> a >> b >> c;
```

D1-3 Count digits

Read n (0 to 1e18). Print how many digits it has.

◀ INPUT

```
12345
```

▶ OUTPUT

```
5
```

▼ Hint

0 has one digit; your loop will say 0 unless you handle it.

▼ Solution

SOLUTION Copy

```
long long n; cin >> n;
int c = (n == 0);
while (n > 0) { c++; n /= 10; }
cout << c << '\n';
```

```
long long n; cin >>
n;
```

D1-4 Reverse a number

Read n (0 to 1e9). Print it reversed, without leading zeros.

◀ INPUT

```
1200
```

▶ OUTPUT

```
21
```

▼ Hint

`r = r * 10 + n % 10; n /= 10;`

▼ Solution

SOLUTION Copy

```
long long n, r = 0; cin >> n;
while (n > 0) { r = r * 10 + n % 10; n /= 10; }
cout << r << '\n';
```

```
long long n, r = 0;
cin >> n;
```

D1-5 Palindrome number

Print YES if n reads the same backwards, else NO.

◀ INPUT

```
121
```

▶ OUTPUT

```
YES
```

▼ Hint

Reverse it and compare with the original.

▼ Solution

SOLUTION Copy

```
long long n, r = 0; cin >> n;
for (long long m = n; m > 0; m /= 10) r = r * 10 + m % 10;
cout << (r == n ? "YES" : "NO") << '\n';
```

```
long long n, r = 0;
cin >> n;
```

D1-6 Prime check

Read n (1 to 1e12). Print YES if prime, else NO, in O(sqrt n).

◂ INPUT

```
97
```

▸ OUTPUT

```
YES
```

▼ Hint

1 is not prime. Loop while `i * i ≤ n` with i as long long.

▼ Solution

SOLUTION Copy

```
long long n; cin >> n;
bool prime = n >= 2;
for (long long i = 2; i * i <= n; i++)
    if (n % i == 0) { prime = false; break; }
cout << (prime ? "YES" : "NO") << '\n';
```

```
long long n; cin >>
n;
```

D1-7 GCD and LCM

Read a and b (1 to 1e9). Print gcd and lcm.

◂ INPUT

```
12 18
```

▸ OUTPUT

```
6 36
```

▼ Hint

lcm = a / gcd * b.

▼ Solution

SOLUTION Copy

```
long long gcdll(long long a, long long b) { return b == 0 ? a : gcdll(b, a % b); }
int main() {
    long long a, b; cin >> a >> b;
    long long g = gcdll(a, b);
    cout << g << ' ' << a / g * b << '\n';
}
```

long long gcdll(long long a, long long b)

D1-8 All divisors

Print all divisors of n (up to 1e12) in increasing order.

◀ INPUT

```
36
```

▶ OUTPUT

```
1 2 3 4 6 9 12 18 36
```

▼ Hint

Pair i with n / i up to sqrt n, then sort. Do not push the square root twice.

▼ Solution

SOLUTION Copy

```
long long n; cin >> n;
vector<long long> d;
for (long long i = 1; i * i ≤ n; i++)
    if (n % i == 0) { d.push_back(i); if (i ≠ n / i) d.push_back(n / i); }
sort(d.begin(), d.end());
for (long long x : d) cout << x << ' ';
cout << '\n';
```

long long n; cin >> n;

D1-9 Right triangle

Print a right triangle of stars with n rows.

◀ INPUT

▶ OUTPUT

```
4
```

```
*
**
***
****
```

▼ Hint

Row i prints i stars.

▼ Solution

SOLUTION Copy

```
int n; cin >> n;
for (int i = 1; i ⩽ n; i++) cout << string(i, '*') << '\n';
```

```
int n; cin >> n;
for (int i = 1; i <=
```

D1-10 Pyramid

Print a centred pyramid of stars with n rows.

◀ INPUT

```
3
```

▶ OUTPUT

```
  *
 ***
*****
```

▼ Hint

Row i: n - i spaces, then 2i - 1 stars.

▼ Solution

SOLUTION Copy

```
int n; cin >> n;
for (int i = 1; i ⩽ n; i++)
    cout << string(n - i, ' ') << string(2 * i - 1, '*') << '\n';
```

```
int n; cin >> n;
for (int i = 1; i <=
```

D2-1 Reverse an array

Read n numbers and print them reversed, reversing in place.

◂ INPUT

```
5
1 2 3 4 5
```

▸ OUTPUT

```
5 4 3 2 1
```

▼ Hint

Two pointers from both ends, swap, move inward.

▼ Solution

● ● ● SOLUTION Copy

```cpp
int n; cin >> n;
vector<int> a(n); for (int &x : a) cin >> x;
for (int i = 0, j = n - 1; i < j; i++, j--) swap(a[i], a[j]);
for (int x : a) cout << x << ' ';
cout << '\n';
```

```
int n; cin >> n;
vector<int> a(n); for
```

D2-2 Largest and second largest

Print the largest and the second largest distinct value (or -1 if there is none).

◂ INPUT

```
6
4 9 9 2 7 1
```

▸ OUTPUT

```
9 7
```

▼ Hint

One pass with two variables. A value equal to the largest must not become second.

▼ Solution

● ● ● SOLUTION Copy

```cpp
int n; cin >> n;
long long big = LLONG_MIN, second = LLONG_MIN;
for (int i = 0; i < n; i++) {
    long long x; cin >> x;
    if (x > big) { second = big; big = x; }
    else if (x < big && x > second) second = x;
}
cout << big << ' ' << (second == LLONG_MIN ? -1 : second) << '\n';
```

```
int n; cin >> n;
long long big =
```

D2-3 Frequency table

Print each distinct value and its count in increasing order of value.

◂ INPUT

```
7
3 1 3 2 1 3 5
```

▸ OUTPUT

```
1 2
2 1
3 3
5 1
```

▼ Hint

A `map<int,int>` iterates in key order.

▼ Solution

SOLUTION Copy

```
int n; cin >> n;
map<int,int> f;
for (int i = 0; i < n; i++) { int x; cin >> x; f[x]++; }
for (auto &[v, c] : f) cout << v << ' ' << c << '\n';
```

```
int n; cin >> n;
map<int,int> f;
```

D2-4 Valid brackets

Read a string of `()[]{}`. Print YES if balanced, else NO.

◂ INPUT

```
({[]})
```

▸ OUTPUT

```
YES
```

▼ Hint

Push openers; on a closer, the top must be its partner. The stack must end empty.

▼ Solution

SOLUTION Copy

```
string s; cin >> s;
stack<char> st; bool ok = true;
```

```
for (char c : s) {
    if (c == '(' || c == '[' || c == '{') { st.push(c); continue; }
    if (st.empty()) { ok = false; break; }
    char o = st.top(); st.pop();
    if ((c == ')' && o != '(') || (c == ']' && o != '[') || (c == '}' && o != '{')) { ok =
false; break; }
}
cout << (ok && st.empty() ? "YES" : "NO") << '\n';
```

```
string s; cin >> s;
stack<char> st; bool
```

D2-5 Kth largest

Read n and k, then n numbers. Print the kth largest.

◂ INPUT

```
6 2
3 2 1 5 6 4
```

▸ OUTPUT

```
5
```

▼ Hint

Min-heap of size k: its top is the answer.

▼ Solution

● ● ● SOLUTION Copy

```
int n, k; cin >> n >> k;
priority_queue<int, vector<int>, greater<int>> pq;
for (int i = 0; i < n; i++) {
    int x; cin >> x;
    pq.push(x);
    if ((int)pq.size() > k) pq.pop();       // drop the smallest
}
cout << pq.top() << '\n';
```

```
int n, k; cin >> n >>
k;
```

D2-6 Sort pairs

Read n pairs. Sort by second value descending, ties by first ascending.

◂ INPUT

▸ OUTPUT

```
4
1 5
2 3
3 5
4 1
```

```
1 5
3 5
2 3
4 1
```

▼ Hint

Lambda comparator with two rules.

▼ Solution

SOLUTION Copy

```
int n; cin >> n;
vector<pair<int,int>> v(n);
for (auto &p : v) cin >> p.first >> p.second;
sort(v.begin(), v.end(), [](const pair<int,int> &a, const pair<int,int> &b) {
    if (a.second != b.second) return a.second > b.second;
    return a.first < b.first;
});
for (auto &[x, y] : v) cout << x << ' ' << y << '\n';
```

```
int n; cin >> n;
vector<pair<int,int>>
```

D2-7 Factorial by recursion

Read n (up to 20). Print n! recursively.

◂ INPUT

```
5
```

▸ OUTPUT

```
120
```

▼ Hint

Base case: n <= 1 returns 1.

▼ Solution

SOLUTION Copy

```
long long fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }
int main() { int n; cin >> n; cout << fact(n) << '\n'; }
```

```
long long fact(int n)
{ return n <= 1 ? 1 :
```

D2-8 Fibonacci with memo

Read n (up to 90). Print F(n) with F(0)=0, F(1)=1, recursively with a memo.

◂ INPUT

```
10
```

▸ OUTPUT

```
55
```

▼ Hint

Store answers in an array so each F(i) is computed once.

▼ Solution

● ● ● SOLUTION Copy

```
long long memo[91];                      // globals start at 0
long long fib(int n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}
int main() { int n; cin >> n; cout << fib(n) << '\n'; }
```

```
long long memo[91];
// globals start at 0
```

D2-9 Two-sum indices

Read n and target, then n numbers. Print the first pair of indices (0-based) i < j with a[i]+a[j]=target, found scanning j left to right, or -1 -1.

◂ INPUT

```
4 9
2 7 11 15
```

▸ OUTPUT

```
0 1
```

▼ Hint

For each j, look up target - a[j] in a map of values already seen.

▼ Solution

● ● ● SOLUTION Copy

```
int n, t; cin >> n >> t;
unordered_map<int,int> seen;              // value → first index
```

```
for (int j = 0; j < n; j++) {
    int x; cin >> x;
    auto it = seen.find(t - x);
    if (it != seen.end()) { cout << it->second << ' ' << j << '\n'; return 0; }
    if (!seen.count(x)) seen[x] = j;
}
cout << "-1 -1\n";
```

```
int n, t; cin >> n >>
t;
```

D2-10 Distinct sorted

Print the distinct values of an array in increasing order.

◀ INPUT

```
7
4 1 4 2 1 9 2
```

▶ OUTPUT

```
1 2 4 9
```

▼ Hint

A `set<int>` does it, or sort plus unique.

▼ Solution

SOLUTION Copy

```
int n; cin >> n;
set<int> s;
for (int i = 0; i < n; i++) { int x; cin >> x; s.insert(x); }
for (int x : s) cout << x << ' ';
cout << '\n';
```

```
int n; cin >> n;
set<int> s;
```

DAY 2 | 7 TO 9 PM

The gate test

RULES

120 minutes for all five. No search, no notes, no AI, no hints. Pass mark: 4 of 5 fully correct.

G1 Leaders

An element is a leader if it is greater than every element to its right. Print all leaders in their original order.

◂ INPUT

```
6
16 17 4 3 5 2
```

▸ OUTPUT

```
17 5 2
```

▼ Solution

● ● ● SOLUTION Copy

```
int n; cin >> n;
vector<int> a(n); for (int &x : a) cin >> x;
vector<int> res; int mx = INT_MIN;
for (int i = n - 1; i >= 0; i--)
    if (a[i] > mx) { res.push_back(a[i]); mx = a[i]; }
reverse(res.begin(), res.end());
for (int x : res) cout << x << ' ';
cout << '\n';
```

```
int n; cin >> n;
vector<int> a(n); for
```

G2 Letter counts

Read a lowercase string. Print each distinct letter and its count, alphabetically, one per line.

◂ INPUT

```
banana
```

▸ OUTPUT

```
a 3
b 1
n 2
```

▼ Solution

● ● ● SOLUTION Copy

```
string s; cin >> s;
int cnt[26] = {0};
for (char c : s) cnt[c - 'a']++;
for (int i = 0; i < 26; i++)
    if (cnt[i]) cout << char('a' + i) << ' ' << cnt[i] << '\n';
```

```
string s; cin >> s;
int cnt[26] = {0};
```

G3 Pair with sum

Read n and T, then n numbers. Print YES if two different positions sum to T, else NO. Must be O(n).

◂ INPUT

```
5 10
1 4 6 8 3
```

▸ OUTPUT

```
YES
```

▼ Solution

SOLUTION Copy

```
int n; long long T; cin >> n >> T;
unordered_set<long long> seen; bool found = false;
for (int i = 0; i < n; i++) {
    long long x; cin >> x;
    if (seen.count(T - x)) found = true;
    seen.insert(x);
}
cout << (found ? "YES" : "NO") << '\n';
```

```
int n; long long T;
cin >> n >> T;
```

G4 Pascal's triangle

Read n (up to 30). Print the first n rows of Pascal's triangle.

◂ INPUT

```
5
```

▸ OUTPUT

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

▼ Solution

SOLUTION Copy

```
int n; cin >> n;
vector<long long> row = {1};
```

```
for (int r = 0; r < n; r++) {
    for (size_t i = 0; i < row.size(); i++) cout << row[i] << (i + 1 < row.size() ? ' ' :
'\n');
    vector<long long> nxt(row.size() + 1, 1);
    for (size_t i = 1; i < row.size(); i++) nxt[i] = row[i - 1] + row[i];
    row = nxt;
}
```

```
int n; cin >> n;
vector<long long> row
```

G5 All subsets

Read n (up to 10). Print every subset of {1..n}, one per line, using recursion. Print the empty subset as {}. Any order.

◀ INPUT

```
2
```

▶ OUTPUT

```
1 2
1
2
{}
```

▼ Solution

● ● ● SOLUTION Copy

```
int n; vector<int> cur;
void go(int i) {
    if (i > n) {
        if (cur.empty()) cout << "{}";
        for (size_t k = 0; k < cur.size(); k++) cout << cur[k] << (k + 1 < cur.size() ? " "
: "");
        cout << '\n';
        return;
    }
    cur.push_back(i); go(i + 1); cur.pop_back();   // take i
    go(i + 1);                                     // skip i
}
int main() { cin >> n; go(1); }
```

```
int n; vector<int>
cur;
```

02 Striver A2Z, built

18 steps: ideas, templates, traps, every problem dated, and a small tool to build with each.

PART 02 | METHOD

The 6-step loop for every problem

STEP	WHAT YOU DO	TIME
1. Restate	Input, output, constraints as a comment. One example by hand.	3 min
2. Brute force	Simplest correct idea and its Big-O.	5 min
3. Optimise	Sort? Hash? Two pointers? Binary search the answer? Repeated work (DP)? Graph?	10 to 20 min
4. Code	Clean, in one go, names by meaning.	10 to 15 min
5. Test	Sample plus two edge cases you invent.	5 min
6. Log	Problem, pattern, time, S/H/F, the mistake.	1 min

> WHEN YOU ARE STUCK
>
> At 30 minutes read only the intuition or topic tag. At 45 to 60 minutes read the solution, close it, write the code from memory, mark it H or F.

STEP 01 | 07 NOV TO 08 NOV | HIGH PRIORITY

Step 1: Learn the basics

Why it matters. You learn C++ on 5 and 6 Nov. Step 1 turns it into practice: patterns, STL, maths, recursion and hashing sit under every later step.

Seen in real systems. Input validation, checksums (digit sums in card numbers), any counting report.

Key ideas

- Digits come off the right end with `n % 10` and `n /= 10`. Count digits, reverse, palindrome and Armstrong are all this one loop.
- Divisors come in pairs (i, n / i), so you only loop to sqrt(n). Same for the prime check.
- Recursion: base case first, then one smaller call. Trust the smaller call.
- Hashing: count once into an array or map, then answer each question in O(1).

Templates

STEP 1 TEMPLATES

Copy

```
// digits, right to left
for (long long m = n; m > 0; m /= 10) { int d = m % 10; /* use d */ }

// divisors in O(sqrt n)
vector<long long> ds;
for (long long i = 1; i * i <= n; i++)
    if (n % i == 0) { ds.push_back(i); if (i != n / i) ds.push_back(n / i); }

// frequencies: an array when values are small, a map otherwise
vector<int> cnt(1000001, 0);  for (int x : a) cnt[x]++;
unordered_map<int,int> f;     for (int x : a) f[x]++;
```

```
// digits, right to
left
```

Traps

⚠ Count digits of 0 is 1, not 0.

⚠ `i * i <= n` overflows int for n near 1e9 or more; make i a long long.

⚠ Recursion with no reachable base case crashes with a segmentation fault.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// C++ ESSENTIALS		
Things to know in C++: input/output, data types, if/else, switch, loops	Easy	Sat 07 Nov
Functions: pass by value vs reference, pointers basics	Easy	Sat 07 Nov
Arrays and strings in C++	Easy	Sat 07 Nov
// PATTERNS		
Patterns 1 to 11 (squares, triangles, number triangles)	Easy	Sat 07 Nov

PROBLEM	LEVEL	DUE
Patterns 12 to 17 (letter and mirrored patterns)	Easy	Sat 07 Nov
Patterns 18 to 22 (hollow, diamond, butterfly)	Easy	Sat 07 Nov
// STL		
STL containers: pair, vector, list, deque, stack, queue, priority_queue	Easy	Sat 07 Nov
STL containers: set, multiset, unordered_set, map, multimap, unordered_map	Easy	Sat 07 Nov
STL algorithms: sort, next_permutation, __builtin_popcount, max_element	Easy	Sat 07 Nov
// BASIC MATHS		
Count digits	Easy	Sat 07 Nov
Reverse a number	Easy	Sat 07 Nov
Check palindrome	Easy	Sat 07 Nov
GCD or HCF	Easy	Sat 07 Nov
Armstrong numbers	Easy	Sun 08 Nov
Print all divisors	Easy	Sun 08 Nov
Check for prime	Easy	Sun 08 Nov
// BASIC RECURSION		
Print name N times	Easy	Sun 08 Nov
Print 1 to N and N to 1	Easy	Sun 08 Nov
Sum of first N numbers	Easy	Sun 08 Nov
Factorial of N	Easy	Sun 08 Nov

PROBLEM	LEVEL	DUE
Reverse an array using recursion	Easy	Sun 08 Nov
Check palindrome using recursion	Easy	Sun 08 Nov
Fibonacci number (multiple recursion calls)	Easy	Sun 08 Nov
// BASIC HASHING		
Hashing theory: division method, collisions, unordered_map worst case	Easy	Sun 08 Nov
Counting frequencies of array elements	Easy	Sun 08 Nov
Highest and lowest frequency element	Easy	Sun 08 Nov

BUILD IT

numtool: a number toolkit

A menu-driven command-line program that bundles everything from Step 1, so you use each idea once more in a real interface.

- ☐ Analyse a number: digit count, reverse, palindrome, Armstrong, prime, all divisors.
- ☐ Pattern printer: choose any of the 22 patterns and a size.
- ☐ Frequency report for a list: each value with its count, plus the most and least frequent.
- ☐ Input validation: reject letters and negative sizes without crashing.

▶ SAMPLE RUN

```
$ run numtool.cpp
1) analyse a number  2) print a pattern  3) frequency report  0) quit
> 1
number: 153
digits 3 | reverse 351 | palindrome no | armstrong yes | prime no
divisors: 1 3 9 17 51 153
> 3
how many: 6
values: 4 2 4 9 2 4
4 x3  2 x2  9 x1   most: 4   least: 9
> 0
```

Stretch. Add a recursion mode that prints each recursive call indented by depth, so you can watch factorial(5) unwind.

Resume line. *Built a C++ command-line number toolkit with input validation and frequency analysis.*

LeetCode companions

- **#9 Palindrome Number** (Easy), Sat 07 Nov
- **#1281 Subtract the Product and Sum of Digits of an Integer** (Easy), Sat 07 Nov
- **#509 Fibonacci Number** (Easy), Sat 07 Nov
- **#412 Fizz Buzz** (Easy), Mon 09 Nov
- **#1365 How Many Numbers Are Smaller Than the Current Number** (Easy), Wed 11 Nov
- **#1748 Sum of Unique Elements** (Easy), Sat 14 Nov
- **#7 Reverse Integer** (Medium), Sat 21 Nov

STEP 02 | MON 09 NOV | MEDIUM PRIORITY

Step 2: Sorting techniques

Why it matters. Merge sort teaches divide and conquer, which returns in counting inversions and trees.

Seen in real systems. Databases sort before joining; `std::sort` is introsort (quick sort that falls back to heap sort).

Key ideas

- Selection sort puts the minimum in front each pass; bubble sort swaps neighbours and can stop early when no swap happens; insertion sort slides each item into the sorted prefix and is fast on nearly sorted data.
- Merge sort: split in half, sort each half, merge two sorted lists. Always O(n log n), stable, needs O(n) extra space.
- Quick sort: pick a pivot, put smaller items left and larger right, recurse. O(n log n) on average, O(n^2) when the pivot is always the smallest or largest.
- The merge step comes back in Count inversions (Step 3) and Sort a linked list (Step 6).

Templates

STEP 2 TEMPLATES

Copy

```
void mergeSort(vector<int> &a, int l, int r) {            // sorts a[l..r]
    if (l >= r) return;
    int m = l + (r - l) / 2;
```

```
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    vector<int> tmp; int i = l, j = m + 1;
    while (i ⩽ m && j ⩽ r) tmp.push_back(a[i] ⩽ a[j] ? a[i++] : a[j++]);
    while (i ⩽ m) tmp.push_back(a[i++]);
    while (j ⩽ r) tmp.push_back(a[j++]);
    for (int k = l; k ⩽ r; k++) a[k] = tmp[k - l];
}

int partition(vector<int> &a, int l, int r) {              // pivot = a[r]
    int p = a[r], i = l;
    for (int j = l; j < r; j++) if (a[j] < p) swap(a[i++], a[j]);
    swap(a[i], a[r]);
    return i;
}
void quickSort(vector<int> &a, int l, int r) {
    if (l ⩾ r) return;
    int p = partition(a, l, r);
    quickSort(a, l, p - 1);
    quickSort(a, p + 1, r);
}
```

```
void
mergeSort(vector<int>
```

Traps

- ⚠ Off-by-one in merge bounds: be consistent that `r` is inclusive.
- ⚠ Quick sort with the last element as pivot on already sorted data is O(n^2). Pick a random pivot to avoid it.
- ⚠ Using `<` instead of `⩽` in the merge makes merge sort unstable.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// SORTING ALGORITHMS		
Selection sort	Easy	Mon 09 Nov
Bubble sort	Easy	Mon 09 Nov
Insertion sort	Easy	Mon 09 Nov
Merge sort	Medium	Mon 09 Nov
Quick sort	Medium	Mon 09 Nov
Recursive bubble sort	Easy	Mon 09 Nov
Recursive insertion sort	Easy	Mon 09 Nov

BUILD IT

sortbench: a sorting benchmark

Times every sort you wrote on different kinds of data and proves each one correct. It turns Big-O from theory into numbers you have measured yourself.

- ☐ Generate random, already sorted, reversed and many-duplicates arrays.
- ☐ Time each algorithm with `chrono::steady_clock` and print a table.
- ☐ Verify every output with `is_sorted` and compare against `std::sort`.
- ☐ Skip the O(n^2) sorts automatically when n is 100000 or more.

▶ SAMPLE RUN

```
$ run sortbench.cpp
n       data     selection  bubble   insertion  merge   quick   std::sort
1000    random   0.9 ms     2.1 ms   0.6 ms     0.1 ms  0.1 ms  0.05 ms
10000   random   88 ms      240 ms   55 ms      1.2 ms  0.9 ms  0.6 ms
10000   sorted   86 ms      0.02 ms  0.02 ms    0.9 ms  41 ms   0.1 ms
100000  random   skipped    skipped  skipped    14 ms   10 ms   7 ms
all outputs verified with is_sorted        (your timings will differ)
```

Stretch. Add a random pivot to quick sort and show the sorted-input time collapse.

Resume line. *Benchmarked six sorting algorithms in C++ across input distributions and explained the results with Big-O.*

LeetCode companions

- **#88 Merge Sorted Array** (Easy), Mon 09 Nov
- **#1051 Height Checker** (Easy), Mon 09 Nov
- **#912 Sort an Array** (Medium), Tue 10 Nov
- **#147 Insertion Sort List** (Medium), Thu 12 Nov

STEP 03 | 10 NOV TO 12 NOV | HIGH PRIORITY

Step 3: Arrays

Why it matters. The most common topic in online assessments. Do all Easy and Medium now, and the Hard tier in the pass-2 window.

Seen in real systems. Analytics dashboards (best and worst streaks), stock tools, any running total over time.

Key ideas

- **Two pointers**: one from each end, or a slow and a fast index (remove duplicates, move zeros).
- **Prefix sum plus a hash map**: the sum of a[l..r] is pre[r] - pre[l-1], so "how many subarrays sum to K" becomes "how many earlier prefixes equal pre - K".
- **Kadane**: carry the best sum ending here; drop it when it goes negative.
- **Dutch national flag** for 0s, 1s, 2s: three pointers in one pass. **Moore's voting** for the majority element.
- Matrix tricks: rotate = transpose, then reverse each row. Spiral = shrink four boundaries.

Templates

STEP 3 TEMPLATES Copy

```
// Kadane: best subarray sum (works when every number is negative)
long long best = LLONG_MIN, cur = 0;
for (int x : a) { cur += x; best = max(best, cur); if (cur < 0) cur = 0; }

// count subarrays with sum k: prefix sums + hash map
unordered_map<long long,int> seen;
seen[0] = 1;                                    // the empty prefix
long long pre = 0; int ways = 0;
for (int x : a) {
    pre += x;
    auto it = seen.find(pre - k);
    if (it != seen.end()) ways += it->second;
    seen[pre]++;
}

// rotate an n x n matrix 90 degrees clockwise in place
for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++) swap(g[i][j], g[j][i]);
for (auto &row : g) reverse(row.begin(), row.end());
```

```
// Kadane: best
subarray sum (works
```

Traps

- ⚠ Forgetting `seen[0] = 1` misses subarrays that start at index 0.
- ⚠ Sums of 1e5 values up to 1e9 overflow int.
- ⚠ The sliding-window version of "longest subarray with sum K" only works when all numbers are positive.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// EASY		
Largest element in an array	Easy	Tue 10 Nov
Second largest element without sorting	Easy	Tue 10 Nov
Check if the array is sorted	Easy	Tue 10 Nov
Remove duplicates from a sorted array	Easy	Tue 10 Nov
Left rotate an array by one place	Easy	Tue 10 Nov
Left rotate an array by D places	Easy	Tue 10 Nov
Move zeros to the end	Easy	Tue 10 Nov
Linear search	Easy	Tue 10 Nov
Union of two sorted arrays	Easy	Tue 10 Nov
Find the missing number	Easy	Wed 11 Nov
Maximum consecutive ones	Easy	Wed 11 Nov
Find the number that appears once	Easy	Wed 11 Nov
Longest subarray with sum K (positives)	Easy	Wed 11 Nov
Longest subarray with sum K (positives and negatives)	Easy	Wed 11 Nov
// MEDIUM		
Two Sum (check and indices)	Medium	Wed 11 Nov
Sort an array of 0s, 1s and 2s	Medium	Wed 11 Nov
Majority element (more than n/2)	Medium	Wed 11 Nov
Maximum subarray sum (Kadane)	Medium	Wed 11 Nov
Print the subarray with maximum sum	Medium	Wed 11 Nov
Best time to buy and sell stock	Medium	Thu 12 Nov
Rearrange array elements by sign	Medium	Thu 12 Nov
Next permutation	Medium	Thu 12 Nov
Leaders in an array	Medium	Thu 12 Nov
Longest consecutive sequence	Medium	Thu 12 Nov

PROBLEM	LEVEL	DUE
Set matrix zeroes	Medium	Thu 12 Nov
Rotate matrix by 90 degrees	Medium	Thu 12 Nov
Spiral traversal of a matrix	Medium	Thu 12 Nov
Count subarrays with sum K	Medium	Thu 12 Nov
// HARD		
Pascal's triangle	Hard	Mon 04 Jan (hard pass)
Majority element (more than n/3)	Hard	Tue 05 Jan (hard pass)
3-sum	Hard	Wed 06 Jan (hard pass)
4-sum	Hard	Thu 07 Jan (hard pass)
Largest subarray with sum 0	Hard	Fri 08 Jan (hard pass)
Count subarrays with XOR K	Hard	Sat 09 Jan (hard pass)
Merge overlapping intervals	Hard	Sun 10 Jan (hard pass)
Merge two sorted arrays without extra space	Hard	Mon 11 Jan (hard pass)
Find the repeating and missing number	Hard	Tue 12 Jan (hard pass)
Count inversions	Hard	Wed 13 Jan (hard pass)
Reverse pairs	Hard	Thu 14 Jan (hard pass)
Maximum product subarray	Hard	Fri 15 Jan (hard pass)

BUILD IT

expenses: a spending analyser

Reads a month of daily spending and answers the questions a finance app would, each with an algorithm from this step.

- ☐ Total, average and most expensive day.
- ☐ Longest run of days over budget (maximum consecutive ones).
- ☐ Best saving stretch: Kadane on budget minus spend.
- ☐ Two purchases that add up to a target (two-sum).
- ☐ How many stretches cost exactly K (prefix sum plus hash map).

▶ SAMPLE RUN

```
$ run expenses.cpp < nov.txt
days: 30   total: 18420   daily budget: 600
most expensive day: 14 (2150)
longest run over budget: 4 days (11 to 14)
best saving stretch: days 18 to 27, saved 2340
two purchases that add up to 1000: day 3 (420) + day 9 (580)
stretches that cost exactly 3000: 2
```

Stretch. Read a CSV export from a real UPI or bank statement (remove personal details first).

Resume line. *Wrote a C++ spending analyser using Kadane's algorithm and prefix-sum hashing for O(n) reports.*

LeetCode companions

- **#217 Contains Duplicate** (Easy), Tue 10 Nov
- **#283 Move Zeroes** (Easy), Tue 10 Nov
- **#189 Rotate Array** (Medium), Wed 11 Nov
- **#238 Product of Array Except Self** (Medium), Wed 11 Nov
- **#11 Container With Most Water** (Medium), Thu 12 Nov
- **#560 Subarray Sum Equals K** (Medium), Thu 12 Nov
- **#128 Longest Consecutive Sequence** (Medium), Fri 13 Nov

STEP 04 | 13 NOV TO 15 NOV | HIGH PRIORITY

Step 4: Binary search

Why it matters. Binary search on the answer solves a whole family of medium and hard problems.

Seen in real systems. `git bisect` finds the commit that broke a build. Database indexes and autocomplete use sorted lookups.

Key ideas

- Keep a clear invariant: the answer is always inside [lo, hi]. Compute `mid = lo + (hi - lo) / 2`.
- **lower_bound** = first index with a[i] >= x; **upper_bound** = first with a[i] > x. Count of x = upper - lower.

- **Rotated array**: at least one half around mid is sorted. Check whether the target lies in that half.
- **Binary search on the answer**: when the question is "smallest X that works" and "works" flips from false to true once, binary search X and write `ok(X)`. Koko, ship capacity, aggressive cows, book allocation all look like this.
- In a 2D matrix sorted row by row, treat it as one flat array of r*c items, or start at the top-right corner.

Templates

STEP 4 TEMPLATES

Copy

```
// smallest x in [L, R] with ok(x) true; ok goes false...false, true...true
long long lo = L, hi = R, ans = R;
while (lo ⩽ hi) {
    long long mid = lo + (hi - lo) / 2;
    if (ok(mid)) { ans = mid; hi = mid - 1; }
    else lo = mid + 1;
}

// example ok(): can Koko eat every pile within h hours at speed k?
bool ok(long long k) {
    long long hours = 0;
    for (long long p : piles) hours += (p + k - 1) / k;     // ceil(p / k)
    return hours ⩽ h;
}

// search in a rotated sorted array (distinct values)
int lo = 0, hi = n - 1;
while (lo ⩽ hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == t) return mid;
    if (a[lo] ⩽ a[mid]) {                                   // left half sorted
        if (a[lo] ⩽ t && t < a[mid]) hi = mid - 1; else lo = mid + 1;
    } else {                                                // right half sorted
        if (a[mid] < t && t ⩽ a[hi]) lo = mid + 1; else hi = mid - 1;
    }
}
return -1;
```

```
// smallest x in [L, R] with ok(x) true;
```

Traps

⚠ `(lo + hi) / 2` can overflow; use `lo + (hi - lo) / 2`.

⚠ `while (lo < hi)` with `lo = mid` loops forever. Pick one template and always use it.

⚠ For binary search on the answer, check that `ok` really is monotonic before you trust it.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// ON 1D ARRAYS		
Binary search to find X	Easy	Fri 13 Nov
Implement lower bound	Easy	Fri 13 Nov
Implement upper bound	Easy	Fri 13 Nov
Search insert position	Easy	Fri 13 Nov
Floor and ceil in a sorted array	Easy	Fri 13 Nov
First and last occurrence	Easy	Fri 13 Nov
Count occurrences in a sorted array	Easy	Fri 13 Nov
Search in a rotated sorted array I	Medium	Fri 13 Nov
Search in a rotated sorted array II (duplicates)	Medium	Sat 14 Nov
Minimum in a rotated sorted array	Medium	Sat 14 Nov
How many times the array is rotated	Medium	Sat 14 Nov
Single element in a sorted array	Medium	Sat 14 Nov
Find a peak element	Medium	Sat 14 Nov
// ON ANSWERS		
Square root of N	Easy	Sat 14 Nov
Nth root of M	Medium	Sat 14 Nov
Koko eating bananas	Medium	Sun 15 Nov
Minimum days to make M bouquets	Medium	Sun 15 Nov
Smallest divisor given a threshold	Medium	Sun 15 Nov
Capacity to ship packages within D days	Medium	Sun 15 Nov
Kth missing positive number	Medium	Sun 15 Nov
Aggressive cows	Hard	Sat 16 Jan (hard pass)
Book allocation	Hard	Sun 17 Jan (hard pass)
Split array largest sum	Hard	Mon 18 Jan (hard pass)

PROBLEM	LEVEL	DUE
Painter's partition	Hard	Tue 19 Jan (hard pass)
Minimise maximum distance to a gas station	Hard	Wed 20 Jan (hard pass)
Median of two sorted arrays	Hard	Thu 21 Jan (hard pass)
Kth element of two sorted arrays	Hard	Fri 22 Jan (hard pass)
// ON 2D ARRAYS		
Row with the maximum number of 1s	Medium	Sun 15 Nov
Search in a 2D matrix I	Medium	Sun 15 Nov
Search in a 2D matrix II	Medium	Sun 15 Nov
Find a peak element II	Hard	Sat 23 Jan (hard pass)
Median of a row-wise sorted matrix	Hard	Sun 24 Jan (hard pass)

BUILD IT

lookup and shipwise: search tools

Two tools. One looks words up in a sorted dictionary file. The other answers a logistics question with binary search on the answer.

- ☐ Load a sorted word list (any public English word list) into a vector.
- ☐ `pre <prefix>` : list every word with that prefix using `lower_bound` .
- ☐ `has <word>` : exact lookup, or the two nearest words if missing.
- ☐ `ship <days> <weights...>` : the smallest truck capacity that ships everything in time, plus the day-by-day loading.

▶ SAMPLE RUN

```
$ run lookup.cpp words.txt
> pre algo
algorithm algorithmic algorithms
> has binery
not found. nearest: binary, binge
> ship 3 1 2 3 4 5 6 7 8 9 10
smallest capacity to ship in 3 days: 21
day 1: 1 2 3 4 5 6 (21)   day 2: 7 8 (15)   day 3: 9 10 (19)
```

Stretch. Time 10000 lookups with linear search vs binary search and print the speed-up.

Resume line. *Implemented prefix search and capacity planning in C++ with binary search on sorted data and on the answer space.*

LeetCode companions

- **#704 Binary Search** (Easy), Fri 13 Nov
- **#35 Search Insert Position** (Easy), Fri 13 Nov
- **#34 Find First and Last Position of Element in Sorted Array** (Medium), Sat 14 Nov
- **#33 Search in Rotated Sorted Array** (Medium), Sat 14 Nov
- **#153 Find Minimum in Rotated Sorted Array** (Medium), Mon 16 Nov
- **#162 Find Peak Element** (Medium), Tue 17 Nov

STEP 05 | 16 NOV TO 17 NOV | HIGH PRIORITY

Step 5: Strings (basic and medium)

Why it matters. Frequently combined with hashing and two pointers.

Seen in real systems. Input parsers, log processing, search boxes, and validating user input in web apps.

Key ideas

- A frequency array `int c[26]` answers anagram, frequency-sort and isomorphism questions.
- Palindromes: expand around every centre (odd and even length) for the longest palindromic substring in O(n^2).
- Parsing (atoi, Roman numerals): walk once, keep a running value, handle sign, spaces and overflow explicitly.
- Rotation check: B is a rotation of A if B appears in A + A and the lengths match.

Templates

STEP 5 TEMPLATES Copy

```
// longest palindromic substring: expand around every centre, O(n^2) time, O(1) space
int n = s.size(), bestL = 0, bestLen = 0;
for (int c = 0; c < n; c++)
    for (int even = 0; even < 2; even++) {       // even = 0: centre at c, 1: between c and c+1
        int l = c, r = c + even;
```

```
        while (l >= 0 && r < n && s[l] == s[r]) { l--; r++; }
        if (r - l - 1 > bestLen) { bestLen = r - l - 1; bestL = l + 1; }
    }
string longest = s.substr(bestL, bestLen);

// anagram check with one count array
bool anagram(const string &a, const string &b) {
    if (a.size() != b.size()) return false;
    int c[26] = {0};
    for (size_t i = 0; i < a.size(); i++) { c[a[i] - 'a']++; c[b[i] - 'a']--; }
    for (int x : c) if (x) return false;
    return true;
}
```

```
// longest
palindromic
```

Traps

- ⚠ atoi: leading spaces, a sign, digits, then stop at the first non-digit; clamp to INT_MIN/INT_MAX before overflowing.
- ⚠ `c - 'a'` assumes lowercase letters; check the constraints.
- ⚠ Building a string with `s = s + c` in a loop is O(n^2).

Every problem, with its due date

PROBLEM	LEVEL	DUE
// BASIC AND MEDIUM		
Remove outermost parentheses	Easy	Mon 16 Nov
Reverse words in a string	Easy	Mon 16 Nov
Largest odd number in a string	Easy	Mon 16 Nov
Longest common prefix	Easy	Mon 16 Nov
Isomorphic strings	Easy	Mon 16 Nov
Check if one string is a rotation of another	Easy	Mon 16 Nov
Valid anagram	Easy	Mon 16 Nov
Sort characters by frequency	Medium	Mon 16 Nov
Maximum nesting depth of parentheses	Easy	Mon 16 Nov
Roman to integer	Easy	Mon 16 Nov
Integer to Roman	Medium	Tue 17 Nov

PROBLEM	LEVEL	DUE
Implement atoi	Medium	Tue 17 Nov
Count number of substrings	Medium	Tue 17 Nov
Longest palindromic substring	Medium	Tue 17 Nov
Sum of beauty of all substrings	Medium	Tue 17 Nov

BUILD IT

textkit: a text toolkit

A command-line tool that reads any text file and runs this step's string algorithms on it.

- ☐ Word and line count; reverse the words of any line.
- ☐ Most common letters (frequency sort).
- ☐ Longest palindrome anywhere in the text.
- ☐ Group the words into anagram families.
- ☐ Convert Roman numerals both ways.

▶ SAMPLE RUN

```
$ run textkit.cpp < notes.txt
words: 312   lines: 41
first line reversed: "placement for practice daily"
most common letters: e 172, t 121, a 118
longest palindrome in the text: "racecar"
anagram groups: [listen silent enlist] [evil live vile]
roman: XLII = 42, 2026 = MMXXVI
```

Stretch. Add a password-strength checker (length, classes of characters, repeated runs). A small cyber tie-in.

Resume line. *Built a C++ text-analysis tool covering palindromes, anagram grouping and numeral parsing.*

LeetCode companions

- **#125 Valid Palindrome** (Easy), Mon 16 Nov
- **#242 Valid Anagram** (Easy), Mon 16 Nov
- **#49 Group Anagrams** (Medium), Wed 18 Nov
- **#151 Reverse Words in a String** (Medium), Thu 19 Nov

STEP 06 | 17 NOV TO 19 NOV | MEDIUM PRIORITY

Step 6: Linked list

Why it matters. Pointer manipulation and a classic interview round filler.

Seen in real systems. Music and video playlists, browser tabs, the OS's free-memory lists, LRU caches.

Key ideas

- A **dummy node** before the head removes special cases when the head itself changes.
- **Slow and fast pointers**: the middle (fast moves 2), cycle detection (they meet), the cycle start (reset one to head, move both by 1).
- **Reverse in place** with three pointers: prev, cur, next. Save next before you rewire.
- Many medium problems are combos: palindrome check = find middle + reverse second half + compare.

Templates

STEP 6 TEMPLATES

Copy

```
ListNode* reverseList(ListNode *head) {
    ListNode *prev = nullptr, *cur = head;
    while (cur) {
        ListNode *nxt = cur->next;          // save before rewiring
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}

ListNode* middle(ListNode *head) {          // second middle when the length is even
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}

ListNode* removeNthFromEnd(ListNode *head, int n) {
    ListNode dummy(0); dummy.next = head;
    ListNode *fast = &dummy, *slow = &dummy;
    for (int i = 0; i <= n; i++) fast = fast->next;      // gap of n nodes
    while (fast) { fast = fast->next; slow = slow->next; }
    slow->next = slow->next->next;
    return dummy.next;
}
```

```
ListNode*
reverseList(ListNode
```

Traps

⚠ Losing the rest of the list because `cur→next` was overwritten before being saved.

⚠ `fast→next→next` when `fast→next` is null: always test `fast && fast→next`.

⚠ Forgetting to update `prev` pointers in a doubly linked list.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// BASICS		
Introduction to a linked list	Easy	Tue 17 Nov
Insert a node	Easy	Tue 17 Nov
Delete a node	Easy	Tue 17 Nov
Length of a linked list and searching	Easy	Tue 17 Nov
// DOUBLY LINKED LIST		
Insert into a doubly linked list	Easy	Tue 17 Nov
Delete a node in a doubly linked list	Easy	Tue 17 Nov
Reverse a doubly linked list	Easy	Wed 18 Nov
Delete all occurrences of a key in a DLL	Medium	Wed 18 Nov
Find pairs with a given sum in a DLL	Medium	Wed 18 Nov
Remove duplicates from a sorted DLL	Medium	Wed 18 Nov
// MEDIUM		
Middle of a linked list	Easy	Wed 18 Nov
Reverse a linked list (iterative)	Easy	Wed 18 Nov
Reverse a linked list (recursive)	Easy	Wed 18 Nov
Detect a loop	Easy	Wed 18 Nov
Find the starting point of the loop	Medium	Wed 18 Nov
Length of the loop	Medium	Wed 18 Nov
Check if a linked list is a palindrome	Medium	Wed 18 Nov

PROBLEM	LEVEL	DUE
Segregate odd and even nodes	Medium	Wed 18 Nov
Remove the Nth node from the back	Medium	Wed 18 Nov
Delete the middle node	Medium	Thu 19 Nov
Merge two sorted linked lists	Easy	Thu 19 Nov
Sort a linked list	Medium	Thu 19 Nov
Sort a linked list of 0s, 1s and 2s	Medium	Thu 19 Nov
Intersection of two linked lists	Medium	Thu 19 Nov
Add 1 to a number in a linked list	Medium	Thu 19 Nov
Add two numbers in linked lists	Medium	Thu 19 Nov
Rotate a linked list	Medium	Thu 19 Nov
// HARD		
Reverse nodes in k-group	Hard	Mon 25 Jan (hard pass)
Flatten a linked list	Hard	Tue 26 Jan (hard pass)
Clone a linked list with random pointers	Hard	Wed 27 Jan (hard pass)

BUILD IT

playlist: a music queue

A playlist manager built on your own doubly linked list, with the operations a music app needs.

- ☐ `add`, `remove`, `next`, `prev` and `show`, all on your own DLL (no `std::list`).
- ☐ Move a song up or down by relinking nodes.
- ☐ `middle` with slow and fast pointers.
- ☐ `repeat on` links the tail back to the head; `check` detects the loop with Floyd's algorithm.

▶ SAMPLE RUN

```
$ run playlist.cpp
> add Kesariya
> add Believer
> add Shape of You
> show
Kesariya ⟷ Believer ⟷ Shape of You     (now playing: Kesariya)
```

```
> next
now playing: Believer
> remove Kesariya
> show
Believer ⟷ Shape of You
> middle
Shape of You      (second of two middles)
```

Stretch. Shuffle: copy node pointers into a vector, Fisher-Yates shuffle them, then relink.

Resume line. *Implemented a playlist engine on a hand-written doubly linked list with cycle detection.*

LeetCode companions

- **#206 Reverse Linked List** (Easy), Tue 17 Nov
- **#21 Merge Two Sorted Lists** (Easy), Tue 17 Nov
- **#141 Linked List Cycle** (Easy), Wed 18 Nov
- **#19 Remove Nth Node From End of List** (Medium), Wed 18 Nov
- **#142 Linked List Cycle II** (Medium), Fri 20 Nov
- **#2 Add Two Numbers** (Medium), Mon 23 Nov

STEP 07 | 19 NOV TO 21 NOV | HIGH PRIORITY

Step 7: Recursion and backtracking

Why it matters. Trains the state-tree thinking that dynamic programming needs.

Seen in real systems. Sudoku and puzzle solvers, schedulers that try options, brute-force password search (and why long passwords defeat it).

Key ideas

- Every backtracking problem is a tree of choices. At each node: **choose**, **explore**, **un-choose**.
- Pick / not pick: at index i, either include a[i] or skip it. That gives every subsequence.
- Skip duplicates: sort first, then `if (i > start && a[i] == a[i-1]) continue;`.
- Prune early: stop a branch as soon as it cannot lead to an answer (sum already too big, queen attacked).
- Count the tree before coding: n items with pick/not pick is 2^n leaves; permutations are n!.

Templates

STEP 7 TEMPLATES

Copy

```cpp
// all subsets without duplicates (sort nums first)
vector<vector<int>> res; vector<int> cur;
void go(const vector<int> &nums, int start) {
    res.push_back(cur);                                  // every node is a subset
    for (int i = start; i < (int)nums.size(); i++) {
        if (i > start && nums[i] == nums[i - 1]) continue;   // skip duplicate branches
        cur.push_back(nums[i]);                          // choose
        go(nums, i + 1);                                 // explore
        cur.pop_back();                                  // un-choose
    }
}

// pick / not pick: count subsequences with sum k
int countK(const vector<int> &a, int i, int sum, int k) {
    if (i == (int)a.size()) return sum == k;
    return countK(a, i + 1, sum + a[i], k) + countK(a, i + 1, sum, k);
}
```

```
// all subsets
without duplicates
```

Traps

⚠ Forgetting `pop_back()` after the recursive call leaves stale choices in `cur`.

⚠ Passing `cur` by value copies it at every node and slows everything down.

⚠ Using a global result without clearing it between test cases.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// SUBSEQUENCES AND COMBINATIONS		
Generate all binary strings	Medium	Thu 19 Nov
Generate parentheses	Medium	Thu 19 Nov
Print all subsequences	Medium	Thu 19 Nov
Count good numbers	Medium	Fri 20 Nov
Reverse a stack using recursion	Medium	Fri 20 Nov
Count subsequences with sum K	Medium	Fri 20 Nov
Check if a subsequence with sum K exists	Medium	Fri 20 Nov
Combination sum I	Medium	Fri 20 Nov

PROBLEM	LEVEL	DUE
Combination sum II	Medium	Fri 20 Nov
Combination sum III	Medium	Fri 20 Nov
Subset sum I	Medium	Fri 20 Nov
Subsets II (with duplicates)	Medium	Sat 21 Nov
Letter combinations of a phone number	Medium	Sat 21 Nov
Print all permutations	Medium	Sat 21 Nov
// TRYING ALL COMBINATIONS		
Palindrome partitioning	Medium	Sat 21 Nov
Word search	Medium	Sat 21 Nov
Rat in a maze	Medium	Sat 21 Nov
Kth permutation sequence	Hard	Thu 28 Jan (hard pass)
N-Queens	Hard	Fri 29 Jan (hard pass)
Sudoku solver	Hard	Sat 30 Jan (hard pass)
M-colouring problem	Hard	Sun 31 Jan (hard pass)
Word break	Hard	Mon 01 Feb (hard pass)
Expression add operators	Hard	Tue 02 Feb (hard pass)

BUILD IT

sudoku and pinspace: search tools

A Sudoku solver, and a small tool that shows how fast a brute-force search space grows, which is the reason short PINs are weak.

- ☐ Read a 9x9 puzzle with `.` for blanks; solve with backtracking.
- ☐ Count recursive calls and print them with the solution.
- ☐ Add pruning (try the cell with fewest options first) and compare the counts.
- ☐ pinspace: generate all PINs of length n under rules, count them, and estimate brute-force time.

▶ SAMPLE RUN

```
$ run sudoku.cpp < puzzle.txt
solved. recursive calls: 4213          (yours will differ)
```

```
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
...
$ run pinspace.cpp 4
4-digit PINs with no digit three times in a row: 9810 of 10000
```

Stretch. Add N-Queens for any n and print the number of solutions up to n = 10.

Resume line. *Wrote a backtracking Sudoku solver in C++ with pruning that cut the search by an order of magnitude.*

LeetCode companions

- **#78 Subsets** (Medium), Thu 19 Nov
- **#46 Permutations** (Medium), Thu 19 Nov
- **#39 Combination Sum** (Medium), Fri 20 Nov
- **#22 Generate Parentheses** (Medium), Fri 20 Nov
- **#17 Letter Combinations of a Phone Number** (Medium), Sat 21 Nov
- **#90 Subsets II** (Medium), Sat 21 Nov
- **#79 Word Search** (Medium), Tue 24 Nov
- **#131 Palindrome Partitioning** (Medium), Wed 25 Nov

STEP 08 | SUN 22 NOV | LOW PRIORITY

Step 8: Bit manipulation

Why it matters. Asked less often. Learn the common tricks and move on.

Seen in real systems. Unix file permissions (rwx), feature flags, network subnet masks, compact sets in games.

Key ideas

- Bit i of x: `(x >> i) & 1`. Set: `x | (1 << i)`. Clear: `x & ~(1 << i)`. Toggle: `x ^ (1 << i)`.
- `x & (x - 1)` removes the lowest set bit: power-of-two test and fast popcount.
- XOR cancels pairs: XOR of all numbers leaves the one that appears once.
- Subsets as bitmasks: for mask from 0 to 2^n - 1, bit i says whether item i is in.
- Sieve of Eratosthenes finds all primes up to n in O(n log log n).

Templates

STEP 8 TEMPLATES

Copy

```cpp
bool isSet(int x, int i)    { return (x >> i) & 1; }
int  setBit(int x, int i)   { return x | (1 << i); }
int  clearBit(int x, int i) { return x & ~(1 << i); }
int  toggle(int x, int i)   { return x ^ (1 << i); }
bool isPow2(long long x)    { return x > 0 && (x & (x - 1)) == 0; }
int  popcount(int x) { int c = 0; while (x) { x &= x - 1; c++; } return c; }

// all subsets with bitmasks
for (int mask = 0; mask < (1 << n); mask++) {
    for (int i = 0; i < n; i++) if (mask & (1 << i)) cout << a[i] << ' ';
    cout << '\n';
}

// sieve: primes up to n (n ≥ 1)
vector<bool> isPrime(n + 1, true); isPrime[0] = isPrime[1] = false;
for (long long i = 2; i * i ≤ n; i++)
    if (isPrime[i]) for (long long j = i * i; j ≤ n; j += i) isPrime[j] = false;
```

```
bool isSet(int x, int
i)    { return (x >>
```

Traps

⚠ `x & 1 == 0` means `x & (1 == 0)`. Always bracket: `(x & 1) == 0`.

⚠ `1 << 31` overflows int; use `1LL << i` for i of 31 or more.

⚠ Right-shifting a negative number is implementation-defined in older C++; use unsigned for bit tricks.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// BASICS		
Introduction to bit manipulation (AND, OR, XOR, shifts)	Easy	Sun 22 Nov
Convert decimal to binary and back	Easy	Sun 22 Nov
1's and 2's complement	Easy	Sun 22 Nov
Check if the i-th bit is set	Easy	Sun 22 Nov
Check if a number is odd or even	Easy	Sun 22 Nov
Check if a number is a power of 2	Easy	Sun 22 Nov

PROBLEM	LEVEL	DUE
Count the number of set bits	Easy	Sun 22 Nov
Set, clear and toggle the i-th bit	Easy	Sun 22 Nov
Remove the last set bit	Easy	Sun 22 Nov
Set the rightmost unset bit	Easy	Sun 22 Nov
Swap two numbers without a temporary	Easy	Sun 22 Nov
Divide two integers without * and /	Medium	Sun 22 Nov
// INTERVIEW PROBLEMS		
Minimum bit flips to convert a number	Easy	Sun 22 Nov
Single number I	Easy	Sun 22 Nov
Single number II	Medium	Sun 22 Nov
Single number III	Medium	Sun 22 Nov
XOR of numbers in a range L to R	Medium	Sun 22 Nov
Power set using bit manipulation	Medium	Sun 22 Nov
// MATHS WITH BITS		
Print prime factors of a number	Medium	Sun 22 Nov
Print all divisors of a number	Easy	Sun 22 Nov
Sieve of Eratosthenes	Medium	Sun 22 Nov
Prime factorisation using a sieve	Medium	Sun 22 Nov
Count primes in a range L to R	Medium	Sun 22 Nov
Power(x, n)	Medium	Sun 22 Nov

BUILD IT

perms: a permission manager

A model of Unix file permissions where each file's rights are 9 bits in one integer. It is a direct bridge to your cyber track.

- ☐ `chmod 754 file` sets the bits from octal.
- ☐ Print rights as `rwxr-xr--` and as binary.
- ☐ `can <owner|group|other> <r|w|x> file` checks one bit.

- ☐ `grant` and `revoke` set or clear bits.
- ☐ Flag world-writable files (the `other w` bit) as a security warning.

▶ SAMPLE RUN

```
$ run perms.cpp
> chmod 754 report.txt
report.txt  rwxr-xr--   (111 101 100)
> can group w report.txt
no
> grant group w report.txt
report.txt  rwxrwxr--
> chmod 777 shared.sh
warning: shared.sh is world-writable
```

Stretch. Add subnet maths: given an IP and a /24 mask, print the network address and host range.

Resume line. *Modelled Unix permissions with bitmasks in C++ and flagged insecure world-writable files.*

LeetCode companions

- **#136 Single Number** (Easy), Thu 26 Nov
- **#338 Counting Bits** (Easy), Fri 27 Nov
- **#191 Number of 1 Bits** (Easy), Sat 28 Nov
- **#268 Missing Number** (Easy), Mon 30 Nov

STEP 09 | 23 NOV TO 25 NOV | HIGH PRIORITY

Step 9: Stacks and queues

Why it matters. Monotonic stack questions are common in medium rounds.

Seen in real systems. Undo/redo, browser back/forward, compilers and calculators, caches in every web server.

Key ideas

- A stack answers "what was the most recent unmatched thing?": brackets, undo, expression parsing.
- **Monotonic stack**: keep the stack sorted (say, decreasing); pop everything that the new item beats. Next greater, next smaller, stock span, histogram all use it. Each index is pushed and

popped once, so O(n).

- Store **indices** on the stack, not values, so you can compute distances.
- Infix to postfix: operands go straight out; operators wait on a stack by precedence.
- Design problems (min stack, LRU, LFU) combine two structures: a list for order and a map for O(1) lookup.

Templates

STEP 9 TEMPLATES

Copy

```
// next greater element to the right (-1 if none), O(n)
vector<int> nge(n, -1);
stack<int> st;                                  // indices; their values are decreasing
for (int i = n - 1; i ⩾ 0; i--) {
    while (!st.empty() && a[st.top()] ⩽ a[i]) st.pop();
    if (!st.empty()) nge[i] = a[st.top()];
    st.push(i);
}

// LRU cache: the list keeps recency order, the map points into the list
list<pair<int,int>> order;                                  // front = most recent
unordered_map<int, list<pair<int,int>>::iterator> where;
int get(int k) {
    auto it = where.find(k);
    if (it == where.end()) return -1;
    order.splice(order.begin(), order, it→second);          // move to front in O(1)
    return it→second→second;
}
void put(int k, int v, int cap) {
    if (where.count(k)) { where[k]→second = v; get(k); return; }
    if ((int)order.size() == cap) { where.erase(order.back().first); order.pop_back(); }
    order.push_front({k, v});
    where[k] = order.begin();
}
```

// next greater element to the right

Traps

⚠ `top()` on an empty stack. Always check `!st.empty()` first in the while condition.

⚠ Choosing `<` vs `⩽` in the pop condition decides how equal values are treated; test with duplicates.

⚠ Erasing from the map but not the list (or the reverse) in LRU.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// IMPLEMENTATION		
Implement a stack using arrays	Easy	Mon 23 Nov
Implement a queue using arrays	Easy	Mon 23 Nov
Implement a stack using queues	Easy	Mon 23 Nov
Implement a queue using stacks	Easy	Mon 23 Nov
Implement a stack using a linked list	Easy	Mon 23 Nov
Implement a queue using a linked list	Easy	Mon 23 Nov
Check for balanced parentheses	Easy	Mon 23 Nov
Implement a min stack	Medium	Mon 23 Nov
// PREFIX, INFIX, POSTFIX		
Infix to postfix	Medium	Mon 23 Nov
Prefix to infix	Medium	Tue 24 Nov
Prefix to postfix	Medium	Tue 24 Nov
Postfix to prefix	Medium	Tue 24 Nov
Postfix to infix	Medium	Tue 24 Nov
Infix to prefix	Medium	Tue 24 Nov
// MONOTONIC STACK		
Next greater element I	Easy	Tue 24 Nov
Next greater element II (circular)	Medium	Tue 24 Nov
Next smaller element	Medium	Tue 24 Nov
Number of NGEs to the right	Hard	Wed 03 Feb (hard pass)
Trapping rain water	Hard	Thu 04 Feb (hard pass)
Sum of subarray minimums	Medium	Tue 24 Nov
Asteroid collision	Medium	Wed 25 Nov
Sum of subarray ranges	Medium	Wed 25 Nov
Remove K digits	Medium	Wed 25 Nov

PROBLEM	LEVEL	DUE
Largest rectangle in a histogram	Hard	Fri 05 Feb (hard pass)
Maximal rectangles	Hard	Sat 06 Feb (hard pass)
// IMPLEMENTATION PROBLEMS		
Sliding window maximum	Hard	Sun 07 Feb (hard pass)
Stock span problem	Medium	Wed 25 Nov
The celebrity problem	Medium	Wed 25 Nov
LRU cache	Hard	Mon 08 Feb (hard pass)
LFU cache	Hard	Tue 09 Feb (hard pass)

BUILD IT

calc and browser: two stack tools

A calculator that evaluates real expressions, and a browser-history simulator with an LRU page cache.

- ☐ calc: tokenise, convert infix to postfix (shunting-yard), evaluate, report errors like unbalanced brackets.
- ☐ browser: `visit`, `back`, `forward` with two stacks; a new visit clears the forward stack.
- ☐ An LRU cache of the last 3 pages, printing hit or miss on each visit.

▶ SAMPLE RUN

```
$ run calc.cpp
> 3 + 4 * (2 - 1)
postfix: 3 4 2 1 - * +
= 7
$ run browser.cpp
> visit a.com
> visit b.com
> back
a.com           (cache hit)
> forward
b.com           (cache hit)
> visit c.com
> forward
nothing to go forward to
```

Stretch. Support unary minus and powers (right-associative) in the calculator.

Resume line. *Built an expression evaluator and an LRU-cached browser-history simulator in C++.*

LeetCode companions

- **#20 Valid Parentheses** (Easy), Mon 23 Nov
- **#155 Min Stack** (Medium), Mon 23 Nov
- **#150 Evaluate Reverse Polish Notation** (Medium), Tue 24 Nov
- **#739 Daily Temperatures** (Medium), Tue 24 Nov
- **#503 Next Greater Element II** (Medium), Tue 01 Dec

STEP 10 | 25 NOV TO 26 NOV | HIGH PRIORITY

Step 10: Sliding window and two pointers

Why it matters. Turns many O(n squared) array and string problems into O(n).

Seen in real systems. Rate limiting APIs, detecting brute-force logins, moving averages on dashboards, network packet windows.

Key ideas

- **Variable window**: move `r` forward every step; while the window breaks the rule, move `l` forward. Update the answer when the window is valid.
- **Fixed window** of size k: add a[r], remove a[r-k].
- **Exactly K** trick: count(exactly K) = count(at most K) - count(at most K-1).
- Works when growing the window can only make it "more invalid" (monotonic). With negative numbers, use prefix sums instead.

Templates

STEP 10 TEMPLATES Copy

```
// longest window with at most k distinct characters
unordered_map<char,int> cnt;
int l = 0, best = 0;
for (int r = 0; r < (int)s.size(); r++) {
    cnt[s[r]]++;
    while ((int)cnt.size() > k) {                // invalid: too many distinct
        if (--cnt[s[l]] == 0) cnt.erase(s[l]);
        l++;
    }
```

```
    best = max(best, r - l + 1);                    // valid here
}

// subarrays with exactly k distinct = atMost(k) - atMost(k - 1)
long long atMost(const vector<int> &a, int k) {
    unordered_map<int,int> c; long long res = 0; int l = 0;
    for (int r = 0; r < (int)a.size(); r++) {
        if (c[a[r]]++ == 0) k--;
        while (k < 0) { if (--c[a[l]] == 0) k++; l++; }
        res += r - l + 1;                           // windows ending at r
    }
    return res;
}
```

```
// longest window
with at most k
```

Traps

- ⚠ Not erasing a key when its count hits 0 makes `cnt.size()` wrong.
- ⚠ Updating the answer while the window is still invalid.
- ⚠ Using a sliding window on arrays with negative numbers when the problem needs prefix sums.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// SLIDING WINDOW		
Maximum points you can obtain from cards	Medium	Wed 25 Nov
Longest substring without repeating characters	Medium	Wed 25 Nov
Max consecutive ones III	Medium	Wed 25 Nov
Fruit into baskets	Medium	Thu 26 Nov
Longest substring with at most K distinct characters	Medium	Thu 26 Nov
Longest repeating character replacement	Medium	Thu 26 Nov
Number of substrings containing all three characters	Medium	Thu 26 Nov
Binary subarrays with sum	Medium	Thu 26 Nov
Count number of nice subarrays	Medium	Thu 26 Nov
Subarrays with K different integers	Hard	Wed 10 Feb (hard pass)
Minimum window substring	Hard	Thu 11 Feb (hard pass)

BUILD IT

logwatch: a log-stream monitor

Reads a web server access log and spots traffic spikes and brute-force patterns with sliding windows. Strong cyber relevance.

- ☐ Parse lines like `14:06:12 10.0.4.17 POST /login 401`.
- ☐ Busiest 5-minute window (fixed window).
- ☐ Alert when one IP makes more than N requests in 60 seconds (variable window per IP).
- ☐ Longest calm stretch with at most 3 distinct IPs.
- ☐ Moving average of requests per minute.

▶ SAMPLE RUN

```
$ run logwatch.cpp < access.log
requests: 48211 over 24 h
busiest 5 minutes: 14:05:00 to 14:10:00 (1904 requests)
alerts (more than 20 requests in 60 s from one IP):
  10.0.4.17   14:06:12   37 failed logins   possible brute force
  10.0.9.2    22:41:50   23 requests
longest calm stretch with at most 3 IPs: 6 min 40 s
```

Stretch. Generate a fake log with a Python or C++ script so you can test with a million lines.

Resume line. *Built a C++ log monitor that flags brute-force login attempts with sliding-window rate detection.*

LeetCode companions

- **#3 Longest Substring Without Repeating Characters** (Medium), Wed 25 Nov
- **#424 Longest Repeating Character Replacement** (Medium), Wed 25 Nov
- **#1004 Max Consecutive Ones III** (Medium), Thu 26 Nov
- **#567 Permutation in String** (Medium), Thu 26 Nov
- **#209 Minimum Size Subarray Sum** (Medium), Wed 02 Dec
- **#438 Find All Anagrams in a String** (Medium), Thu 03 Dec
- **#76 Minimum Window Substring** (Hard), Fri 04 Dec

STEP 11 | 27 NOV TO 28 NOV | MEDIUM PRIORITY

Step 11: Heaps

Why it matters. Top-K and scheduling problems. Priority queue syntax in C++ is worth drilling.

Seen in real systems. OS and job schedulers, trending topics, Dijkstra in maps, event simulations.

Key ideas

- A heap gives the best item in O(1) and removes it in O(log n). It never sorts everything.
- **Top K**: keep a min-heap of size k. Anything smaller than its top cannot be in the top k. O(n log k).
- **Two heaps** (max-heap for the lower half, min-heap for the upper half) give a running median.
- **K-way merge**: push the head of each sorted list; pop the smallest and push its successor.
- Greedy plus heap: always take the cheapest or most urgent item (connect ropes, task scheduler).

Templates

STEP 11 TEMPLATES

Copy

```
// top k frequent: min-heap of (count, value), never larger than k
unordered_map<int,int> f; for (int x : a) f[x]++;
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
for (auto &[v, c] : f) { pq.push({c, v}); if ((int)pq.size() > k) pq.pop(); }

// running median: lower half in a max-heap, upper half in a min-heap
priority_queue<int> lo;
priority_queue<int, vector<int>, greater<int>> hi;
void add(int x) {
    lo.push(x); hi.push(lo.top()); lo.pop();       // pass the largest of lo to hi
    if (hi.size() > lo.size()) { lo.push(hi.top()); hi.pop(); }
}
double median() { return lo.size() > hi.size() ? lo.top() : (1.0 * lo.top() + hi.top()) /
2; }
```

```
// top k frequent:
min-heap of (count,
```

Traps

- ⚠ Default `priority_queue` is a max-heap; top-K smallest wants a max-heap of size k, top-K largest a min-heap.
- ⚠ A custom comparator struct returns true when a should come **after** b (the reverse of sort).
- ⚠ Pushing n items then popping k is O(n log n); the size-k heap is O(n log k).

Every problem, with its due date

PROBLEM	LEVEL	DUE
// BASICS		
Introduction to a priority queue	Easy	Fri 27 Nov
Implement a min heap	Medium	Fri 27 Nov
Implement a max heap	Medium	Fri 27 Nov
Heapify and build a heap	Medium	Fri 27 Nov
Check if an array represents a min heap	Easy	Fri 27 Nov
Convert a min heap to a max heap	Medium	Fri 27 Nov
Heap sort	Medium	Fri 27 Nov
// PROBLEMS		
Kth largest element in an array	Medium	Fri 27 Nov
Kth smallest element	Medium	Fri 27 Nov
Sort a k-sorted array	Medium	Fri 27 Nov
Merge K sorted arrays	Medium	Fri 27 Nov
Replace elements by their rank	Medium	Fri 27 Nov
Task scheduler	Medium	Fri 27 Nov
Hand of straights	Medium	Sat 28 Nov
Design Twitter	Medium	Sat 28 Nov
Connect N ropes	Medium	Sat 28 Nov
Kth largest element in a stream	Medium	Sat 28 Nov
Top K frequent elements	Medium	Sat 28 Nov
Maximum sum combination	Hard	Fri 12 Feb (hard pass)
Find median from a data stream	Hard	Sat 13 Feb (hard pass)

BUILD IT

trending and scheduler: heap tools

A trending-words counter and a priority job scheduler, the two classic heap jobs.

- ☐ trending: top K words in a text file, ignoring common stop words.
- ☐ scheduler: jobs arrive with priority and duration; run highest priority first, earliest arrival on ties.
- ☐ Report the running median of waiting times with two heaps.

▶ SAMPLE RUN

```
$ run trending.cpp 5 < posts.txt
top 5: placement 41, leetcode 33, offer 29, interview 24, striver 19
$ run scheduler.cpp < jobs.txt
t=0  run backup  (p9)
t=3  run email   (p7)   arrived t=1
t=4  run report  (p7)   arrived t=2   tie: earlier arrival first
median wait so far: 1.5 s
```

Stretch. Add aging: a job's priority rises the longer it waits, so low-priority jobs cannot starve.

Resume line. *Implemented a priority job scheduler and streaming top-K counter in C++ with binary heaps.*

LeetCode companions

- **#1046 Last Stone Weight** (Easy), Fri 27 Nov
- **#703 Kth Largest Element in a Stream** (Easy), Fri 27 Nov
- **#215 Kth Largest Element in an Array** (Medium), Sat 05 Dec

STEP 12 | 28 NOV TO 29 NOV | MEDIUM PRIORITY

Step 12: Greedy algorithms

Why it matters. Mostly interval and scheduling problems. Prove why greedy works before you code.

Seen in real systems. Meeting schedulers, CPU scheduling (shortest job first), cash dispensers, bandwidth allocation.

Key ideas

- Greedy makes the best local choice and never undoes it. It is right only when you can argue no better choice exists (the exchange argument).
- Intervals: sort by **end** time to fit the most; sort by **start** to merge.

- Minimum platforms or rooms: sort arrivals and departures separately and sweep.
- When greedy fails (coins {1, 3, 4} for 6: greedy gives 4+1+1, best is 3+3), the problem needs DP.

Templates

STEP 12 TEMPLATES

Copy

```cpp
// most meetings in one room: sort by end, take each that starts after the last end
sort(m.begin(), m.end(), [](const pair<int,int> &a, const pair<int,int> &b) {
    return a.second < b.second;                        // (start, end)
});
int taken = 0, lastEnd = INT_MIN;
for (auto &[s, e] : m) if (s > lastEnd) { taken++; lastEnd = e; }

// jump game II: fewest jumps, BFS by ranges, O(n)
int jumps = 0, curEnd = 0, far = 0;
for (int i = 0; i + 1 < n; i++) {
    far = max(far, i + a[i]);
    if (i == curEnd) { jumps++; curEnd = far; }
}
```

```
// most meetings in
one room: sort by
```

Traps

⚠ Sorting intervals by start when the problem needs end (or the reverse).

⚠ A comparator with `⩽`.

⚠ Assuming greedy works without testing a counterexample.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// EASY AND MEDIUM		
Assign cookies	Easy	Sat 28 Nov
Fractional knapsack	Medium	Sat 28 Nov
Minimum coins (greedy)	Easy	Sat 28 Nov
Lemonade change	Easy	Sat 28 Nov
Valid parenthesis string	Medium	Sat 28 Nov
Jump game	Medium	Sun 29 Nov
Jump game II	Medium	Sun 29 Nov

PROBLEM	LEVEL	DUE
Job sequencing problem	Medium	Sun 29 Nov
N meetings in one room	Easy	Sun 29 Nov
Minimum number of platforms	Medium	Sun 29 Nov
Shortest job first scheduling	Easy	Sun 29 Nov
Page faults in LRU	Easy	Sun 29 Nov
Non-overlapping intervals	Medium	Sun 29 Nov
Insert interval	Medium	Sun 29 Nov
// HARD		
Candy	Hard	Sun 14 Feb (hard pass)
Merge intervals (revisit)	Medium	Sun 29 Nov

BUILD IT

planner and atm: greedy tools

A meeting planner for a week of requests and a cash-machine note dispenser, including the case where greedy breaks.

- ☐ Most meetings one person can attend (sort by end).
- ☐ Rooms needed so no meeting waits (platforms sweep).
- ☐ Merge overlapping busy blocks into a clean calendar.
- ☐ ATM: dispense an amount with the fewest notes from 500, 200, 100, 50, 20, 10.
- ☐ Show a coin system where greedy fails and print the DP answer next to it.

▶ SAMPLE RUN

```
$ run planner.cpp < week.txt
meetings requested: 9
most you can attend in one room: 5
rooms needed so nobody waits: 3
busy blocks: 09:00-10:30, 11:00-13:00, 14:00-17:30
$ run atm.cpp 2780
500 x5, 200 x1, 50 x1, 20 x1, 10 x1   (9 notes)
coins {1,3,4} for 6: greedy 3 coins, optimal 2 (3+3)
```

Stretch. Read meeting times from an `.ics` calendar export.

Resume line. *Built a greedy meeting planner and cash dispenser in C++, with a DP fallback where greedy fails.*

LeetCode companions

- **#455 Assign Cookies** (Easy), Sat 28 Nov
- **#55 Jump Game** (Medium), Sat 28 Nov

STEP 13 | 30 NOV TO 02 DEC | HIGH PRIORITY

Step 13: Binary trees

Why it matters. Interviewers love trees. Recursion plus a clear return value is the whole trick.

Seen in real systems. File systems, the HTML DOM, JSON documents, decision trees, organisation charts.

Key ideas

- Decide what each call **returns** (height, sum, found or not) and what it **receives** from the parent (depth, path so far).
- Many answers are updated while returning: diameter = max over nodes of left height + right height.
- Level order uses a queue and processes one level per loop by reading `q.size()` first.
- Views (top, bottom, vertical) attach a column number: left child is col - 1, right is col + 1.
- Rebuild a tree from preorder + inorder: preorder's first item is the root; find it in inorder to split left and right.

Templates

STEP 13 TEMPLATES Copy

```
// height, with the diameter updated on the way back up
int diameter = 0;
int height(TreeNode *root) {
    if (!root) return 0;
    int l = height(root->left), r = height(root->right);
    diameter = max(diameter, l + r);              // longest path through this node, in
edges
    return 1 + max(l, r);
}
```

```cpp
// level order traversal
vector<vector<int>> levels(TreeNode *root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        res.push_back({});
        while (sz--) {
            TreeNode *t = q.front(); q.pop();
            res.back().push_back(t->val);
            if (t->left) q.push(t->left);
            if (t->right) q.push(t->right);
        }
    }
    return res;
}

// height, with the
diameter updated on
```

Traps

- ⚠ Not resetting globals like `diameter` between test cases.
- ⚠ Height defined as nodes in one place and edges in another; be explicit.
- ⚠ Recursion depth on a skewed tree of 1e5 nodes can overflow the stack.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// TRAVERSALS		
Introduction to trees and representation	Easy	Mon 30 Nov
Preorder traversal	Easy	Mon 30 Nov
Inorder traversal	Easy	Mon 30 Nov
Postorder traversal	Easy	Mon 30 Nov
Level order traversal	Medium	Mon 30 Nov
Iterative preorder	Medium	Mon 30 Nov
Iterative inorder	Medium	Mon 30 Nov
Iterative postorder (two stacks)	Medium	Mon 30 Nov
Iterative postorder (one stack)	Medium	Mon 30 Nov

PROBLEM	LEVEL	DUE
Preorder, inorder and postorder in one traversal	Hard	Mon 15 Feb (hard pass)
// MEDIUM PROBLEMS		
Maximum depth of a binary tree	Easy	Tue 01 Dec
Check for a balanced binary tree	Medium	Tue 01 Dec
Diameter of a binary tree	Medium	Tue 01 Dec
Maximum path sum	Hard	Tue 16 Feb (hard pass)
Check if two trees are identical	Easy	Tue 01 Dec
Zigzag traversal	Medium	Tue 01 Dec
Boundary traversal	Medium	Tue 01 Dec
Vertical order traversal	Hard	Wed 17 Feb (hard pass)
Top view	Medium	Tue 01 Dec
Bottom view	Medium	Tue 01 Dec
Right and left view	Medium	Tue 01 Dec
Symmetric binary tree	Easy	Wed 02 Dec
// HARD PROBLEMS		
Root to node path	Medium	Wed 02 Dec
Lowest common ancestor	Medium	Wed 02 Dec
Maximum width of a binary tree	Medium	Wed 02 Dec
Children sum property	Medium	Wed 02 Dec
Print all nodes at distance K	Hard	Thu 18 Feb (hard pass)
Minimum time to burn a tree	Hard	Fri 19 Feb (hard pass)
Count nodes in a complete binary tree	Medium	Wed 02 Dec
Construct a tree from preorder and inorder	Medium	Wed 02 Dec
Construct a tree from postorder and inorder	Medium	Wed 02 Dec
Serialise and deserialise a binary tree	Hard	Sat 20 Feb (hard pass)
Morris inorder traversal	Hard	Sun 21 Feb (hard pass)

PROBLEM	LEVEL	DUE
Morris preorder traversal	Hard	Mon 22 Feb (hard pass)
Flatten a binary tree to a linked list	Medium	Wed 02 Dec

BUILD IT

fstree: a file-system explorer

Builds a directory tree from a list of paths and answers the questions `du`, `tree` and `find` answer.

- ☐ Read lines like `/home/kvp/dsa/a2z/arrays.cpp 4` (path and size in KB).
- ☐ Print the tree with indentation (preorder).
- ☐ Folder sizes computed bottom-up (postorder).
- ☐ Deepest path and its depth (height).
- ☐ Lowest common folder of two paths (LCA).

▶ SAMPLE RUN

```
$ run fstree.cpp < paths.txt
/
  home/
    kvp/                  (196 KB)
      dsa/                (184 KB)
      notes.txt           (12 KB)
  etc/
    hosts                 (1 KB)
deepest: /home/kvp/dsa/a2z/13-trees/lca.cpp (depth 6)
lowest common folder of /home/kvp/dsa/x.cpp and /home/kvp/notes.txt:
/home/kvp
```

Stretch. Run it on your real disk: `find ~/Documents -type f` piped into the program.

Resume line. *Wrote a C++ file-system explorer computing folder sizes and lowest common ancestors over a tree.*

LeetCode companions

- **#104 Maximum Depth of Binary Tree** (Easy), Mon 30 Nov
- **#226 Invert Binary Tree** (Easy), Mon 30 Nov
- **#543 Diameter of Binary Tree** (Easy), Tue 01 Dec
- **#102 Binary Tree Level Order Traversal** (Medium), Tue 01 Dec
- **#199 Binary Tree Right Side View** (Medium), Wed 02 Dec

- **#236 Lowest Common Ancestor of a Binary Tree** (Medium), Wed 02 Dec

STEP 14 | 03 DEC TO 04 DEC | MEDIUM PRIORITY

Step 14: Binary search trees

Why it matters. Inorder gives sorted order. Most problems reduce to that.

Seen in real systems. Ordered indexes in databases (B-trees are wide BSTs), `std::map` and `std::set` (red-black trees).

Key ideas

- Left subtree < node < right subtree, for every node, not just parent and child.
- Inorder traversal of a BST is sorted. Kth smallest, validation and recover-BST all use that.
- Search, insert, floor and ceil walk one root-to-leaf path: O(height).
- Delete has three cases: leaf, one child, two children (replace with the inorder successor).

Templates

STEP 14 TEMPLATES

Copy

```
// validate: each node must sit inside the range its ancestors allow
bool valid(TreeNode *r, long long lo = LLONG_MIN, long long hi = LLONG_MAX) {
    if (!r) return true;
    if (r->val ⩽ lo || r->val ⩾ hi) return false;
    return valid(r->left, lo, r->val) && valid(r->right, r->val, hi);
}

TreeNode* insert(TreeNode *r, int x) {
    if (!r) return new TreeNode(x);
    if (x < r->val) r->left = insert(r->left, x);
    else r->right = insert(r->right, x);
    return r;
}

int ceilBST(TreeNode *r, int x) {               // smallest value ⩾ x, or -1
    int ans = -1;
    while (r) {
        if (r->val == x) return x;
        if (r->val > x) { ans = r->val; r = r->left; } else r = r->right;
    }
    return ans;
}
```

```
// validate: each
node must sit inside
```

Traps

- ⚠ Checking only `left→val < node→val` misses a deep node that breaks the rule.
- ⚠ Using `INT_MIN` / `INT_MAX` as bounds fails when the tree contains those values; use long long.
- ⚠ An unbalanced BST degrades to a linked list (O(n) per operation).

Every problem, with its due date

PROBLEM	LEVEL	DUE
// BASICS AND PROBLEMS		
Introduction to a BST	Easy	Thu 03 Dec
Search in a BST	Easy	Thu 03 Dec
Minimum and maximum in a BST	Easy	Thu 03 Dec
Ceil in a BST	Medium	Thu 03 Dec
Floor in a BST	Medium	Thu 03 Dec
Insert a node	Medium	Thu 03 Dec
Delete a node	Hard	Tue 23 Feb (hard pass)
Kth smallest element	Medium	Thu 03 Dec
Kth largest element	Medium	Thu 03 Dec
Check if a tree is a BST	Medium	Thu 03 Dec
Lowest common ancestor in a BST	Medium	Fri 04 Dec
Construct a BST from preorder	Medium	Fri 04 Dec
Inorder successor and predecessor	Medium	Fri 04 Dec
BST iterator	Medium	Fri 04 Dec
Two sum IV in a BST	Medium	Fri 04 Dec
Recover a BST	Hard	Wed 24 Feb (hard pass)
Largest BST in a binary tree	Hard	Thu 25 Feb (hard pass)

BUILD IT

contacts: a BST contacts book

A contacts store on your own BST, compared at the end with `std::map`.

- ☐ `add`, `find`, `delete` by name.
- ☐ `list` alphabetically (inorder).
- ☐ `range A K`: every name between two letters.
- ☐ `kth n`: the nth name alphabetically.
- ☐ Report tree height after inserting sorted names, to show why balancing matters.

▶ SAMPLE RUN

```
$ run contacts.cpp
> add Ravi
> add Anu
> add Kiran
> list
Anu, Kiran, Ravi
> range K S
Kiran, Ravi
> kth 2
Kiran
> delete Kiran
> list
Anu, Ravi
```

Stretch. Load 10000 names in sorted order, print the height, then load them shuffled and compare.

Resume line. *Implemented a BST-backed contacts store in C++ with range queries and order statistics.*

LeetCode companions

- **#700 Search in a Binary Search Tree** (Easy), Thu 03 Dec
- **#98 Validate Binary Search Tree** (Medium), Thu 03 Dec

STEP 15 | 04 DEC TO 08 DEC | HIGH PRIORITY

Step 15: Graphs

Why it matters. BFS and DFS on grids appear everywhere. Shortest paths and topological sort follow.

Seen in real systems. Maps and ride-hailing routes, social networks, build systems (topological order), network reachability in security audits.

Key ideas

- Store graphs as an adjacency list: `vector<vector<int>> adj(n)` (or pairs for weights).
- **BFS** gives the fewest edges in an unweighted graph; mark visited when you push, not when you pop.
- **DFS** finds components, cycles and topological order. Grids are graphs with 4 neighbours.
- **Topological sort** (Kahn): repeatedly take nodes with in-degree 0. If some never reach 0, there is a cycle.
- **Dijkstra** for non-negative weights; **Bellman-Ford** when there are negative edges; **Floyd-Warshall** for all pairs on small graphs.
- **DSU** (union-find) answers "are these connected?" as edges arrive; Kruskal's MST uses it.

Templates

STEP 15 TEMPLATES

Copy

```
// BFS: fewest edges from src
vector<int> dist(n, -1); queue<int> q;
dist[src] = 0; q.push(src);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
}

// Dijkstra: adj[u] holds (v, w); no negative weights
vector<long long> d(n, LLONG_MAX);
priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
d[src] = 0; pq.push({0, src});
while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du > d[u]) continue;                                  // stale entry
    for (auto [v, w] : wadj[u])
        if (du + w < d[v]) { d[v] = du + w; pq.push({d[v], v}); }
}

// disjoint set union with path compression and union by rank
struct DSU {
    vector<int> p, r;
    DSU(int n) : p(n), r(n, 0) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
```

```
        if (a == b) return false;
        if (r[a] < r[b]) swap(a, b);
        p[b] = a; if (r[a] == r[b]) r[a]++;
        return true;
    }
};

// BFS: fewest edges
from src
```

Traps

⚠ Marking visited on pop in BFS lets a node enter the queue many times.

⚠ Dijkstra with a negative edge gives wrong answers silently.

⚠ Grid bounds: check `0 ⩽ nr < rows` before reading `grid[nr][nc]`.

⚠ Directed vs undirected: add the reverse edge only for undirected graphs.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// BFS AND DFS		
Graph representation (matrix and list)	Easy	Fri 04 Dec
BFS traversal	Easy	Fri 04 Dec
DFS traversal	Easy	Fri 04 Dec
Number of provinces	Medium	Fri 04 Dec
Connected components	Medium	Fri 04 Dec
Number of islands	Medium	Fri 04 Dec
Flood fill	Medium	Sat 05 Dec
Rotten oranges	Medium	Sat 05 Dec
Detect a cycle in an undirected graph (BFS)	Medium	Sat 05 Dec
Detect a cycle in an undirected graph (DFS)	Medium	Sat 05 Dec
Distance of the nearest 1 in a 0/1 matrix	Medium	Sat 05 Dec
Surrounded regions	Medium	Sat 05 Dec
Number of enclaves	Medium	Sat 05 Dec
Word ladder I	Hard	Fri 26 Feb (hard pass)

PROBLEM	LEVEL	DUE
Word ladder II	Hard	Sat 27 Feb (hard pass)
Number of distinct islands	Medium	Sat 05 Dec
Bipartite graph	Medium	Sat 05 Dec
// TOPOLOGICAL SORT		
Detect a cycle in a directed graph (DFS)	Medium	Sun 06 Dec
Find eventual safe states	Medium	Sun 06 Dec
Topological sort (DFS)	Medium	Sun 06 Dec
Kahn's algorithm (BFS)	Medium	Sun 06 Dec
Cycle detection in a directed graph (BFS)	Medium	Sun 06 Dec
Course schedule I	Medium	Sun 06 Dec
Course schedule II	Medium	Sun 06 Dec
Alien dictionary	Hard	Sun 28 Feb (hard pass)
// SHORTEST PATHS		
Shortest path in an undirected graph with unit weights	Medium	Sun 06 Dec
Shortest path in a DAG	Medium	Sun 06 Dec
Dijkstra's algorithm	Medium	Sun 06 Dec
Print the shortest path	Hard	Mon 01 Mar (hard pass)
Shortest path in a binary maze	Medium	Mon 07 Dec
Path with minimum effort	Medium	Mon 07 Dec
Cheapest flights within K stops	Medium	Mon 07 Dec
Network delay time	Medium	Mon 07 Dec
Minimum multiplications to reach the end	Medium	Mon 07 Dec
Number of ways to arrive at the destination	Medium	Mon 07 Dec
Bellman-Ford algorithm	Medium	Mon 07 Dec
Floyd-Warshall algorithm	Medium	Mon 07 Dec
City with the smallest number of neighbours at a threshold	Medium	Mon 07 Dec

PROBLEM	LEVEL	DUE
// SPANNING TREES AND DISJOINT SETS		
Minimum spanning tree: Prim's algorithm	Medium	Mon 07 Dec
Disjoint set (rank and path compression)	Medium	Tue 08 Dec
Kruskal's algorithm	Medium	Tue 08 Dec
Number of operations to make a network connected	Medium	Tue 08 Dec
Most stones removed	Medium	Tue 08 Dec
Accounts merge	Medium	Tue 08 Dec
Number of islands II	Hard	Tue 02 Mar (hard pass)
Making a large island	Hard	Wed 03 Mar (hard pass)
Swim in rising water	Hard	Thu 04 Mar (hard pass)
// OTHER ALGORITHMS		
Bridges in a graph	Hard	Fri 05 Mar (hard pass)
Articulation points	Hard	Sat 06 Mar (hard pass)
Kosaraju's algorithm (strongly connected components)	Hard	Sun 07 Mar (hard pass)

BUILD IT

routes and netscan: graph tools

A city route planner and a network reachability scanner. The second one maps to a real security task: what can an attacker reach from one compromised host?

- ☐ routes: read `A B minutes` edges; shortest route and its path (Dijkstra with parent array).
- ☐ netscan: hosts and links; BFS from a start host prints hosts by hop count.
- ☐ Connected components: isolated hosts.
- ☐ Articulation points: single points of failure in the network.
- ☐ Topological order of a build or course plan as a bonus.

▶ SAMPLE RUN

```
$ run routes.cpp < navimumbai.txt
> route Nerul CSMT
shortest: 52 min   Nerul → Vashi → Kurla → CSMT
$ run netscan.cpp < lab.txt
start: 10.0.0.5
```

```
hop 1: 10.0.0.1  10.0.0.9
hop 2: 10.0.1.4  10.0.1.7
hop 3: 10.0.2.2  10.0.2.8  10.0.2.9
isolated: 10.0.3.3
single point of failure: 10.0.0.1
```

Stretch. Feed netscan with real data: the hosts `nmap -sn` finds on your own home network.

Resume line. *Built a C++ network reachability scanner (BFS, articulation points) and a Dijkstra route planner.*

LeetCode companions

- **#200 Number of Islands** (Medium), Fri 04 Dec
- **#994 Rotting Oranges** (Medium), Fri 04 Dec
- **#207 Course Schedule** (Medium), Sat 05 Dec
- **#133 Clone Graph** (Medium), Sat 05 Dec

STEP 16 | 08 DEC TO 12 DEC | HIGH PRIORITY

Step 16: Dynamic programming

Why it matters. The most feared topic and the best differentiator. Always go recursion, memoisation, tabulation.

Seen in real systems. `diff` and `git` use LCS; spell checkers use edit distance; DNA alignment, route pricing, resource allocation.

Key ideas

- Write the **recursion** first: define f(state) in words, list choices, combine them.
- **Memoise**: same recursion, store answers in `dp[state]`. Time = states x work per state.
- **Tabulate**: fill dp from base cases upward in the order the recursion needs.
- **Space-optimise**: keep only the rows or values the next step reads.
- Families: 1D (stairs, robber), grids (paths), pick/not-pick (subset sum, knapsack), two strings (LCS, edit distance), stocks (state = holding or not), LIS, partition (matrix chain).

Templates

STEP 16 TEMPLATES Copy

```cpp
// house robber: the four rungs of the DP ladder
// 1. recursion:   f(i) = max(a[i] + f(i - 2), f(i - 1))
// 2. memo:        same recursion, cache f(i) in dp[i]
// 3. tabulation:
vector<long long> dp(n);
dp[0] = a[0];
for (int i = 1; i < n; i++) dp[i] = max(dp[i - 1], a[i] + (i ⩾ 2 ? dp[i - 2] : 0));
// 4. space-optimised:
long long prev2 = 0, prev1 = 0;
for (int x : a) { long long cur = max(prev1, x + prev2); prev2 = prev1; prev1 = cur; }

// LCS: shift indices by one so row 0 and column 0 mean "empty prefix"
vector<vector<int>> L(n + 1, vector<int>(m + 1, 0));
for (int i = 1; i ⩽ n; i++)
    for (int j = 1; j ⩽ m; j++)
        L[i][j] = (s[i - 1] == t[j - 1]) ? 1 + L[i - 1][j - 1] : max(L[i - 1][j], L[i][j -
1]);
```

```
// house robber: the
four rungs of the DP
```

Traps

- ⚠ Initialising the memo with -1 when -1 is also a valid answer.
- ⚠ Wrong base case (dp[0] vs dp[1]) shifts every answer by one.
- ⚠ Jumping straight to tabulation without the recursion; you lose the ability to derive the next problem.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// 1D DP		
Introduction to DP (Fibonacci with memoisation)	Easy	Tue 08 Dec
Climbing stairs	Easy	Tue 08 Dec
Frog jump	Easy	Tue 08 Dec
Frog jump with K distances	Medium	Tue 08 Dec
Maximum sum of non-adjacent elements	Medium	Wed 09 Dec
House robber II	Medium	Wed 09 Dec
// 2D AND GRID DP		
Ninja's training	Medium	Wed 09 Dec
Grid unique paths	Medium	Wed 09 Dec

PROBLEM	LEVEL	DUE
Unique paths II (obstacles)	Medium	Wed 09 Dec
Minimum path sum in a grid	Medium	Wed 09 Dec
Triangle minimum path sum	Medium	Wed 09 Dec
Minimum and maximum falling path sum	Medium	Wed 09 Dec
Cherry pickup II	Hard	Mon 08 Mar (hard pass)
// SUBSEQUENCES AND KNAPSACK		
Subset sum equal to target	Medium	Wed 09 Dec
Partition equal subset sum	Medium	Thu 10 Dec
Partition into two subsets with minimum absolute difference	Medium	Thu 10 Dec
Count subsets with sum K	Medium	Thu 10 Dec
Count partitions with a given difference	Medium	Thu 10 Dec
0/1 knapsack	Medium	Thu 10 Dec
Minimum coins	Medium	Thu 10 Dec
Target sum	Medium	Thu 10 Dec
Coin change II	Medium	Thu 10 Dec
Unbounded knapsack	Medium	Thu 10 Dec
Rod cutting	Medium	Fri 11 Dec
// STRINGS		
Longest common subsequence	Medium	Fri 11 Dec
Print the LCS	Medium	Fri 11 Dec
Longest common substring	Medium	Fri 11 Dec
Longest palindromic subsequence	Medium	Fri 11 Dec
Minimum insertions to make a string a palindrome	Medium	Fri 11 Dec
Minimum insertions and deletions to convert a string	Medium	Fri 11 Dec

PROBLEM	LEVEL	DUE
Shortest common supersequence	Hard	Tue 09 Mar (hard pass)
Distinct subsequences	Hard	Wed 10 Mar (hard pass)
Edit distance	Hard	Thu 11 Mar (hard pass)
Wildcard matching	Hard	Fri 12 Mar (hard pass)
// STOCKS		
Best time to buy and sell stock II	Medium	Fri 11 Dec
Best time to buy and sell stock III	Hard	Sat 13 Mar (hard pass)
Best time to buy and sell stock IV	Hard	Sun 14 Mar (hard pass)
Stock with cooldown	Medium	Fri 11 Dec
Stock with transaction fee	Medium	Sat 12 Dec
// LONGEST INCREASING SUBSEQUENCE		
Longest increasing subsequence	Medium	Sat 12 Dec
Print the LIS	Medium	Sat 12 Dec
LIS using binary search	Hard	Mon 15 Mar (hard pass)
Largest divisible subset	Medium	Sat 12 Dec
Longest string chain	Medium	Sat 12 Dec
Longest bitonic subsequence	Medium	Sat 12 Dec
Number of longest increasing subsequences	Medium	Sat 12 Dec
// PARTITION DP AND SQUARES		
Matrix chain multiplication	Hard	Tue 16 Mar (hard pass)
Minimum cost to cut a stick	Hard	Wed 17 Mar (hard pass)
Burst balloons	Hard	Thu 18 Mar (hard pass)

PROBLEM	LEVEL	DUE
Evaluate a boolean expression to true	Hard	Fri 19 Mar (hard pass)
Palindrome partitioning II	Hard	Sat 20 Mar (hard pass)
Partition array for maximum sum	Medium	Sat 12 Dec
Maximum rectangle area with all 1s	Hard	Sun 21 Mar (hard pass)
Count square submatrices with all 1s	Medium	Sat 12 Dec

BUILD IT

mydiff and spell: DP tools

A line diff like `git diff`, built on LCS, and a spelling suggester built on edit distance.

- ☐ mydiff: compare two files line by line; print kept, removed (-) and added (+) lines.
- ☐ Report the LCS length and how many lines changed.
- ☐ spell: load a word list; for a typo, show the 3 closest words by edit distance.
- ☐ Speed: filter candidates by length first, so it stays fast on 100000 words.

▶ SAMPLE RUN

```
$ run mydiff.cpp old.cpp new.cpp
  int main() {
-     int x = 5;
+     long long x = 5;
      cout << x;
+     return 0;
  }
LCS: 3 lines kept, 1 removed, 2 added
$ run spell.cpp words.txt
> placment
did you mean: placement (1), placements (2)
```

Stretch. Add word-level diff inside changed lines, like GitHub's highlighting.

Resume line. *Implemented a line diff (LCS) and a spelling suggester (edit distance) in C++ with dynamic programming.*

STEP 17 | SUN 13 DEC | LOW PRIORITY

Step 17: Tries

Why it matters. Rarely asked in campus tests. Do it after the high-priority steps are solid.

Seen in real systems. Autocomplete, spell checkers, IP routing tables (longest prefix match).

Key ideas

- Each node has up to 26 children and a flag for "a word ends here".
- Insert and search cost O(length of the word), independent of how many words are stored.
- Keep a counter of how many words pass through each node to answer prefix counts.
- A binary trie over bits finds the maximum XOR pair in O(n x 32).

Templates

STEP 17 TEMPLATES

Copy

```
struct Node { Node *ch[26] = {}; bool end = false; int pass = 0; };
Node *root = new Node();

void insert(const string &w) {
    Node *cur = root;
    for (char c : w) {
        int i = c - 'a';
        if (!cur->ch[i]) cur->ch[i] = new Node();
        cur = cur->ch[i];
        cur->pass++;
    }
    cur->end = true;
}

int countPrefix(const string &p) {
    Node *cur = root;
    for (char c : p) { cur = cur->ch[c - 'a']; if (!cur) return 0; }
    return cur->pass;
}
```

```
struct Node { Node
*ch[26] = {}; bool
```

Traps

⚠ Characters outside a to z index out of the array.

⚠ Memory: a million long words can mean tens of millions of nodes.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// TRIES		
Implement a trie (prefix tree) I	Medium	Sun 13 Dec
Implement a trie II	Medium	Sun 13 Dec
Longest word with all prefixes	Medium	Sun 13 Dec
Number of distinct substrings in a string using a trie	Medium	Sun 13 Dec
Maximum XOR of two numbers in an array	Hard	Mon 22 Mar (hard pass)
Maximum XOR with an element from an array	Hard	Tue 23 Mar (hard pass)

BUILD IT

autocomplete: a suggestion engine

Type a prefix, get the five most frequent words that start with it.

- ☐ Load words with frequencies.
- ☐ Store the top 5 words at each node, or DFS from the prefix node and use a heap.
- ☐ Report how many words share the prefix.

▶ SAMPLE RUN

```
$ run autocomplete.cpp words_with_counts.txt
> pla
placement (912)  plan (640)  platform (233)  play (198)  plate (75)
> xq
no words start with "xq"
```

Stretch. Add fuzzy matching: allow one typo by combining the trie with edit distance.

Resume line. *Built a trie-based autocomplete engine in C++ ranking suggestions by frequency.*

STEP 18 | SUN 13 DEC | LOW PRIORITY

Step 18: Strings (advanced)

Why it matters. KMP and the Z-function are bonus material for product-company interviews.

Seen in real systems. `grep` and text editors' search, intrusion-detection systems matching attack signatures in packets (Snort, Suricata).

Key ideas

- The naive search re-checks characters after a mismatch: O(n x m).
- KMP precomputes the LPS array (longest proper prefix that is also a suffix) so it never moves backwards in the text: O(n + m).
- The Z-function gives, for each position, the length of the longest match with the prefix.
- Rabin-Karp compares rolling hashes; use a large mod (or two) to make collisions unlikely.

Templates

STEP 18 TEMPLATES

Copy

```
// lps[i] = length of the longest proper prefix of p[0..i] that is also its suffix
vector<int> buildLps(const string &p) {
    vector<int> lps(p.size(), 0);
    int len = 0;
    for (int i = 1; i < (int)p.size(); ) {
        if (p[i] == p[len]) lps[i++] = ++len;
        else if (len) len = lps[len - 1];
        else lps[i++] = 0;
    }
    return lps;
}

vector<int> kmp(const string &s, const string &p) {      // every start of p in s
    vector<int> lps = buildLps(p), res;
    int j = 0;
    for (int i = 0; i < (int)s.size(); ) {
        if (s[i] == p[j]) {
            i++; j++;
            if (j == (int)p.size()) { res.push_back(i - j); j = lps[j - 1]; }
        } else if (j) j = lps[j - 1];
        else i++;
    }
    return res;
}
```

```
// lps[i] = length of
the longest proper
```

Traps

⚠ Moving `i` backwards in the text: KMP never does.

⚠ Hash collisions in Rabin-Karp: always confirm a hash match character by character.

Every problem, with its due date

PROBLEM	LEVEL	DUE
// ADVANCED STRINGS		
Shortest palindrome	Hard	Wed 24 Mar (hard pass)
Rabin-Karp algorithm	Hard	Thu 25 Mar (hard pass)
Z-function	Hard	Fri 26 Mar (hard pass)
KMP algorithm and the LPS array	Hard	Sat 27 Mar (hard pass)
Longest happy prefix	Hard	Sun 28 Mar (hard pass)
Count and say	Medium	Sun 13 Dec
Minimum bracket reversals to balance an expression	Medium	Sun 13 Dec

BUILD IT

kgrep: a mini grep

Searches files for a pattern with KMP and reports every match, with a speed comparison against the naive method.

- ☐ `kgrep pattern file` : print line numbers and positions.
- ☐ Case-insensitive flag.
- ☐ Count matches and time KMP vs naive on a large file.
- ☐ Signature mode: load several attack strings (like `' OR 1=1`) and scan a log for them.

▶ SAMPLE RUN

```
$ run kgrep.cpp "sql injection" notes.txt
12: practise sql injection on the lab
47: blind sql injection uses time delays
2 matches. KMP 0.8 ms, naive 3.1 ms on 5 MB
```

Stretch. Aho-Corasick: match many signatures in one pass (research it once KMP is solid).

Resume line. *Wrote a KMP-based signature scanner in C++ that detects injection strings in web logs.*

PART 02B | 4 JAN TO 28 MAR

The hard pass

All 84 Hard problems, about one a day after the core pass.

DUE	STEP	HARD PROBLEM
Mon 04 Jan	3. Arrays	Pascal's triangle
Tue 05 Jan	3. Arrays	Majority element (more than n/3)
Wed 06 Jan	3. Arrays	3-sum
Thu 07 Jan	3. Arrays	4-sum
Fri 08 Jan	3. Arrays	Largest subarray with sum 0
Sat 09 Jan	3. Arrays	Count subarrays with XOR K
Sun 10 Jan	3. Arrays	Merge overlapping intervals
Mon 11 Jan	3. Arrays	Merge two sorted arrays without extra space
Tue 12 Jan	3. Arrays	Find the repeating and missing number
Wed 13 Jan	3. Arrays	Count inversions
Thu 14 Jan	3. Arrays	Reverse pairs
Fri 15 Jan	3. Arrays	Maximum product subarray
Sat 16 Jan	4. Binary search	Aggressive cows
Sun 17 Jan	4. Binary search	Book allocation
Mon 18 Jan	4. Binary search	Split array largest sum
Tue 19 Jan	4. Binary search	Painter's partition
Wed 20 Jan	4. Binary search	Minimise maximum distance to a gas station
Thu 21 Jan	4. Binary search	Median of two sorted arrays
Fri 22 Jan	4. Binary search	Kth element of two sorted arrays
Sat 23 Jan	4. Binary search	Find a peak element II
Sun 24 Jan	4. Binary search	Median of a row-wise sorted matrix
Mon 25 Jan	6. Linked list	Reverse nodes in k-group
Tue 26 Jan	6. Linked list	Flatten a linked list
Wed 27 Jan	6. Linked list	Clone a linked list with random pointers

DUE	STEP	HARD PROBLEM
Thu 28 Jan	7. Recursion	Kth permutation sequence
Fri 29 Jan	7. Recursion	N-Queens
Sat 30 Jan	7. Recursion	Sudoku solver
Sun 31 Jan	7. Recursion	M-colouring problem
Mon 01 Feb	7. Recursion	Word break
Tue 02 Feb	7. Recursion	Expression add operators
Wed 03 Feb	9. Stacks/queues	Number of NGEs to the right
Thu 04 Feb	9. Stacks/queues	Trapping rain water
Fri 05 Feb	9. Stacks/queues	Largest rectangle in a histogram
Sat 06 Feb	9. Stacks/queues	Maximal rectangles
Sun 07 Feb	9. Stacks/queues	Sliding window maximum
Mon 08 Feb	9. Stacks/queues	LRU cache
Tue 09 Feb	9. Stacks/queues	LFU cache
Wed 10 Feb	10. Sliding window	Subarrays with K different integers
Thu 11 Feb	10. Sliding window	Minimum window substring
Fri 12 Feb	11. Heaps	Maximum sum combination
Sat 13 Feb	11. Heaps	Find median from a data stream
Sun 14 Feb	12. Greedy	Candy
Mon 15 Feb	13. Binary trees	Preorder, inorder and postorder in one traversal
Tue 16 Feb	13. Binary trees	Maximum path sum
Wed 17 Feb	13. Binary trees	Vertical order traversal
Thu 18 Feb	13. Binary trees	Print all nodes at distance K
Fri 19 Feb	13. Binary trees	Minimum time to burn a tree
Sat 20 Feb	13. Binary trees	Serialise and deserialise a binary tree
Sun 21 Feb	13. Binary trees	Morris inorder traversal

DUE	STEP	HARD PROBLEM
Mon 22 Feb	13. Binary trees	Morris preorder traversal
Tue 23 Feb	14. BST	Delete a node
Wed 24 Feb	14. BST	Recover a BST
Thu 25 Feb	14. BST	Largest BST in a binary tree
Fri 26 Feb	15. Graphs	Word ladder I
Sat 27 Feb	15. Graphs	Word ladder II
Sun 28 Feb	15. Graphs	Alien dictionary
Mon 01 Mar	15. Graphs	Print the shortest path
Tue 02 Mar	15. Graphs	Number of islands II
Wed 03 Mar	15. Graphs	Making a large island
Thu 04 Mar	15. Graphs	Swim in rising water
Fri 05 Mar	15. Graphs	Bridges in a graph
Sat 06 Mar	15. Graphs	Articulation points
Sun 07 Mar	15. Graphs	Kosaraju's algorithm (strongly connected components)
Mon 08 Mar	16. DP	Cherry pickup II
Tue 09 Mar	16. DP	Shortest common supersequence
Wed 10 Mar	16. DP	Distinct subsequences
Thu 11 Mar	16. DP	Edit distance
Fri 12 Mar	16. DP	Wildcard matching
Sat 13 Mar	16. DP	Best time to buy and sell stock III
Sun 14 Mar	16. DP	Best time to buy and sell stock IV
Mon 15 Mar	16. DP	LIS using binary search
Tue 16 Mar	16. DP	Matrix chain multiplication
Wed 17 Mar	16. DP	Minimum cost to cut a stick
Thu 18 Mar	16. DP	Burst balloons

DUE	STEP	HARD PROBLEM
Fri 19 Mar	16. DP	Evaluate a boolean expression to true
Sat 20 Mar	16. DP	Palindrome partitioning II
Sun 21 Mar	16. DP	Maximum rectangle area with all 1s
Mon 22 Mar	17. Tries	Maximum XOR of two numbers in an array
Tue 23 Mar	17. Tries	Maximum XOR with an element from an array
Wed 24 Mar	18. Adv. strings	Shortest palindrome
Thu 25 Mar	18. Adv. strings	Rabin-Karp algorithm
Fri 26 Mar	18. Adv. strings	Z-function
Sat 27 Mar	18. Adv. strings	KMP algorithm and the LPS array
Sun 28 Mar	18. Adv. strings	Longest happy prefix

PART 03

03 LeetCode month

Two topic problems and one recall problem a day, a contest every Sunday, spaced revision.

PART 03 | RULES

Rules and contests

RULE	DETAIL
Daily quota	Mon to Sat: 3 problems (2 topic + 1 recall), after that day's Striver work.
Time box	45 minutes of thinking. At 30 read only hints; at 60 read the editorial, close it, code it from memory.
Sundays	Weekly contest, then upsolve the next problem within 24 hours.

RULE	DETAIL
Revision	Before new problems, re-solve those listed under Revise (solved 1, 3, 7 days ago).
Never	Paste a solution, skip the log, or leave an F without a redo date.

> FALL-BEHIND RULE
>
> Behind a day: skip the recall problem. Behind three days: 2 a day for a week, keep revision and contests. Never a 10-problem catch-up day.

PART 03 | DAY BY DAY

The 30-day plan

T topic (today's Striver step) R recall (an older step)

DATE	STRIVER STEP	PROBLEMS	REVISE FIRST
Sat 07 Nov	1. Basics	T **#9** Palindrome Number (Easy) T **#1281** Subtract the Product and Sum of Digits of an Integer (Easy) T **#509** Fibonacci Number (Easy)	none
Sun 08 Nov	1. Basics	**Contest day.** Weekly Contest, then upsolve one problem. Write the weekly review.	#9, #1281, #509
Mon 09 Nov	2. Sorting	T **#88** Merge Sorted Array (Easy) T **#1051** Height Checker (Easy) R **#412** Fizz Buzz (Easy)	none
Tue 10 Nov	3. Arrays	T **#217** Contains Duplicate (Easy) T **#283** Move Zeroes (Easy) R **#912** Sort an Array (Medium)	#9, #1281, #509, #88, #1051, #412
Wed 11 Nov	3. Arrays	T **#189** Rotate Array (Medium) T **#238** Product of Array Except Self (Medium) R **#1365** How Many Numbers Are Smaller Than the Current Number (Easy)	#217, #283, #912
Thu 12 Nov	3. Arrays	T **#11** Container With Most Water (Medium)	#88, #1051, #412, #189, #238, #1365

DATE	STRIVER STEP	PROBLEMS	REVISE FIRST
		T **#560** Subarray Sum Equals K (Medium) R **#147** Insertion Sort List (Medium)	
Fri 13 Nov	4. Binary search	T **#704** Binary Search (Easy) T **#35** Search Insert Position (Easy) R **#128** Longest Consecutive Sequence (Medium)	#217, #283, #912, #11, #560, #147
Sat 14 Nov	4. Binary search	T **#34** Find First and Last Position of Element in Sorted Array (Medium) T **#33** Search in Rotated Sorted Array (Medium) R **#1748** Sum of Unique Elements (Easy)	#9, #1281, #509, #189, #238, #1365, #704, #35, #128
Sun 15 Nov	4. Binary search	**Contest day.** Weekly Contest, then upsolve one problem. Write the weekly review.	#11, #560, #147, #34, #33, #1748
Mon 16 Nov	5. Strings	T **#125** Valid Palindrome (Easy) T **#242** Valid Anagram (Easy) R **#153** Find Minimum in Rotated Sorted Array (Medium)	#88, #1051, #412, #704, #35, #128
Tue 17 Nov	6. Linked list	T **#206** Reverse Linked List (Easy) T **#21** Merge Two Sorted Lists (Easy) R **#162** Find Peak Element (Medium)	#217, #283, #912, #34, #33, #1748, #125, #242, #153
Wed 18 Nov	6. Linked list	T **#141** Linked List Cycle (Easy) T **#19** Remove Nth Node From End of List (Medium) R **#49** Group Anagrams (Medium)	#189, #238, #1365, #206, #21, #162
Thu 19 Nov	7. Recursion	T **#78** Subsets (Medium) T **#46** Permutations (Medium) R **#151** Reverse Words in a String (Medium)	#11, #560, #147, #125, #242, #153, #141, #19, #49
Fri 20 Nov	7. Recursion	T **#39** Combination Sum (Medium) T **#22** Generate Parentheses (Medium) R **#142** Linked List Cycle II (Medium)	#704, #35, #128, #206, #21, #162, #78, #46, #151
Sat 21 Nov	7. Recursion	T **#17** Letter Combinations of a Phone Number (Medium)	#34, #33, #1748, #141, #19, #49, #39, #22, #142

DATE	STRIVER STEP	PROBLEMS	REVISE FIRST
		T **#90** Subsets II (Medium) R **#7** Reverse Integer (Medium)	
Sun 22 Nov	8. Bits	**Contest day.** Weekly Contest, then upsolve one problem. Write the weekly review.	#78, #46, #151, #17, #90, #7
Mon 23 Nov	9. Stacks/queues	T **#20** Valid Parentheses (Easy) T **#155** Min Stack (Medium) R **#2** Add Two Numbers (Medium)	#125, #242, #153, #39, #22, #142
Tue 24 Nov	9. Stacks/queues	T **#150** Evaluate Reverse Polish Notation (Medium) T **#739** Daily Temperatures (Medium) R **#79** Word Search (Medium)	#206, #21, #162, #17, #90, #7, #20, #155, #2
Wed 25 Nov	10. Sliding window	T **#3** Longest Substring Without Repeating Characters (Medium) T **#424** Longest Repeating Character Replacement (Medium) R **#131** Palindrome Partitioning (Medium)	#141, #19, #49, #150, #739, #79
Thu 26 Nov	10. Sliding window	T **#1004** Max Consecutive Ones III (Medium) T **#567** Permutation in String (Medium) R **#136** Single Number (Easy)	#78, #46, #151, #20, #155, #2, #3, #424, #131
Fri 27 Nov	11. Heaps	T **#1046** Last Stone Weight (Easy) T **#703** Kth Largest Element in a Stream (Easy) R **#338** Counting Bits (Easy)	#39, #22, #142, #150, #739, #79, #1004, #567, #136
Sat 28 Nov	12. Greedy	T **#455** Assign Cookies (Easy) T **#55** Jump Game (Medium) R **#191** Number of 1 Bits (Easy)	#17, #90, #7, #3, #424, #131, #1046, #703, #338
Sun 29 Nov	12. Greedy	**Contest day.** Weekly Contest, then upsolve one problem. Write the weekly review.	#1004, #567, #136, #455, #55, #191
Mon 30 Nov	13. Binary trees	T **#104** Maximum Depth of Binary Tree (Easy) T **#226** Invert Binary Tree (Easy) R **#268** Missing Number (Easy)	#20, #155, #2, #1046, #703, #338

DATE	STRIVER STEP	PROBLEMS	REVISE FIRST
Tue 01 Dec	13. Binary trees	T **#543** Diameter of Binary Tree (Easy) T **#102** Binary Tree Level Order Traversal (Medium) R **#503** Next Greater Element II (Medium)	#150, #739, #79, #455, #55, #191, #104, #226, #268
Wed 02 Dec	13. Binary trees	T **#199** Binary Tree Right Side View (Medium) T **#236** Lowest Common Ancestor of a Binary Tree (Medium) R **#209** Minimum Size Subarray Sum (Medium)	#3, #424, #131, #543, #102, #503
Thu 03 Dec	14. BST	T **#700** Search in a Binary Search Tree (Easy) T **#98** Validate Binary Search Tree (Medium) R **#438** Find All Anagrams in a String (Medium)	#1004, #567, #136, #104, #226, #268, #199, #236, #209
Fri 04 Dec	15. Graphs	T **#200** Number of Islands (Medium) T **#994** Rotting Oranges (Medium) R **#76** Minimum Window Substring (Hard)	#1046, #703, #338, #543, #102, #503, #700, #98, #438
Sat 05 Dec	15. Graphs	T **#207** Course Schedule (Medium) T **#133** Clone Graph (Medium) R **#215** Kth Largest Element in an Array (Medium)	#455, #55, #191, #199, #236, #209, #200, #994, #76
Sun 06 Dec	15. Graphs	**Contest day.** Weekly Contest, then upsolve one problem. Write the weekly review.	#700, #98, #438, #207, #133, #215

PART 03 | CP31, 1 HOUR A DAY FROM 14 DEC

The CP31 sheet

CP31 is a free sheet by TLE Eliminators (search "TLE Eliminators CP-31 sheet") that groups Codeforces problems by rating, about 31 per rating. It builds the speed that online assessments

reward, and keeps the HFT door open. One hour a day from 14 Dec, once the LeetCode month is over.

RATING	DATES
800	14 Dec to 28 Dec
900	29 Dec to 12 Jan
1000	13 Jan to 27 Jan
1100	28 Jan to 11 Feb
1200	12 Feb to 26 Feb
1300	27 Feb to 13 Mar
1400	14 Mar to 28 Mar

- [ ] 2 problems a day, oldest unsolved first
- [ ] 30 minutes each at most, then read the editorial and code it yourself
- [ ] Weekends: a Codeforces Div 3 or Div 4 round when one is on, then upsolve one problem
- [ ] Log each problem in the problem log, the same as Striver

> **TARGET**
>
> 800 to 1400 done by 28 Mar. Keep going toward 1600 in the summer (see Part 11).

PART 04

04 Core CS

OOP, OS, DBMS and SQL, computer networks and system design, one at a time.

PART 04 | 45 MIN A DAY

Five core CS subjects

SUBJECT	DATES
Object-oriented programming (C++)	07 Nov to 20 Nov

SUBJECT	DATES
Operating systems	21 Nov to 18 Dec
DBMS and SQL	19 Dec to 15 Jan
Computer networks	16 Jan to 12 Feb
System design basics	13 Feb to 07 Mar
Revision and mock vivas	08 Mar to 28 Mar

MINUTES	WHAT
0 to 5	Answer yesterday's 3 flashcards from memory.
5 to 30	Learn today's topic from one source.
30 to 40	Make 3 flashcards.
40 to 45	Say one interview answer out loud.

> **HOW TO ANSWER ANY THEORY QUESTION**
>
> Define it in one sentence, give an example, name a trade-off, say where you used it.

CORE CS | 07 NOV TO 20 NOV

Object-oriented programming (C++)

Why it matters. Asked in nearly every fresher interview, often as the first theory question. You just learned C++, so this is the cheapest subject to master first, and it makes your builds cleaner.

WEEK	FOCUS	TOPICS
07 Nov to 13 Nov	**Classes and encapsulation**	• Classes and objects, members, `this` • Access specifiers: public, private, protected • Constructors: default, parameterised, copy; initialiser lists • Destructors and object lifetime; shallow vs deep copy • Static members and functions; `const` member functions

WEEK	FOCUS	TOPICS
14 Nov to 20 Nov	**Inheritance and polymorphism**	• Inheritance types and access in inheritance • Function overloading and operator overloading (compile-time) • Virtual functions, overriding, vtable and vptr (run-time) • Abstract classes, pure virtual functions, interfaces • Virtual destructors, diamond problem, virtual inheritance • Composition vs inheritance; the SOLID principles

Do it practically

☐ Refactor your Step 6 playlist into a `Playlist` class with a private `Node` struct and public methods.

☐ Write the `Shape` hierarchy below, then add `Triangle` without touching existing classes (open/closed principle).

☐ Explain out loud why removing `virtual` from the destructor is dangerous.

SHAPES.CPP Copy

```cpp
class Shape {
public:
    virtual double area() const = 0;        // pure virtual: Shape is abstract
    virtual ~Shape() = default;             // safe delete through a base pointer
};
class Circle : public Shape {
    double r;
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};
class Rect : public Shape {
    double w, h;
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
};
int main() {
    vector<unique_ptr<Shape>> shapes;
    shapes.push_back(make_unique<Circle>(1));
    shapes.push_back(make_unique<Rect>(2, 3));
    for (auto &s : shapes) cout << s->area() << '\n';   // run-time polymorphism
}
```

```
class Shape {
public:
```

▶ OUTPUT

```
3.14159
6
```

Interview questions

Answer out loud first, then open the model answer.

▼ Q01 What are the four pillars of OOP?

Encapsulation: data and the methods on it live together and the data is hidden behind an interface. **Abstraction**: expose what an object does, not how. **Inheritance**: a class reuses and extends another. **Polymorphism**: one interface, many behaviours (overloading at compile time, virtual functions at run time).

▼ Q02 Class vs struct in C++?

Identical except for defaults: struct members and inheritance are public by default, class members and inheritance are private by default. Convention: struct for plain data, class for objects with invariants.

▼ Q03 When is a copy constructor called?

When an object is initialised from another object of the same type: `T b = a;`, passing by value, and returning by value (often optimised away). The default copy is shallow; if the class owns heap memory, write a deep copy (or follow the rule of three/five).

▼ Q04 Compile-time vs run-time polymorphism?

Compile-time: function and operator overloading and templates, resolved by the compiler. Run-time: virtual functions called through a base pointer or reference, resolved through the vtable when the program runs.

▼ Q05 How does a virtual function call work?

Each class with virtual functions has a vtable of function pointers. Each object stores a hidden vptr to its class's vtable. A virtual call follows the vptr and jumps to the right function: one extra indirection.

▼ Q06 Why make a base class destructor virtual?

If you `delete` a derived object through a base pointer and the destructor is not virtual, only the base destructor runs (undefined behaviour, usually a leak). A virtual destructor makes the derived destructor run first.

▼ Q07 Abstract class vs interface?

An abstract class has at least one pure virtual function (`= 0`) and cannot be instantiated; it can still hold data and implemented methods. An interface is the special case where every method is pure virtual and there is no data.

▼ Q08 Overloading vs overriding?

Overloading: same name, different parameters, in the same scope, chosen at compile time. Overriding: a derived class redefines a virtual function with the same signature, chosen at run time.

▼ Q09 What is the diamond problem?

B and C inherit from A, and D inherits from both, so D holds two copies of A and calls to A's members are ambiguous. Virtual inheritance (`class B : virtual public A`) keeps one shared A.

▼ Q10 Composition or inheritance?

Inheritance models is-a; composition models has-a. Prefer composition: it couples classes less and lets you swap parts. Use inheritance when you need polymorphism through a common interface.

▼ Q11 What are the SOLID principles?

Single responsibility (one reason to change), Open/closed (extend without editing), Liskov substitution (a subclass works wherever its base does), Interface segregation (small focused interfaces), Dependency inversion (depend on abstractions).

▼ Q12 What does `static` mean on a member?

A static data member is shared by all objects of the class; a static member function has no `this` and can only touch static members. Call it as `ClassName::f()`.

Sources

- learncpp.com chapters on classes, inheritance and virtual functions
- Your own builds: turn two of them into classes

CORE CS | 21 NOV TO 18 DEC

Operating systems

Why it matters. Product companies and service companies both ask OS: processes, threads, scheduling, deadlocks and memory. It also underpins Linux skills for your cyber track.

WEEK	FOCUS	TOPICS
21 Nov to 27 Nov	**Processes and threads**	• What an OS does; kernel mode vs user mode; system calls • Process vs thread; the PCB; process states • Context switching and its cost • `fork`, `exec`, `wait`; zombie and orphan processes • Inter-process communication: pipes, shared memory, messages
28 Nov to 04 Dec	**CPU scheduling**	• FCFS, SJF, SRTF, priority, round robin • Waiting, turnaround and response time; Gantt charts • Starvation and aging • Choosing a time quantum for round robin
05 Dec to 11 Dec	**Synchronisation and deadlock**	• Race conditions, critical sections • Mutex vs semaphore (binary and counting) • Producer-consumer, readers-writers, dining philosophers • Deadlock: the four Coffman conditions • Prevention, avoidance (Banker's algorithm), detection and recovery
12 Dec to 18 Dec	**Memory and files**	• Contiguous allocation, fragmentation • Paging, page tables, the TLB; segmentation • Virtual memory, demand paging, page faults • Page replacement: FIFO, LRU, Optimal; Belady's anomaly; thrashing • File systems basics: inodes, directories

Do it practically

☐ Build a CPU scheduling simulator: read processes (arrival, burst) and print the Gantt chart and average waiting time for FCFS, SJF and round robin.

☐ Run the page replacement code below and reproduce Belady's anomaly, then add LRU (reuse your Step 9 LRU cache).

☐ On your Mac: `ps aux | head`, `top -l 1 | head -20`, and `vm_stat`. Name what each column means.

BELADY.CPP Copy

```cpp
int fifoFaults(const vector<int> &refs, int frames) {
    deque<int> q; unordered_set<int> in; int faults = 0;
    for (int p : refs) {
        if (in.count(p)) continue;                  // hit
        faults++;
        if ((int)q.size() == frames) { in.erase(q.front()); q.pop_front(); }
        q.push_back(p); in.insert(p);
    }
    return faults;
}
int main() {
    vector<int> refs = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    cout << fifoFaults(refs, 3) << ' ' << fifoFaults(refs, 4) << '\n';   // more frames,
more faults
}
```

```
int fifoFaults(const
vector<int> &refs,
```

▶ OUTPUT

```
9 10
```

Interview questions

Answer out loud first, then open the model answer.

▼ Q01 Process vs thread?

A process is a running program with its own address space and resources. Threads live inside a process and share its memory and files but each has its own stack and registers. Threads are cheaper to create and switch, but a bug in one can corrupt the others.

▼ Q02 What happens during a context switch?

The CPU saves the running process's registers, program counter and state into its PCB, the scheduler picks the next process, and its saved state is loaded. It is pure overhead, and it also cools the caches and TLB.

▼ Q03 Name the process states.

New, ready, running, waiting (blocked), terminated. Ready means it could run but has no CPU; waiting means it needs an event such as I/O.

▼ Q04 Which scheduling algorithm minimises average waiting time?

SJF when all jobs are known (non-preemptive), SRTF in the preemptive case. Both need burst estimates and can starve long jobs; aging fixes starvation. Round robin gives better response time for interactive systems.

▼ Q05 Mutex vs semaphore?

A mutex is a lock with an owner: only the thread that locked it unlocks it, and it guards one critical section. A semaphore is a counter with wait and signal; a counting semaphore allows up to N users of a resource and can be signalled by another thread.

▼ Q06 What are the conditions for deadlock?

All four must hold: mutual exclusion, hold and wait, no preemption, circular wait. Break any one to prevent deadlock, for example by always acquiring locks in a fixed global order (breaks circular wait).

▼ Q07 What does the Banker's algorithm do?

Before granting a request it simulates the allocation and checks whether some order still lets every process finish (a safe state). If not, the request waits. It avoids deadlock but needs each process's maximum demand in advance.

▼ Q08 Paging vs segmentation?

Paging splits memory into fixed-size pages and frames, so no external fragmentation, but some internal fragmentation. Segmentation splits by logical units (code, stack, heap) of variable size, which matches the program's view but fragments externally.

▼ Q09 What is virtual memory and a page fault?

Virtual memory gives each process a large private address space backed partly by disk. When a process touches a page not in RAM, the hardware raises a page fault, the OS loads the page (evicting another if needed), and the instruction retries.

▼ Q10 What is Belady's anomaly?

With FIFO page replacement, adding frames can increase page faults. For the reference string 1 2 3 4 1 2 5 1 2 3 4 5, FIFO gives 9 faults with 3 frames and 10 with 4. LRU and Optimal never show it.

▼ Q11 What is thrashing?

The system spends more time swapping pages than running, because the active processes' working sets do not fit in RAM. Fixes: reduce the number of running processes, or give them more memory.

▼ Q12 **Kernel mode vs user mode, and a system call?**

User programs run in user mode with restricted instructions. A system call (read, write, fork) is a controlled trap into kernel mode, where the OS does the privileged work and returns.

Sources

- Operating Systems: Three Easy Pieces (free at ostep.org)
- Operating System Concepts by Silberschatz, Galvin and Gagne (your college library)

CORE CS | 19 DEC TO 15 JAN

DBMS and SQL

Why it matters. Every backend role and most online tests include SQL queries, and interviews ask about keys, normal forms, transactions and indexes.

WEEK	FOCUS	TOPICS
19 Dec to 25 Dec	**Modelling**	• DBMS vs file system • ER model: entities, attributes, relationships, cardinality • Keys: super, candidate, primary, alternate, foreign, composite • Relational model and relational algebra basics
26 Dec to 01 Jan	**SQL**	• SELECT, WHERE, ORDER BY, GROUP BY, HAVING • Joins: inner, left, right, full, self, cross • Subqueries and common table expressions • Window functions: ROW_NUMBER, RANK, DENSE_RANK • Start LeetCode's SQL 50 study plan: about 7 a week
02 Jan to 08 Jan	**Normalisation**	• Functional dependencies • Insert, update and delete anomalies • 1NF, 2NF, 3NF, BCNF with examples • When to denormalise

WEEK	FOCUS	TOPICS
09 Jan to 15 Jan	**Transactions and indexes**	• ACID properties • Isolation levels and the anomalies they prevent • Locks and two-phase locking • Indexes: B+ trees, clustered vs non-clustered • SQL vs NoSQL; CAP theorem in one paragraph

Do it practically

☐ Your Mac already has SQLite. Run `sqlite3 tracker.db`, create the schema below, insert 20 rows by hand, and run the three queries.

☐ Finish at least 25 problems from LeetCode's SQL 50 by 15 Jan.

☐ Normalise a messy table: one row per student with their courses in a comma list. Take it to 3NF and explain each step.

TRACKER.SQL Copy

```sql
CREATE TABLE steps (id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE problems (
  id INTEGER PRIMARY KEY,
  step_id INTEGER REFERENCES steps(id),
  title TEXT NOT NULL,
  level TEXT CHECK (level IN ('E', 'M', 'H'))
);
CREATE TABLE attempts (
  id INTEGER PRIMARY KEY,
  problem_id INTEGER REFERENCES problems(id),
  day TEXT,                                  -- '2026-11-12'
  minutes INTEGER,
  result TEXT CHECK (result IN ('S', 'H', 'F'))
);

-- problems solved alone, per step
SELECT s.name, COUNT(DISTINCT a.problem_id) AS solved
FROM steps s
JOIN problems p ON p.step_id = s.id
JOIN attempts a ON a.problem_id = p.id AND a.result = 'S'
GROUP BY s.id ORDER BY s.id;

-- steps where you average more than 40 minutes
SELECT p.step_id, AVG(a.minutes) AS avg_min
FROM attempts a JOIN problems p ON p.id = a.problem_id
GROUP BY p.step_id HAVING AVG(a.minutes) > 40;

-- your slowest problem in each step
SELECT step_id, title, minutes FROM (
  SELECT p.step_id, p.title, a.minutes,
         DENSE_RANK() OVER (PARTITION BY p.step_id ORDER BY a.minutes DESC) AS rk
```

```
  FROM attempts a JOIN problems p ON p.id = a.problem_id
) WHERE rk = 1;
```

```
CREATE TABLE steps
(id INTEGER PRIMARY
```

Interview questions

Answer out loud first, then open the model answer.

▼ Q01 What does ACID mean?

Atomicity: a transaction happens fully or not at all. **Consistency**: it moves the database from one valid state to another. **Isolation**: concurrent transactions do not see each other's partial work. **Durability**: once committed, it survives crashes.

▼ Q02 Primary key vs unique key vs foreign key?

A primary key uniquely identifies a row and cannot be NULL; one per table. A unique key also forbids duplicates but (in most databases) allows NULL, and a table can have several. A foreign key references another table's key to keep relationships valid.

▼ Q03 Explain 1NF, 2NF, 3NF and BCNF.

1NF: atomic values, no repeating groups. 2NF: 1NF and no non-key attribute depends on part of a composite key. 3NF: 2NF and no non-key attribute depends on another non-key attribute. BCNF: for every dependency X to Y, X is a superkey.

▼ Q04 WHERE vs HAVING?

WHERE filters rows before grouping and cannot use aggregates. HAVING filters groups after GROUP BY and can use aggregates like `COUNT(*) > 5`.

▼ Q05 DELETE vs TRUNCATE vs DROP?

DELETE removes chosen rows, is logged per row and can be rolled back. TRUNCATE removes all rows quickly and resets storage (rollback support depends on the database). DROP removes the table itself, structure included.

▼ Q06 What are the types of join?

Inner: matching rows only. Left: all left rows plus matches (NULL where none). Right: the mirror. Full outer: all rows from both. Self join: a table joined to itself. Cross join: every combination.

▼ Q07 What is an index, and what does it cost?

A data structure (usually a B+ tree) that finds rows by key in O(log n) instead of scanning. It speeds up reads but slows inserts and updates and uses space. A clustered index decides the physical row order (one per table); non-clustered indexes point to rows.

▼ Q08 **Name the isolation levels and what each prevents.**

Read uncommitted prevents nothing. Read committed prevents dirty reads. Repeatable read also prevents non-repeatable reads. Serializable also prevents phantom reads. Stronger isolation means more locking and less concurrency.

▼ Q09 **Query the Nth highest salary.**

`SELECT DISTINCT salary FROM Employee ORDER BY salary DESC LIMIT 1 OFFSET N-1;` or rank with `DENSE_RANK() OVER (ORDER BY salary DESC)` and pick rank N. Handle the case where fewer than N distinct salaries exist.

▼ Q10 **ROW_NUMBER vs RANK vs DENSE_RANK?**

For values 100, 90, 90, 80: ROW_NUMBER gives 1 2 3 4, RANK gives 1 2 2 4 (gaps after ties), DENSE_RANK gives 1 2 2 3 (no gaps).

▼ Q11 **SQL vs NoSQL?**

SQL databases have fixed schemas, joins and strong transactions; they suit structured, related data. NoSQL (document, key-value, column, graph) trades some of that for flexible schemas and easier horizontal scaling.

▼ Q12 **What is a view?**

A saved query that behaves like a virtual table. It simplifies complex joins and can restrict which columns users see, which is also a security tool.

Sources

- SQLBolt (sqlbolt.com) for the first SQL week
- LeetCode SQL 50 study plan
- Database System Concepts by Silberschatz, Korth and Sudarshan

CORE CS | 16 JAN TO 12 FEB

Computer networks

Why it matters. The most important subject for your cyber goal and a favourite in interviews ("what happens when you type a URL?"). It also overlaps your semester CN course.

WEEK	FOCUS	TOPICS
16 Jan to 22 Jan	**Models and devices**	• OSI 7 layers vs TCP/IP 4 layers; encapsulation • Hub, switch, router, gateway; collision and broadcast domains • MAC vs IP addresses; ARP • Ethernet frames basics
23 Jan to 29 Jan	**IP and routing**	• IPv4 addressing, subnetting and CIDR • Private ranges, NAT, DHCP • ICMP, ping and traceroute • Routing basics; IPv6 at a glance
30 Jan to 05 Feb	**Transport**	• TCP vs UDP; ports • Three-way handshake and four-way close • Flow control (sliding window) vs congestion control (slow start, AIMD) • Reliability: sequence numbers, ACKs, retransmission
06 Feb to 12 Feb	**Application**	• DNS resolution and record types • HTTP methods, status codes, headers, cookies • HTTPS and the TLS handshake • What happens when you type a URL, end to end

Do it practically

☐ Capture your own traffic in Wireshark while opening a website. Find the DNS query, the TCP handshake (SYN, SYN-ACK, ACK) and the TLS Client Hello.

☐ Write a subnet calculator in C++: input `192.168.1.0/26`, output network, broadcast, host range and host count.

☐ Run the commands below and explain every line of output.

TERMINAL Copy

```
dig google.com +short                      # DNS A records
dig google.com MX +short                   # mail servers for the domain
traceroute -m 15 google.com                # every router hop on the way
curl -sv https://example.com -o /dev/null  # TLS handshake and HTTP headers
nc -vz example.com 443                     # is TCP port 443 open?
ifconfig en0 | grep "inet "                # your IP on Wi-Fi
```

```
dig google.com +short
# DNS A records
```

Interview questions

Answer out loud first, then open the model answer.

▼ Q01 **Name the OSI layers with one protocol each.**

Physical (Ethernet cabling), Data link (Ethernet, ARP), Network (IP, ICMP), Transport (TCP, UDP), Session (RPC), Presentation (TLS encryption, encoding), Application (HTTP, DNS, SMTP). TCP/IP folds the top three into Application and the bottom two into Link.

▼ Q02 **TCP vs UDP?**

TCP is connection-oriented, reliable and ordered, with flow and congestion control: web, email, SSH. UDP is connectionless, with no guarantees and lower overhead: DNS queries, video calls, games.

▼ Q03 **Explain the three-way handshake.**

Client sends SYN with its initial sequence number. Server replies SYN-ACK with its own sequence number and acknowledges the client's. Client sends ACK. Both sides now agree on sequence numbers and the connection is open.

▼ Q04 **What happens when you type a URL and press Enter?**

The browser checks its cache, then resolves the name through DNS (cache, resolver, root, TLD, authoritative). It opens a TCP connection to the IP (port 443), runs the TLS handshake, sends an HTTP request, receives the response, then parses HTML and fetches CSS, JS and images, often over the same connection.

▼ Q05 **How does DNS resolution work, and name some record types?**

A recursive resolver asks a root server, which points to the TLD server (.com), which points to the domain's authoritative server, which returns the record; answers are cached by TTL. Records: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), NS (name servers), TXT (text, used for SPF and verification).

▼ Q06 **How many hosts in a /26, and what are the subnets of 192.168.1.0/24 split into /26?**

A /26 has 2^6 = 64 addresses, 62 usable (minus network and broadcast). The /24 splits into 192.168.1.0, .64, .128 and .192, each /26.

▼ Q07 **Hub vs switch vs router?**

A hub repeats every frame to every port (one collision domain). A switch learns MAC addresses and forwards frames only to the right port (layer 2). A router forwards packets between networks using IP addresses (layer 3).

▼ Q08 **What does ARP do?**

It maps an IP address to a MAC address on the local network: "who has 192.168.1.1? tell me". Replies are cached. ARP has no authentication, which is why ARP spoofing works.

▼ Q09 **What are NAT and the private IP ranges?**

NAT lets many devices with private addresses share one public IP by rewriting addresses and ports at the router. Private ranges: 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16.

▼ Q10 **HTTP vs HTTPS?**

HTTPS is HTTP inside TLS. TLS authenticates the server with a certificate, agrees on session keys with asymmetric cryptography, then encrypts and integrity-protects traffic with fast symmetric ciphers.

▼ Q11 **Name common HTTP status codes.**

200 OK, 201 Created, 301 Moved Permanently, 302 Found, 304 Not Modified, 400 Bad Request, 401 Unauthorized (not logged in), 403 Forbidden (logged in, not allowed), 404 Not Found, 429 Too Many Requests, 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable.

▼ Q12 **Flow control vs congestion control?**

Flow control protects the receiver: the receiver advertises a window of how much it can accept. Congestion control protects the network: the sender's congestion window grows (slow start, then additive increase) and halves on loss.

Sources

- Computer Networking: A Top-Down Approach by Kurose and Ross
- Wireshark's own sample captures and display-filter reference

CORE CS | 13 FEB TO 07 MAR

System design basics

Why it matters. Product companies ask freshers for basic design: how to scale a service, what to cache, how to model classes. You already built TenantWall on AWS, which gives you a real system to talk about.

WEEK	FOCUS	TOPICS
13 Feb to 19 Feb	**Scaling a web service**	• Client-server, latency vs throughput • Vertical vs horizontal scaling; stateless services • Load balancers and their algorithms • Caching: where, what, eviction (LRU), TTL; CDNs
20 Feb to 26 Feb	**Data at scale**	• Replication (leader-follower) and read replicas • Sharding and consistent hashing • CAP theorem; consistency choices • Message queues and async work; rate limiting
27 Feb to 07 Mar	**Design practice**	• Low-level design: parking lot, LRU cache, library system as classes • High-level design: URL shortener, then a chat service • Write TenantWall's design document with a diagram • Practise talking through a design for 20 minutes

Do it practically

☐ Write a one-page design document for TenantWall: requirements, diagram, data model, how it scales, and its security choices (IAM, ABAC).

☐ Low-level design of a parking lot in C++ classes: spots, levels, tickets, payment.

☐ Run the token bucket below; then plug it into your Step 10 logwatch so it decides which requests to reject.

TOKEN_BUCKET.CPP

Copy

```cpp
class TokenBucket {
    double tokens, cap, rate, last;          // rate = tokens added per second
public:
    TokenBucket(double cap, double rate) : tokens(cap), cap(cap), rate(rate), last(0) {}
    bool allow(double now) {
        tokens = min(cap, tokens + (now - last) * rate);
        last = now;
        if (tokens < 1) return false;
        tokens -= 1;
        return true;
    }
};
int main() {
    TokenBucket b(3, 1);                     // burst of 3, then 1 request per second
    for (double t : {0.0, 0.1, 0.2, 0.3, 1.5, 1.6}) cout << (b.allow(t) ? "ok " : "429 ");
    cout << '\n';
}
```

```
class TokenBucket {
    double tokens,
```

▶ OUTPUT

```
ok ok ok 429 ok 429
```

Interview questions

Answer out loud first, then open the model answer.

Q01 Vertical vs horizontal scaling?

Vertical: a bigger machine; simple, but it has a ceiling and is a single point of failure. Horizontal: more machines behind a load balancer; needs stateless services and shared storage, but scales further and survives failures.

Q02 What does a load balancer do, and how does it choose?

It spreads requests across servers and removes unhealthy ones using health checks. Common algorithms: round robin, least connections, and IP hash (keeps a client on the same server).

Q03 Name caching strategies.

Cache-aside: the app reads the cache, falls back to the database, then fills the cache. Write-through: write cache and database together. Write-back: write the cache now and the database later (fast, risk of loss). Evict with LRU or a TTL.

Q04 What is the CAP theorem?

When a network partition happens, a distributed store must choose between consistency (every read sees the latest write) and availability (every request gets an answer). Without a partition you can have both.

Q05 Replication vs sharding?

Replication copies the same data to several machines for availability and read throughput. Sharding splits the data across machines by a key so each holds a part, which scales writes and storage.

Q06 What is consistent hashing?

Servers and keys are placed on a hash ring; each key goes to the next server clockwise. Adding or removing a server moves only the keys next to it instead of reshuffling everything.

Q07 Why use a message queue?

It decouples producers from consumers, absorbs traffic spikes, and lets slow work (emails, thumbnails) happen asynchronously with retries.

▼ Q08 **Name rate-limiting algorithms.**

Token bucket (allows bursts up to a cap), leaky bucket (smooth constant outflow), fixed window counters (simple, bursty at the edges), and sliding window logs or counters (accurate).

▼ Q09 **Design a URL shortener in a few sentences.**

An API takes a long URL, generates a short ID (base62 of a counter or a hash), stores id to URL in a key-value store, and redirects with 301 or 302. Cache popular IDs, use read replicas because reads dominate, and record click analytics asynchronously through a queue.

▼ Q10 **Monolith vs microservices?**

A monolith is one deployable unit: simple to build, test and run while small. Microservices split by business area and deploy independently, at the cost of network calls, distributed failures and harder debugging. Start with a monolith.

▼ Q11 **Stateless vs stateful services?**

A stateless service keeps no session data between requests, so any instance can serve any request and scaling is easy. Keep state in a database, a cache, or a signed token.

▼ Q12 **What is a CDN?**

A network of edge servers near users that caches static content (images, scripts, video) so requests do not travel to your origin server. Lower latency, less origin load.

Sources

- The System Design Primer (github.com/donnemartin/system-design-primer)
- Designing Data-Intensive Applications by Martin Kleppmann, as a stretch read

PART 05

05 Cyber track

One focused hour a day: Linux, networks, web attacks, machines, blue team, a CTF by March.

PART 05 | 1 HOUR A DAY, 2 ON SUNDAYS

Cyber security track

1 HOUR A DAY, 2 HOURS ON SUNDAYS

Start a timer. When it rings, write notes and stop, even mid-lab. Missed time is never made up. No cyber on the two C++ days.

ONLY HACK WHAT YOU ARE ALLOWED TO

Practise only on platforms built for it (TryHackMe, Hack The Box, PortSwigger Academy, OverTheWire, picoCTF, CTFs) and machines you own. Anything else, including your college network, is illegal under India's IT Act.

MINUTES	WHAT
0 to 5	Read yesterday's notes.
5 to 45	One lab task, one goal.
45 to 55	Notes: goal, steps, commands, what failed, lesson.
55 to 60	Log the hour.

CYBER | WEEK BY WEEK

The 20-week lab plan

WEEK	FOCUS	LABS INSIDE THE HOUR	PROOF OF WORK
W01 07 Nov to 13 Nov	**Linux command line**	TryHackMe Linux Fundamentals parts 1 to 3; OverTheWire Bandit levels 0 to 5	A notes file of 30 commands, each with one example
W02 14 Nov	**Permissions and the shell**	Bandit 6 to 15; file permissions, SUID, pipes and redirection	Bandit notes 0 to 15 (what each level taught)

WEEK	FOCUS	LABS INSIDE THE HOUR	PROOF OF WORK
to 20 Nov			
W03 21 Nov to 27 Nov	**Networking for hackers**	The networking module of TryHackMe's Pre Security path; Wireshark: capture your own DNS lookup and TCP handshake	A screenshot of SYN, SYN-ACK, ACK with your explanation
W04 28 Nov to 04 Dec	**Scanning and bash**	Bandit 16 to 20; TryHackMe's Nmap rooms; nmap your own home router only	Your nmap cheat sheet (template on the next pages)
W05 05 Dec to 11 Dec	**HTTP and Burp Suite**	Install Burp Suite Community; HTTP methods, headers, cookies, sessions; first PortSwigger Academy SQL injection labs	Burp history of one full login request, annotated
W06 12 Dec to 18 Dec	**SQL injection**	Remaining apprentice SQLi labs including UNION attacks; picoCTF web exploitation challenges	Writeup: one SQLi lab, payload by payload
W07 19 Dec to 25 Dec	**Cross-site scripting**	PortSwigger reflected, stored and DOM XSS apprentice labs	A table: XSS type, where it lands, how to fix it
W08 26 Dec to 01 Jan	**Access control and auth**	PortSwigger access control (IDOR) and authentication apprentice labs	Writeup: one IDOR, with the fix in code
W09 02 Jan to 08 Jan	**OWASP Top 10 review**	Read the OWASP Top 10; PortSwigger CSRF and SSRF apprentice labs	Your OWASP map: each item linked to a lab you did
W10 09 Jan to 15 Jan	**Cryptography basics**	Hashing vs encryption, salts, symmetric vs asymmetric, certificates, JWTs; picoCTF cryptography challenges (5)	One-page crypto cheat sheet in your own words
W11 16 Jan to 22 Jan	**Packet analysis**	Wireshark display filters (`dns` , `http` , `tcp.flags.syn == 1`), Follow TCP Stream; picoCTF forensics pcap challenges. Pairs with your CN month.	Writeup: one pcap challenge

WEEK	FOCUS	LABS INSIDE THE HOUR	PROOF OF WORK
W12 23 Jan to 29 Jan	**First machines**	Hack The Box Starting Point, Tier 0 machines	Personal notes for each machine (keep them private)
W13 30 Jan to 05 Feb	**Privilege escalation**	Starting Point Tier 1; TryHackMe Linux privilege escalation room: `sudo -l`, SUID, cron, PATH	PrivEsc checklist you can reuse
W14 06 Feb to 12 Feb	**Logs**	Linux auth logs, web server logs, Windows event IDs 4624, 4625, 4688, 4720; run your Step 10 logwatch on a sample log	Five detection rules written in plain English
W15 13 Feb to 19 Feb	**SIEM**	TryHackMe SOC Level 1 introductory rooms, or Splunk Free / Wazuh in a VM	Three saved searches that would catch an attack
W16 20 Feb to 26 Feb	**ATT&CK and incident response**	MITRE ATT&CK navigator; the incident response lifecycle; map one machine's attack chain to ATT&CK techniques	An ATT&CK mapping table for that machine
W17 27 Feb to 05 Mar	**Email and cloud**	Phishing analysis: headers, SPF, DKIM, DMARC; AWS IAM least privilege, CloudTrail, GuardDuty (tie to TenantWall)	TenantWall security notes: what is logged, who can do what
W18 06 Mar to 12 Mar	**Play a CTF**	Pick a beginner-friendly event on CTFtime; play solo or with classmates	Writeups for 2 challenges after the event ends
W19 13 Mar to 19 Mar	**Portfolio**	Clean up a public `security-notes` repo: only writeups that platforms allow (your labs, retired machines, ended CTFs)	Repo with 8 or more writeups and a README
W20 20 Mar to 28 Mar	**Review and next target**	Revisit weak areas; pick a certification target (for example CompTIA Security+ or eJPT) and draft a 3-month plan	A one-page plan for April to June

CYBER | SETUP AND REFERENCE

Lab setup, commands, ports, OWASP

1. **Weeks 1 to 4** need only a browser and Terminal: `ssh bandit0@bandit.labs.overthewire.org -p 2220`.
2. **Tools:** `brew install nmap`; Wireshark and Burp Suite Community have Apple Silicon builds.
3. **Kali VM:** UTM plus Kali's ARM64 image, 4 GB RAM, 2 cores. For x86-only targets use HTB Pwnbox or THM AttackBox.
4. **Close the VM** when the hour ends.

TERMINAL Copy

```
nmap -sn 192.168.1.0/24                # which hosts are up (your own network only)
nmap -sV -sC -T4 10.10.10.10           # versions and default scripts
nmap -p- --min-rate 1000 10.10.10.10   # every TCP port
sudo -l                                # what may I run as root?
find / -perm -4000 -type f 2>/dev/null # SUID binaries
ss -tulpn                              # listening ports
```

```
nmap -sn
192.168.1.0/24
```

PORT	SERVICE
20, 21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
67, 68	DHCP
80	HTTP
110	POP3
123	NTP
135	MS RPC
139, 445	SMB
143	IMAP

PORT	SERVICE
161	SNMP
389	LDAP
443	HTTPS
587	SMTP submission
636	LDAPS
993	IMAPS
1433	MS SQL
3306	MySQL
3389	RDP
5432	PostgreSQL
5900	VNC
6379	Redis
8080	HTTP alternate
27017	MongoDB

OWASP Top 10, 2021 edition; check owasp.org for newer editions.

ID	RISK	IN ONE LINE	PRACTISE
A01	Broken Access Control	Users act outside their permissions (IDOR, missing checks)	PortSwigger access control labs
A02	Cryptographic Failures	Sensitive data exposed by weak or missing crypto	picoCTF crypto, TLS settings
A03	Injection	SQL, OS command and other injection; XSS is grouped here	PortSwigger SQLi and XSS labs
A04	Insecure Design	Flaws in the design itself, not the code	Threat-model TenantWall
A05	Security Misconfiguration	Defaults, open buckets, verbose errors	AWS S3 bucket policies
A06	Vulnerable and Outdated Components	Libraries with known CVEs	Check one project with `npm audit` or similar

ID	RISK	IN ONE LINE	PRACTISE
A07	Identification and Authentication Failures	Weak login, session and credential handling	PortSwigger authentication labs
A08	Software and Data Integrity Failures	Untrusted updates, insecure deserialisation, CI/CD trust	Read one supply-chain incident report
A09	Security Logging and Monitoring Failures	Attacks go unnoticed	Your logwatch build, SIEM week
A10	Server-Side Request Forgery	Server fetches a URL the attacker controls	PortSwigger SSRF labs

Where your DSA and CS work meets cyber

YOUR WORK	TEACHES	CYBER USE
Step 5 textkit	Password-strength checker	Why length beats complexity
Step 7 pinspace	Size of a brute-force search space	Why long passphrases defeat cracking
Step 8 perms	Unix permission bits, world-writable files	SUID hunting in privilege escalation
Step 10 logwatch	Brute-force detection with sliding windows	SOC alerting, rate limiting
Step 15 netscan	BFS reachability, single points of failure	Network mapping, lateral movement
Step 18 kgrep	Signature matching with KMP	How intrusion-detection systems match attacks
CS: CN month	TCP, DNS, TLS in depth	Packet analysis week (week 11)
CS: System design	Rate limiting, caching, CDNs	DDoS mitigation, WAFs

PART 06

06 Career skill maps

Required skills for SDE, network security and software security roles, and what to add.

PART 06 | THREE TARGETS

Which role, which skills

	SDE	NETWORK SECURITY	SOFTWARE SECURITY
DSA and coding	**Very high**	Low to medium	High at product companies
Core CS	OOP, DBMS, OS	**CN above all**	CN (web), OS, DBMS
Networking	Basics	**Deep**: routing, firewalls, VPN	HTTP, TLS, DNS in depth
Security	Basics	Network attacks, IDS, SIEM	**Deep**: OWASP, code review
Certifications	Rarely needed	**Valued**: CCNA, Security+	Optional: BSCP, Security+
Where you are now	Strongest after this plan	Needs routing and firewall labs	Close: add code review, threat modelling

A SENSIBLE STRATEGY

Apply to SDE and application security together; both reward your DSA, and the cyber hour gives you an edge. Keep network security as a parallel option and decide after your CN month.

CAREER | SKILL MAP

Software Development Engineer (SDE)

What the job is. Builds and maintains software features: writes code and tests, reviews other people's code, debugs production issues, and works with product managers on what to build

next.

Fresher titles. SDE-1, Software Engineer, Associate Software Engineer, Graduate Engineer Trainee (service companies).

SKILL	LEVEL	WHERE YOU LEARN IT
Data structures and algorithms through graphs and DP	MUST	Parts 01 to 03, hard pass
One language in depth (your C++), a second for scripting (Python)	MUST	Part 01; Python on your own
OOP and clean code	MUST	Part 04: OOP
DBMS and SQL	MUST	Part 04: DBMS
Operating systems and networking basics	MUST	Part 04: OS, CN
Git, GitHub, pull requests	MUST	Your `dsa-in-practice` repo
Writing tests and debugging	MUST	Interview kit, every build
One application stack: REST APIs and a database	MUST	**Gap**: see below
System design basics	GOOD	Part 04: System design
Cloud (AWS), Docker, Linux, CI/CD	GOOD	TenantWall, cyber weeks 1 to 4

Tools. VS Code, Git, GitHub, Linux shell, Postman, a database (PostgreSQL or MySQL), Docker, AWS.

Certifications. Not required. Projects and DSA count far more. AWS Cloud Practitioner or Solutions Architect Associate helps for cloud-heavy roles.

How hiring usually works

1. **Online assessment**: 2 or 3 DSA problems in 60 to 90 minutes, sometimes MCQs on OS, DBMS, CN and OOP.
2. **Technical rounds (1 or 2)**: DSA live coding, explaining complexity, then follow-ups.
3. **Fundamentals and project round**: core CS questions and a deep dive into one of your projects.
4. **Design round** at some product companies: low-level design of classes, or basic system design.
5. **HR or behavioural**: teamwork, a hard problem you solved, why this company.

> WHAT TO ADD TO THE PLAN

> Your builds are command-line tools. Add one web backend: a REST API for your placement tracker (problems, attempts, stats) with a real database, tests, and deployment on AWS. That one project covers APIs, SQL, testing and cloud together.

Portfolio projects

- ☐ REST API for the tracker: endpoints, SQL schema from Part 04, auth, tests, deployed.
- ☐ TenantWall written up with an architecture diagram.
- ☐ Two of your best DSA builds polished with READMEs.

Resume keywords. `C++, Python, Data Structures, Algorithms, OOP, SQL, REST APIs, Git, Linux, AWS, Docker, Unit Testing`

CAREER | SKILL MAP

Network Security Engineer

What the job is. Designs, secures and monitors networks: firewall rules, VPNs, segmentation, intrusion detection, and investigating suspicious traffic. Early-career work is often in a SOC or NOC, triaging alerts and changes.

Fresher titles. Network Security Engineer (L1), SOC Analyst L1, NOC Engineer, Security Analyst, Network Engineer Trainee.

SKILL	LEVEL	WHERE YOU LEARN IT
TCP/IP in depth: OSI, handshakes, DNS, DHCP, ARP	MUST	Part 04: CN; cyber week 3
Subnetting and CIDR on paper, fast	MUST	Part 04: CN week 2, subnet calculator task
Routing and switching: VLANs, static routes, OSPF basics, ACLs	MUST	**Gap**: Packet Tracer labs
Firewalls: stateful rules, NAT, zones	MUST	**Gap**: iptables and AWS security groups
VPNs: IPsec and SSL/TLS VPN concepts	MUST	**Gap**: read and lab
IDS/IPS: Snort or Suricata rules	MUST	Step 18 kgrep idea; **gap** for a real IDS
Packet analysis with Wireshark and tcpdump	MUST	Cyber week 11

SKILL	LEVEL	WHERE YOU LEARN IT
Scanning and enumeration with nmap	MUST	Cyber week 4, Step 15 netscan
Logs and SIEM, incident response	MUST	Cyber weeks 14 to 16, Step 10 logwatch
Linux administration	MUST	Cyber weeks 1, 2, 13
Scripting (Python or bash) for automation	GOOD	Your C++ builds; add Python
Cloud networking: VPC, security groups, NACLs	GOOD	TenantWall, cyber week 17

Tools. Wireshark, tcpdump, nmap, Cisco Packet Tracer, iptables or nftables, Suricata, Wazuh or Splunk, AWS VPC.

Certifications. Matter more here than for SDE. A common path: Cisco CCNA (networking) or CompTIA Network+, then CompTIA Security+. Vendor firewall certifications (Fortinet, Palo Alto) come later on the job.

How hiring usually works

1. **Aptitude or online test** at service companies and MSSPs, often with networking MCQs.
2. **Networking deep dive**: OSI layers, the TCP handshake, subnetting a /26 on paper, what a switch does vs a router.
3. **Security scenarios**: "a host is scanning the network, what do you do?", reading a firewall rule set, reading a packet capture.
4. **Tools round**: nmap flags, Wireshark filters, a SIEM search.
5. **HR**: shift work and on-call are common in SOC and NOC roles; be ready to discuss them.

> WHAT TO ADD TO THE PLAN
>
> Three things the plan does not yet cover: routing and switching hands-on, firewall configuration, and a real IDS. Use Cisco Packet Tracer (free through Cisco Networking Academy, runs on macOS) for VLANs and routing; write iptables rules in your Kali VM; run Suricata in the VM and watch it alert on your own nmap scan. Fit these into cyber weeks 18 to 20, or after March.

Portfolio projects

- ☐ Home-lab network diagram in Packet Tracer: 3 VLANs, a router, ACLs, documented.
- ☐ Suricata in a VM detecting your own nmap scan, with the alert and the rule explained.
- ☐ AWS VPC with public and private subnets, security groups and NACLs, written up.

Resume keywords. `TCP/IP, Subnetting, Routing and Switching, VLAN, Firewall, VPN, IDS/IPS, Suricata, Wireshark, Nmap, SIEM, Linux, Incident Response`

CAREER | SKILL MAP

Software Security Engineer (Application Security)

What the job is. Makes software secure before and after it ships: reviews code for vulnerabilities, threat-models new features, tests web apps and APIs, runs security scanners in CI/CD, and helps developers fix what is found.

Fresher titles. Application Security Engineer, Product Security Engineer (Associate), Security Engineer, VAPT Analyst, DevSecOps Engineer (junior).

SKILL	LEVEL	WHERE YOU LEARN IT
Strong programming: read and write real code in 2 languages	MUST	Part 01 C++; add Python or JavaScript
DSA (product companies still test it)	MUST	Parts 01 to 03
Web fundamentals: HTTP, cookies, sessions, CORS, same-origin policy	MUST	Part 04: CN week 4; cyber week 5
OWASP Top 10 and the OWASP API Security Top 10	MUST	Cyber weeks 5 to 9
Authentication: sessions, OAuth 2.0, JWT and their pitfalls	MUST	Cyber week 10; **gap** for OAuth
Applied cryptography: password hashing (bcrypt, Argon2), TLS	MUST	Cyber week 10
Secure code review	MUST	**Gap**: exercise below
Threat modelling (STRIDE)	MUST	**Gap**: threat-model TenantWall
Burp Suite and manual web testing	MUST	Cyber weeks 5 to 9
Memory-safety bugs in C and C++	GOOD	Your C++; **gap**: buffer overflows
SAST and DAST tools in CI/CD	GOOD	**Gap**: Semgrep in GitHub Actions
Cloud and container security	GOOD	Cyber week 17, TenantWall

Tools. Burp Suite, OWASP ZAP, Semgrep, CodeQL, Trivy, gitleaks, GitHub Actions, sqlmap (labs only).

Certifications. Optional; skill shows best through findings and writeups. PortSwigger's Burp Suite Certified Practitioner suits this role; CompTIA Security+ is a general starting point; OSCP is a later, harder goal.

How hiring usually works

1. **Online assessment**: DSA problems, sometimes security MCQs.
2. **Web security deep dive**: XSS vs CSRF, how to fix SQL injection, what SameSite cookies do, how JWTs fail.
3. **Code review exercise**: find the vulnerabilities in a snippet and fix them (practise with the one below).
4. **Threat modelling**: "here is a feature, what could go wrong?"
5. **Project and CTF discussion**, then HR.

> WHAT TO ADD TO THE PLAN
>
> Practise secure code review weekly, threat-model one real system (TenantWall is ideal), and add Semgrep to a GitHub repo through GitHub Actions so every push is scanned. A few bug bounty attempts on in-scope programs (HackerOne or Bugcrowd, following their rules) are a strong signal later.

Portfolio projects

- ☐ TenantWall threat model (STRIDE) with the fixes you made.
- ☐ A deliberately vulnerable mini web app plus a fixed version, with a writeup of each bug.
- ☐ A GitHub repo with Semgrep and gitleaks running in CI, findings triaged.

Resume keywords. `Application Security, OWASP Top 10, Secure Code Review, Threat Modelling, Burp Suite, SAST, DAST, OAuth, JWT, Python, C++, CI/CD`

CAREER | PRACTICE

Secure code review drill

REVIEW_ME.CPP Copy

```cpp
// Exercise: find three security bugs before reading the answers below.
void login(const char *user, const char *pass) {
    char buf[16];
    strcpy(buf, user);                                  // (1)
```

```
    string q = "SELECT * FROM users WHERE name = '" + string(user) +
               "' AND password = '" + string(pass) + "'";      // (2) and (3)
    db.exec(q);
}

// Exercise: find
three security bugs
```

▼ Answers

1. **Buffer overflow.** `strcpy` into a 16-byte buffer with no length check. Use `std::string` or a bounded copy.
2. **SQL injection.** The query is built by concatenation. Use parameterised queries.
3. **Plaintext passwords.** Store a slow salted hash (bcrypt or Argon2) and compare hashes; rate-limit logins.

PART 07

07 Off-campus internships

Programs open to you, what real off-campus interns were asked, and how to apply.

PART 07 | OFF-CAMPUS INTERNSHIPS

Getting a summer 2027 internship

You are in semester 5, so your target is a **summer 2027 internship** (the summer before your final year). Many companies hire for next summer between roughly July and January, so the window is open now. Your college's internship rules came up earlier: confirm with the T&P cell whether a summer internship needs an NOC, and ask for it early.

> WHAT THIS PART IS BASED ON
>
> I could not access RAIT's own alumni records, so the selection stories here are published off-campus interview write-ups by other students, with their sources. They show what companies actually asked. To add your own college's alumni, use the method and the form at the end of this part. Program details change every year: confirm eligibility and dates on each company's official page before applying.

PROGRAM	WHO IT IS FOR	WHERE	WHAT IT TESTS	NOTE
Amazon SDE Intern	Penultimate-year B.E./B.Tech (graduation year varies by cycle)	amazon.jobs, off-campus portal	Online assessment (MCQs plus 2 coding questions on some cycles), then 2 or 3 DSA interviews; Leadership Principles questions	Listings ask for data structures, algorithms, OOP, OS and one of C/C++, Java, Python.
Google SWE Intern	Penultimate-year students	careers.google.com	Online challenge on some cycles, then 2 technical interviews (DSA)	STEP is a separate program for first and second years, historically focused on women and students with disabilities in India; not your track now.
Microsoft SWE Intern	Penultimate-year students	careers.microsoft.com, off-campus drives	Online assessment (2 or 3 coding questions), then technical interviews	A published tier-3 attempt failed partly because DSA prep started only a month before the test.
Flipkart GRiD	Engineering students, all years per most editions	Unstop (hackathon / case competition)	Online quiz or coding test, then case-study rounds; finalists get a PPI for an internship or full-time role	Third-year finalists are reported to get summer SDE internship interviews.

PROGRAM	WHO IT IS FOR	WHERE	WHAT IT TESTS	NOTE
JPMorgan Code for Good	Bachelor's students (2nd and 3rd year tracks in past editions)	JPMorgan careers site	Online coding test (HackerRank), virtual interview, then a 24-hour team hackathon for NGOs	Participation may lead to consideration for the Software Engineer Program summer internship.
Myntra HackerRamp / RampUp	Engineering students (check the year)	Unstop	Screening, then DSA, core CS and a close look at your resume	A 2025 off-campus selection came entirely through Unstop.
Startups (fintech, edtech, SaaS)	Anyone who can ship code	LinkedIn, Wellfound, Internshala, company sites, cold email	Often no OA: a call, then technical rounds on DSA, APIs, databases and your past projects; sometimes a take-home project	Faster process (days, not months). Real projects matter more than the college name.
Google Summer of Code	18+, any country not under embargo; enrolment not required	summerofcode.withgoogle.com	A project proposal to an open-source organisation, plus your earlier contributions	Paid open-source work over the summer; stipend depends on project size and location. A strong resume signal.
Amazon ML Summer School	Engineering students in India	Amazon's announcement page	An online test; top performers are shortlisted	Learning program, not an internship;

PROGRAM	WHO IT IS FOR	WHERE	WHAT IT TESTS	NOTE
	(graduation years vary)		for free online ML classes	no interview is promised.
Security internships	Varies	IIT Madras CyStar; AICTE virtual cyber internship; I4C (MHA); company SOC or AppSec intern roles	CyStar asks for OS knowledge, cybersecurity projects and a public portfolio; I4C's 2025 round was limited to NIRF top institutions	AICTE's virtual internship is free and online. Most security interns are hired on labs, CTF results and a portfolio.

PART 07 | WHAT SELECTED INTERNS FACED

Real off-campus selection stories

Four selections and two misses, each mapped to your plan.

Amazon SDE Intern: selected

Source: GeeksforGeeks, updated Jul 2025

- Online test on Mettl: 28 MCQs on data structures, algorithms and code output, plus 2 coding questions.
- Two face-to-face problem-solving rounds, back to back; HR confirmed the offer minutes after the last one.

What they were asked

- Euler's totient function
- Count inversions in an array
- Best time to buy and sell stock
- Count non-overlapping occurrences of a pattern as a subsequence
- Connect N ropes with minimum cost (spot that it is a heap problem)
- Steps between two nodes in a directed graph (BFS by levels)

> LESSON
>
> Explain your approach before coding; interviewers hint and then ask you to optimise; write clean code on paper. Steps 3, 8, 11 and 15 of your A2Z sheet cover every question here.

INDmoney SDE Intern (6 months): selected

Source: GeeksforGeeks

- A recruiter called about the candidate's earlier internship; the first round was the next day, with no online test.
- Three technical rounds, the last two with the Director of Engineering and the CTO.

What they were asked

- Producer-consumer problem with a real-life analogy
- Architecture and API flow of an e-commerce order
- ER model for a student report card
- Settle debts among n people with the fewest transactions (greedy)
- DBMS indexing; normalise tables to 3NF; HTTP status codes
- Sum of distances between all node pairs in a binary tree (DFS, LCA)
- A spiral-matrix variation

> LESSON
>
> Never stretch your resume; ask clarifying questions; stay confident when the interviewer gives no feedback. Core CS (OS, DBMS, system design basics) mattered as much as DSA. Steps 12 and 13 plus Part 04.

Scaler SDE Intern: selected

Source: GeeksforGeeks

- Applied on the company's careers portal; HR called two days later.
- Online test: 2 easy coding questions plus MCQs on JavaScript, SQL and MVC.
- Coding interview, then a take-home full-stack project, a development round reviewing it, a culture-fit round, and a hiring manager round. About 2 weeks in total.

What they were asked

- Spiral order of a matrix
- Questions on the candidate's current internship

- Defend the project's tech stack, schema and APIs; redesign an inefficient database schema on the spot

> LESSON
>
> Know your own project's database design deeply; be ready to change it when challenged. This is the SDE gap from Part 06: one real backend project with a database you can defend.

Intuit SDE Intern: reached interviews, not selected

Source: GeeksforGeeks, updated Jul 2025

- Resume shortlist, then a HackerRank test: 4 questions on arrays, strings and trees (1 easy, 2 medium, 1 hard).
- Technical round 1: projects, then CS fundamentals, then DSA and rapid-fire questions.

What they were asked

- DBMS vs file system; one SQL query written two ways
- OOP pillars, polymorphism, overloading vs overriding
- OS threads
- Reverse a doubly linked list
- Maximise the sum of K elements taken from the array's ends (sliding window)
- list vs vector; insertion and search cost in a binary tree vs a BST

> LESSON
>
> The first idea failed and the candidate switched to a sliding window; practise finding a counterexample fast. Know the costs of every data structure. Steps 6 and 10, and the Core CS Q&As in Part 04, are exactly this round.

Myntra RampUp SDE Intern via Unstop: selected (2025)

Source: Medium (summary from search results)

- Applied entirely off-campus through Unstop's open hiring; the first screening was competitive.
- Technical rounds on DSA, core CS fundamentals and a close review of the resume.

What they were asked

- DSA
- Core CS fundamentals

- Every line of the resume

> LESSON
>
> Unstop competitions are a real off-campus door, not just practice. Register on Unstop now and enter one hiring challenge a month.

Microsoft SDE Intern, tier-3 college in Pune: not selected

Source: Medium (summary from search results)

- Off-campus application in September of the penultimate year; online assessment followed.

What they were asked

- Online assessment with DSA problems

> LESSON
>
> The candidate had started DSA only about a month before the test. Starting early is the single biggest lever. You start in November with a full plan: you are months ahead of this candidate.

PART 07 | THE PATTERN

What every selected intern had

ASKED ABOUT	SEEN AT	WHERE YOUR PLAN COVERS IT
DSA: arrays, strings, sliding window	Amazon, Intuit, Scaler	Steps 3, 5, 10
DSA: heaps, greedy	Amazon, INDmoney	Steps 11, 12
DSA: trees and graphs	Amazon, INDmoney, Intuit	Steps 13 to 15
Count inversions, linked lists, matrices	Amazon, Intuit, Scaler, INDmoney	Steps 2, 3, 6
DBMS and SQL (indexing, 3NF, queries)	INDmoney, Intuit, Scaler	Part 04: DBMS
OOP and OS (threads, producer-consumer)	Intuit, INDmoney	Part 04: OOP, OS

ASKED ABOUT	SEEN AT	WHERE YOUR PLAN COVERS IT
APIs, HTTP, system design basics	INDmoney, Scaler	Part 04: CN, System design
Your own projects, defended in depth	All six	Builds, plus one web backend
Earlier internship or real work	INDmoney, Scaler	A first small internship or open source

Internship-ready checklist

By 31 Dec 2026

- ☐ A2Z core done (Easy and Medium)
- ☐ 60 to 75 LeetCode problems
- ☐ OOP and OS answered aloud
- ☐ One-page resume v1 with 3 builds
- ☐ GitHub with READMEs; LinkedIn complete

By 31 Jan 2027

- ☐ A web backend project (REST API plus database) deployed
- ☐ SQL: 25 problems
- ☐ First 30 applications sent; 2 Unstop challenges entered
- ☐ 5 alumni or senior conversations done

By 31 Mar 2027

- ☐ Hard pass under way; contest Q1 and Q2 reliably
- ☐ CN done; system design basics started
- ☐ 60 or more applications; referrals asked for the top 10 targets
- ☐ 8 cyber writeups public (for security roles)

PART 07 | HOW TO APPLY

Where to apply and how to ask

WHERE	WHAT YOU FIND	HOW TO USE IT
Company career pages	Amazon, Google, Microsoft, Atlassian, Intuit and others post internship roles directly	Set alerts; apply in the first week a role opens

WHERE	WHAT YOU FIND	HOW TO USE IT
LinkedIn Jobs	Startups and mid-size companies	Filter: Internship, India or Remote, past week
Unstop	Hiring challenges and hackathons (Flipkart GRiD, Myntra and others)	One challenge a month
Wellfound (AngelList)	Startups that hire fast	A short note about one project that matches their product
Internshala	Many small and remote internships	Good for a first internship; check the company is real
Hackathons	SIH, Code for Good, company hackathons	Finalists often get interviews; you already have SIH experience
Open source	GSoC and project contributors	Start contributing in January if you aim for GSoC
Referrals and cold email	Engineers at your target companies, especially RAIT alumni	Short, specific, with a link to your best project

WEEKLY RHYTHM FROM DECEMBER

10 applications a week, 3 referral or alumni messages, one Unstop challenge a month. Track every application: company, role, date, source, referral, stage, next step.

Your application tracker

COMPANY	ROLE	DATE	SOURCE	REFERRAL	STAGE	NEXT STEP	LAST CHANGE
No rows yet. Add your first one below.							

+ Add row Saving to this browser

▼ Change history (0)

REFERRAL MESSAGE Copy

```
Hi <name>, I'm <you>, a third-year <branch> student at RAIT. I saw you joined <company>
as <role> from RAIT and I'm preparing for the <role> internship (job ID <id>).

I've solved <N> DSA problems and built <one project, one line, with a link>.
Would you be open to referring me, or telling me what helped you most in the process?

Resume: <link>   Thanks for your time either way.  <you>
```

Hi <name>, I'm <you>, a third-year <branch>

COLD EMAIL TO A STARTUP

Copy

```
Subject: SDE intern, summer 2027: built <thing related to their product>

Hi <first name>,
I use <product> and built <small project> that <does X>, here: <link>.
I'm a third-year <branch> student at RAIT, strong in C++ and DSA (<N> problems), and I'd
like to intern on your engineering team for summer 2027 (May to July).

Could I send my resume, or is there someone better to ask?
<you>  |  GitHub <link>  |  LinkedIn <link>
```

Subject: SDE intern, summer 2027: built

PART 07 | YOUR OWN ALUMNI

Collect RAIT's real off-campus stories

1. On LinkedIn, search: `Ramrao Adik Institute of Technology` and filter People by Current company (Amazon, Microsoft, startups) and by keyword `intern`.
2. Also search `RAIT` with `SDE Intern`, `Security Intern`, `SOC Analyst` and the batches 2023 to 2026.
3. Ask the T&P cell and your department for the list of last year's off-campus internship offers (company, role, student).
4. Ask seniors in your college clubs and the SIH team; most will answer five questions over chat.
5. Message 5 people a week with the short questionnaire below; record answers in the table.

Five questions to ask

1. How did you apply: portal, referral, hackathon, cold email?
2. What were the rounds, and which questions do you remember?
3. Which project did they ask about most?
4. What did you study, and for how many months?
5. What would you do differently?

NAME, BATCH	COMPANY, ROLE	HOW APPLIED	ROUNDS AND QUESTIONS	PREP	ADVICE	LAST CHANGE

No rows yet. Add your first one below.

+ Add row

Saving to this browser

▼ Change history (0)

PART 08

08 Interview kit

What to reach for, how to talk, how to debug.

PART 08 | READ CONSTRAINTS FIRST

Constraints and pattern signals

CONSTRAINT	TARGET	TYPICAL APPROACH
n <= 10	O(n!) or O(n^6)	Permutations, brute force
n <= 20	O(2^n x n)	Subsets, bitmask, backtracking
n <= 100	O(n^4)	Small DP
n <= 500	O(n^3)	Floyd-Warshall, interval DP
n <= 5000	O(n^2)	Two loops, 2D DP
n <= 1e6	O(n log n) or O(n)	Sort, heap, binary search, two pointers, hashing
n <= 1e8	O(n) with a small constant	One pass, sieve
n > 1e9	O(log n) or O(1)	Maths, binary search on the answer, fast power

WHEN THE PROBLEM SAYS	REACH FOR
Sorted array, or "find position"	Binary search, two pointers
"Minimum maximum" or "smallest X such that"	Binary search on the answer
Subarray or substring, contiguous, with a condition	Sliding window (positives) or prefix sum + hash map
"Count pairs" or "has a pair" with a sum	Hash set, or sort + two pointers
Next greater or smaller, spans, histograms	Monotonic stack

WHEN THE PROBLEM SAYS	REACH FOR
Top K, Kth largest, merge K sorted	Heap
All combinations, subsets, permutations	Backtracking
Optimal value with choices that overlap	Dynamic programming
Intervals, meetings, scheduling	Sort + greedy, or heap
Grid, islands, spreading, shortest steps	BFS or DFS on a grid
Dependencies, ordering, prerequisites	Topological sort
Weighted shortest path	Dijkstra (non-negative), Bellman-Ford (negative)
Connected as edges arrive, grouping	Disjoint set union
Prefix lookups over many words	Trie
XOR, "appears once", powers of two	Bit manipulation
Pattern search in text	KMP, Z-function, rolling hash

PART 08 | LIVE INTERVIEWS

How to talk, test and debug

1. **Clarify**: input size, ranges, duplicates, negatives, empty input.
2. **Example**: walk one by hand out loud.
3. **Brute force** and its Big-O.
4. **Optimise**: name the pattern, state the new complexity.
5. **Code** while talking.
6. **Test**: dry-run the example and an edge case.
7. **Wrap up**: time, space, one follow-up.

Edge cases every time

- ☐ n = 0 or n = 1
- ☐ All elements equal
- ☐ Already sorted, or reverse sorted
- ☐ Negative numbers and zero
- ☐ Maximum values (overflow in sums and products)

- [ ] Duplicates
- [ ] Disconnected graph, self-loops
- [ ] Empty string, single character
- [ ] Target not present
- [ ] Answer at the first or last index

PART 09

09 Logs and reviews

The habits that make the plan work.

PART 09 | EVERY DAY

Daily checklist and college-day split

- [ ] Revise: re-solve every problem due today
- [ ] Striver: today's problems by due date
- [ ] Build: one feature, commit
- [ ] LeetCode: 2 topic + 1 recall
- [ ] Core CS: 45 minutes, 3 flashcards
- [ ] Cyber: one lab, timer on, stop at 60 minutes
- [ ] Log every problem: time, result, mistake
- [ ] Tick the tracker and push to GitHub

TIME	WEEKDAY (COLLEGE)	WEEKEND
Morning, 1 h 15	Revision + Core CS	Revision + Core CS, then 3 h Striver
After college, 2.5 h	Striver (2 h) + build (30 min)	Build (1.5 h)
Evening, 2 h	LeetCode, then log	LeetCode, or the Sunday contest
Night, max 1 h	Cyber lab with a timer	Cyber lab; weekly review on Sunday

Problem log

DATE	PROBLEM	STEP OR TAG	MIN	S/H/F	MISTAKE OR LESSON	REDO	LAST CHANGE

No rows yet. Add your first one below.

+ Add row Saving to this browser

▼ Change history (0)

WHERE TO TICK

Tick each problem in the DSA card on Today. Ticks earn XP and save to your account.

PART 09 | BACKUP

Back up and move your progress

Your ticks, XP, readiness checks and tracker rows are saved where the line under Today says. Export them to a file before your Claude plan ends, or whenever you want a backup. To move to another browser or to the offline copy, import that file there.

Export progress (.json) Copy as text

Import

Choose an exported file, or paste its text into the box. Importing replaces the progress saved here.

Choose File No file chosen

…or paste exported text here

Import

THE OFFLINE COPY

The offline copy is this whole playbook as one HTML file. It opens in any browser without Claude and saves progress in that browser. Open it on your laptop, import your exported

file, and carry on. Export now and then from there too, so a cleared browser never costs you your ticks.

PART 10

10 AI track

Two hours a day from 4 Jan: build with AI for SDE roles, then attack and defend it for security roles.

PART 10 | 2 HOURS A DAY, FROM 4 JAN

AI track: build it, then break it

Weeks 1 to 6 teach you to build with large language models, the AI skill SDE interviews now ask about. Weeks 7 to 10 teach you to attack and defend them, which joins this track to your cyber track. Weeks 11 and 12 turn it into portfolio work. Machine-learning maths and training models are left out on purpose: you do not need them for SDE or security roles.

TWO HOURS, THEN STOP

Start a timer, stop when it rings, write notes. On Sunday, use the time to finish the week's proof of work.

FREE ALL THE WAY

Everything here runs on free tools: local models with Ollama, free API tiers, free courses and labs. You do not need a paid plan. Keep API keys in environment variables and never push them to GitHub.

ONLY ATTACK WHAT YOU ARE ALLOWED TO

Practise prompt injection only on Gandalf, PortSwigger's labs and apps you built yourself.

MINUTES	WHAT
0 to 10	Read yesterday's notes and set today's goal.
10 to 100	Build or lab work toward that goal.

MINUTES	WHAT
100 to 120	Commit to GitHub, write notes, tick AI on Today.

AI | WEEK BY WEEK

The 12-week plan

WEEK	FOCUS	INSIDE THE 2 HOURS	PROOF OF WORK
W01 04 Jan to 10 Jan	**How LLMs work, and your first API call**	Andrej Karpathy's one-hour talk "Intro to Large Language Models" and 3Blue1Brown's videos on GPT and attention. Then get a free API key (for example Google AI Studio) or run a local model with Ollama, call it from a Python script, and keep the key in an environment variable.	One page on how LLMs work in your own words, and a script that summarises any text file with 3 temperature settings compared
W02 11 Jan to 17 Jan	**Prompting and structured output**	System prompts, examples in the prompt (few-shot), asking for JSON and validating it. Read one model provider's prompt-engineering guide.	A prompt that turns a job post into JSON (role, skills, stipend), tested on 10 real posts, with its pass rate
W03 18 Jan to 24 Jan	**Embeddings and search**	Embeddings and cosine similarity; splitting text into chunks; a free local embedding model (sentence-transformers) and a vector store (Chroma or FAISS).	Search over your own Core CS notes: 5 queries and what came back
W04 25 Jan to 31 Jan	**RAG: ask your notes**	Retrieve the best chunks, put them in the prompt, answer with citations, and say "not in my notes" when nothing matches.	An "Ask my notes" repo on GitHub with a README and 5 example questions
W05 01 Feb to 07 Feb	**Tool calling and a small agent**	Let the model call your own functions (search notes, a calculator, read a file). Log every step it takes.	A command-line assistant with 2 tools and a printed trace of one run
W06 08	**Ship an AI feature and**	Add one AI endpoint to your web backend project: input limits, timeouts, error	A working endpoint, a before-and-after score

WEEK	FOCUS	INSIDE THE 2 HOURS	PROOF OF WORK
Feb to 14 Feb	**test it**	handling, the key in an environment variable, cost and latency logged. Then write 20 test questions with expected answers, measure accuracy and fix the worst 3 failures.	table, and a resume line with one real number
W07 15 Feb to 21 Feb	**OWASP Top 10 for LLM apps**	Read all ten risks (prompt injection, sensitive information disclosure, system prompt leakage, excessive agency and the rest). Link each one to the web bug it resembles from cyber weeks 6 to 9.	A one-page map: LLM risk, its web cousin, and the fix
W08 22 Feb to 28 Feb	**Prompt injection, hands-on**	Lakera's Gandalf game (as many levels as you can); the "Web LLM attacks" labs in PortSwigger Web Security Academy.	A writeup of 2 labs, prompt by prompt
W09 01 Mar to 07 Mar	**Attack your own RAG app**	Plant a poisoned note (indirect injection), try to leak the system prompt, misuse a tool. Then fix it: keep instructions and data apart, give tools the least power they need, confirm before actions.	3 findings, each with its fix and a retest
W10 08 Mar to 14 Mar	**Reviewing AI-written code**	Ask a model for 3 small features (login, file upload, a SQL query). Review them for security bugs, name each bug's CWE, and run Semgrep.	Review notes: each bug, its CWE, and the fixed code
W11 15 Mar to 21 Mar	**Machine learning for intrusion detection**	Train a scikit-learn classifier for normal versus attack traffic on the NSL-KDD dataset, or for normal versus suspicious calls in TenantWall's CloudTrail logs. Measure precision, recall and false positives.	A notebook with a confusion matrix, and a paragraph on what a false positive costs a security team
W12 22 Mar to 28 Mar	**Capstone polish and interview prep**	Pick one project (the secured "Ask my notes" app or the intrusion-detection notebook): clean code, README, architecture diagram, a 2-minute demo. Answer 15 interview questions out loud (LLMs, embeddings, RAG, evaluation, prompt injection) and update your resume.	One portfolio-ready repo, an updated resume, and your answers recorded

AI | BY 28 MARCH

What you will have by March

- ☐ An "Ask my notes" RAG app on GitHub, with a README and a test score
- ☐ One AI feature live in your web backend project, with a resume line
- ☐ 4 AI-security writeups: Gandalf, PortSwigger LLM labs, your own app, AI-written code
- ☐ An intrusion-detection notebook with a confusion matrix
- ☐ 15 interview answers on LLMs, RAG and prompt injection, practised out loud

> WHERE TO TICK
>
> From 4 Jan an AI card appears on Today. Mark it done after your hour; it counts toward your streak and XP.

PART 11 | LATER: APRIL TO JULY 2027

11 HFT track: maths and probability

For a chance at high-frequency trading firms. Not now: it starts after this plan ends, and nothing here appears on Today.

PART 11 | WHY AND WHICH ROLE

What HFT firms test

Trading firms (for example Tower Research, Graviton, Quadeye and Optiver) hire freshers mostly into three roles, and their tests mix hard coding with maths:

ROLE	WHAT THEY TEST	YOUR FIT
Software engineer (low-latency C++)	Hard coding tests, C++ depth, operating systems, networks	**Best fit.** Your C++, DSA, CP31 and Core CS carry straight over.
Quant trader	Probability, mental maths speed, betting and market-making games	Possible with this part
Quant researcher	Deeper probability, statistics, research projects	A stretch; usually wants more maths than a CS degree gives

BE REALISTIC

These firms are very selective and often hire from strong competitive-programming profiles. Treat this as a stretch goal. The same practice also makes ordinary online assessments much easier, so it pays off either way.

Before April: keep the doors open

- ☐ Keep CP31 going and play Codeforces Div 2 rounds; aim for a 1600+ rating by the summer
- ☐ Keep your C++ sharp; these firms test it more deeply than most

HFT | 16 WEEKS, 1 TO 1.5 HOURS A DAY

The 16-week plan

WEEKS	FOCUS	INSIDE THE HOUR	PROOF OF WORK
W01–W04 05 Apr to 02 May	**Counting and basic probability**	Permutations and combinations, inclusion–exclusion, conditional probability, Bayes' rule. Blitzstein and Hwang, Introduction to Probability, chapters 1 and 2 (free online), with Harvard's Stat 110 lectures. 10 easy Brainstellar puzzles a week.	40 solved problems in a notebook, each with the idea in one line
W05–W08 03 May to 30 May	**Random variables and expected value**	Linearity of expectation, indicator variables, binomial, geometric and Poisson, variance. Dice, coin and card games: what is the fair price to play? First 20 problems of Mosteller's Fifty Challenging Problems in Probability.	20 expected-value problems solved without looking at answers
W09–W12 31 May to 27 Jun	**Interview-style probability and puzzles**	The probability and brain-teaser chapters of Zhou's A Practical Guide to Quantitative Finance Interviews (the "green book"). Markov chains and random walks at a basic level (gambler's ruin). Brainstellar medium puzzles.	A personal sheet of 30 patterns (symmetry, recursion on states, conditioning on the first step)
W13–W16 28 Jun to 25 Jul	**Timed practice and mocks**	Timed puzzle sets, Crack's Heard on the Street, and simple market-making games for trader roles. For SDE roles at these firms: low-latency C++ (move semantics, memory layout, cache behaviour, the basics of lock-free code).	Two timed mock tests a week, scored and logged

Every day, all 16 weeks

- ☐ 10 minutes of mental maths on the Zetamac arithmetic game: aim for 40+, then 50+
- ☐ Codeforces: one Div 2 round a week, then upsolve one problem

HFT | TOPIC CHECKLIST

Topics to cover

- ☐ Counting: permutations, combinations, stars and bars, inclusion–exclusion
- ☐ Conditional probability and Bayes' rule
- ☐ Random variables, expectation, linearity of expectation, indicator variables
- ☐ Distributions: uniform, binomial, geometric, Poisson, normal
- ☐ Variance and covariance
- ☐ Conditional expectation; "condition on the first step"
- ☐ Markov chains and random walks (gambler's ruin)
- ☐ Expected-value games and fair prices
- ☐ Number theory for coding tests: modular arithmetic, GCD, primes
- ☐ Mental arithmetic speed

RESOURCES

Free: Blitzstein and Hwang's Introduction to Probability and the Stat 110 lectures, Brainstellar, Zetamac, Codeforces, the CSES problem set. Books: Zhou's green book, Mosteller's Fifty Challenging Problems in Probability, Crack's Heard on the Street.

PART 12

12 TenantWall research

Your paper with the professor: four Saturdays to add what was asked, then submission.

PART 12 | SATURDAYS, 21 NOV TO 12 DEC, 3 HOURS

The professor's changes, one Saturday each

Saturday's three hours take LeetCode's slot. Each Saturday uses what that week's AWS block taught: week 1 is IAM, week 2 is monitoring.

DAY	WORK	PROOF
Sat 1 21 Nov	**Policies: Policy Generator and explicit deny** • Rebuild the bucket policy (Listing 3) in the AWS Policy Generator and compare it with yours • Run IAM Access Analyzer's policy check on all three policies and fix any warning • Write a short subsection on explicit versus implicit deny, and mark each test in Table II as one or the other	Policy Generator and Access Analyzer screenshots, plus the new subsection
Sat 2 28 Nov	**CloudTrail trail** • Create tenantwall-trail, with its logs in a separate bucket • Turn on management events, and S3 data events for tenantwallbucket only • Re-run tenantwall_tests.py and find each refusal in the logs	A screenshot of alpha_user's refused GetObject on beta/secret.txt in CloudTrail
Sat 3 05 Dec	**CloudWatch alarm** • Send the trail to CloudWatch Logs (log group tenantwall-cloudtrail) • A metric filter for refused calls, and an alarm at 3 or more in 5 minutes that emails the admin through SNS • A second filter for tag tampering: CreateTags, TagUser, UntagUser	The alarm email after a test run, and the alarm's graph
Sat 4 12 Dec	**Diagrams and paper update** • Redraw Fig. 1 with AWS icons in draw.io, adding CloudTrail, CloudWatch and SNS • Draw the method flowchart: set up, onboard a team, run the tests, log, alarm • Add a section "Layer 5: Audit and alerting", a detection column in Table III, and update the	The updated paper sent to the professor

DAY	WORK	PROOF
	limitations	

TENANTWALL | WHAT EACH REQUEST MEANS

The professor's requests, explained

CloudTrail

A record of every AWS call: who, what, when, from where, and whether it was allowed. Your paper uses only event history, which leaves out S3 object reads. A trail with S3 data events records alpha's refused read of beta/secret.txt.

CloudWatch

Watches CloudTrail's records. A metric filter counts refused calls, and an alarm emails the admin through SNS. TenantWall goes from "prevent" to "prevent and detect".

METRIC FILTER: REFUSED CALLS

```
{ ($.errorCode = "*UnauthorizedOperation") || ($.errorCode = "AccessDenied*")
}
```

Flowchart

An AWS-style architecture diagram (draw.io has the AWS icons), plus a flowchart of your method. Keep Fig. 2, the decision path; it is already a flowchart.

Policy Generator

AWS's free form that writes policy JSON: pick the policy type, Allow or Deny, the service, the actions, the ARN and the conditions. Generate the bucket policy with it, then check every policy with IAM Access Analyzer.

"Something explicit"

Most likely explicit deny: a Deny statement that beats every Allow. Implicit deny means no Allow matched. Your paper already uses both; show them clearly. Confirm with the professor that this is what was meant.

TENANTWALL | AFTER DECEMBER

From changes to a published paper

- ☐ Ask the professor where to submit (a college journal or a student conference) and the deadline
- ☐ Optional future work from Section XII: a ListBucket Deny on the bucket, one NotResource Deny for all teams, permission boundaries, Team labels on volumes
- ☐ A GitHub repo with the policies, scripts, diagrams and a README
- ☐ AI week 11 can use the trail's logs for anomaly detection
- ☐ Resume line: "TenantWall: ABAC isolation for teams in one AWS account; fixed policy count for any number of teams, 14 of 14 security tests passed, refusals logged and alerted."

Ticks and tracker rows sync through your claude.ai account, so phone and laptop share them.
Tick problems on Today.